

Dossier de Projet

SkillSwap

Plateforme communautaire d'échange de compétences

Présenté par

Jérémy Soriano

en vue de l'obtention du

Titre Professionnel Concepteur Développeur d'Applications

(Niveau 6 — RNCP)

École O'clock — Promotion Dublin

Soutenance : Mercredi 26 août 2026

Application en production : <https://skill-swap.fr>

Documentation technique (extension post-projet) : <https://skillswap-docs.vercel.app>

Sommaire

Sommaire	1
1. Introduction	4
1.1. Présentation du candidat	4
2. Compétences du référentiel CDA couvertes	5
2.1. AT1 — Développer une application sécurisée	5
2.1.1. CP1 — Installer et configurer son environnement de travail	5
2.1.2. CP2 — Développer des interfaces utilisateur	5
2.1.3. CP3 — Développer des composants métier	5
2.1.4. CP4 — Contribuer à la gestion d'un projet informatique	5
2.2. AT2 — Concevoir et développer une application sécurisée organisée en couches	5
2.2.1. CP5 — Analyser les besoins et maquetter une application	5
2.2.2. CP6 — Définir l'architecture logicielle d'une application	6
2.2.3. CP7 — Concevoir et mettre en place une base de données relationnelle	6
2.2.4. CP8 — Développer des composants d'accès aux données SQL et NoSQL	6
2.3. AT3 — Préparer le déploiement d'une application sécurisée	7
2.3.1. CP9 — Préparer et exécuter les plans de tests d'une application ...	7
2.3.2. CP10 — Préparer et documenter le déploiement d'une application ...	7
2.3.3. CP11 — Contribuer à la mise en production dans une démarche DevOps	7
3. Cahier des charges	8
3.1. Présentation du projet	8
3.2. Public cible	8
3.3. Besoins fonctionnels — le MVP	8
3.4. Évolutions envisagées et périmètre effectivement livré	8
3.5. Périmètre fonctionnel détaillé — user stories	9
3.6. Règles de gestion arbitrées en cadrage	9
3.7. Contraintes techniques et de compatibilité	9
4. Présentation du commanditaire et de l'équipe	11
4.1. O'clock — école de développement à distance synchrone	11
4.2. L'apothéose — dispositif simulant l'entreprise	11
4.3. L'équipe SkillSwap	11
4.4. Le projet	11
5. Gestion de projet	12
5.1. Cadre et organisation de l'équipe	12
5.2. Méthodologie : Scrum adapté à quatre semaines	12
5.3. Le sprint 0 : cadrage et conception	12
5.4. Déroulé des sprints	12
5.5. Ma contribution et sa progression	13
5.6. Outils et flux de travail	13
5.7. Anticipation des risques	13
6. Spécifications fonctionnelles	14
6.1. Contraintes du projet et livrables attendus	14

6.1.1.	Contraintes	14
6.1.2.	Livrables attendus	15
6.2.	Architecture logicielle du projet	16
6.2.1.	Écarts relevés lors de la reconstruction	23
6.2.2.	Communications inter-composants	24
6.3.	Maquettes et enchaînement des maquettes	24
6.3.1.	Arborescence de l'application	24
6.3.2.	Parcours utilisateur principal	26
6.3.3.	Écarts relevés lors de la reconstruction	28
6.4.	Modèle entités-associations et modèle physique de la base de données . .	28
6.4.1.	Domaines fonctionnels du modèle	30
6.5.	Script de création et de modification de la base de données	31
6.6.	Diagramme du comportement des fonctionnalités — cas d'utilisation	32
6.6.1.	Écarts relevés lors de la reconstruction	38
6.6.2.	Acteurs et cas d'utilisation principaux	39
6.7.	Diagramme du détail du cas d'utilisation le plus significatif	40
6.7.1.	Lecture du diagramme	41
7.	Spécifications techniques	43
7.1.	Stack technique	43
7.2.	Choix architecturaux — synthèse des ADRs	43
7.3.	Patterns transversaux	44
7.4.	Sécurité transversale	48
8.	Réalisations — Messagerie temps réel	50
8.1.	Captures d'écran d'interfaces utilisateur	51
8.1.1.	Liste des conversations — vue desktop	51
8.1.2.	Thread de message ouvert — vue desktop	52
8.1.3.	Dialogue d'évaluation après clôture	53
8.2.	Composants métier — orchestrateur <code>useMessaging</code> et optimistic UI	54
8.2.1.	Architecture des hooks — le pattern de composition	54
8.2.2.	Hook racine — composition	55
8.2.3.	Optimistic UI — gestion du <code>tempId</code> négatif	56
8.3.	Composants d'accès aux données — services Prisma	57
8.3.1.	Création atomique d'un message + mise à jour de la conversation .	57
8.3.2.	Pagination cursor-based des messages	58
8.3.3.	Schéma Prisma — extrait des modèles principaux	59
8.4.	Autres composants — Socket.IO server et middleware métier	59
8.4.1.	Authentification Socket.IO par cookie JWT	60
8.4.2.	Middleware métier <code>requireSimpleFollow</code> — gating de la création de conversation	61
8.4.3.	Modèle de rooms Socket.IO	61
8.4.4.	Diagramme de séquence — envoi d'un message	62
8.5.	Bilan technique de la section	63
8.5.1.	Dette technique assumée	64
9.	Éléments de sécurité	65
9.1.	Authentification du canal Socket.IO	65
9.2.	Contrôles d'accès appliqués à chaque event	65
9.3.	Défense en profondeur — front, back, base	66
9.4.	Dettes de sécurité assumées	66

10.	Plan de tests	69
10.1.	Stratégie décidée et stratégie réalisée	69
10.2.	Les tests d'intégration backend	69
10.3.	État d'exécution réel	70
10.4.	Absence de tests automatisés côté frontend	71
10.5.	La validation manuelle, méthode réellement employée	71
10.6.	Ce qui manque et ce que je mettrais en place	71
10.6.1.	Ce que cette dette a concrètement coûté	72
11.	Jeu d'essai	73
11.1.	Contexte d'exécution	73
11.2.	Étape 1 — Le lien de suivi, pré-requis du gating	73
11.3.	Étape 2 — Création de la conversation	74
11.4.	Étape 3 — Envoi du message et affichage optimiste	75
11.5.	Étape 4 — Réception temps réel par Bob	76
11.6.	Étape 5 — Réconciliation côté Alice	77
11.7.	Preuves en base de données	78
11.8.	Analyse des écarts	80
12.	Veille de sécurité	82
12.1.	Périmètre et constat	82
12.2.	Une vulnérabilité identifiée et traitée pendant le projet	82
12.3.	Mesures de sécurité retenues pendant le projet	82
12.4.	Analyse OWASP conduite sur l'application	82
12.5.	Ce que je mettrais en place	83
13.	Difficultés rencontrées et solutions	84
13.1.	La mise au point de l'environnement conteneurisé	84
13.2.	Le contrat d'interface entre le frontend et le backend	84
13.3.	Une revue de code à mi-parcours	84
13.4.	Le cycle de vie de la session	85
13.5.	Les tests d'intégration backend	85
13.6.	Ce que ces difficultés ont en commun	85
14.	Conclusion	86
14.1.	Bilan du projet	86
14.2.	Évolutions identifiées pour la V2	86
14.3.	Apprentissage personnel	86
15.	Lexique	88
16.	Annexes	92
16.1.	Annexe A — Schéma Prisma de la fonctionnalité messagerie	92
16.1.1.	Extraits SQL générés par Prisma — migrations clés	93
16.2.	Annexe B — Handler Socket.IO <code>message:send</code> (code complet)	94
16.3.	Annexe C — Orchestrateur frontend <code>useMessaging</code> (139 LOC, complet)	97
16.4.	Annexe D — Composants UI clés de la messagerie	100
16.4.1.	<code>MessageList.tsx</code> — affichage virtualisé du fil de discussion	100
16.4.2.	<code>MessageInput.tsx</code> — saisie utilisateur et envoi	101
16.5.	Annexe E — Plan de tests détaillé de la messagerie	103
16.6.	Annexe F — Maquettes Figma des écrans principaux	103
16.7.	Annexe G — Modèle de données détaillé par domaine	106
16.8.	Liens utiles	112
16.9.	Annexe H — Chemin d'exécution de <code>message:send</code>	112

1. Introduction

Ce dossier de projet présente SkillSwap, plateforme web d'échange de compétences entre particuliers, développée dans le cadre de l'apothéose finale du Titre Professionnel Concepteur Développeur d'Applications. Le projet a été porté pendant quatre semaines par une équipe de cinq apprenants de la promotion Dublin.

J'y ai tenu le rôle de Lead Front : architecture et développement des interfaces, conception des maquettes, chaîne de déploiement de production, et documentation technique du projet.

Le dossier suit la structure du plan-type : recensement des onze compétences professionnelles mobilisées (section 2), cadrage du besoin (section 3), présentation du commanditaire pédagogique et de l'équipe (section 4), méthodologie de gestion de projet (section 5), spécifications fonctionnelles et techniques (section 6 et section 7), description détaillée de la fonctionnalité représentative — la messagerie temps réel — sous l'angle des réalisations (section 8) et de la sécurité (section 9), plan de tests (section 10), jeu d'essai (section 11), démarche de veille (section 12), retour d'expérience sur les difficultés rencontrées (section 13), conclusion ouvrant sur les évolutions V2 (section 14), lexique (section 15), et annexes regroupant les extraits de code et artefacts visuels.

Un parti pris a guidé sa rédaction : chaque affirmation factuelle y est adossée à une source vérifiable dans le code livré. Les artefacts de modélisation ont été reconstruits de façon déterministe depuis la base réelle, les éléments produits après la période de projet sont explicitement datés, et les dettes techniques identifiées sont documentées plutôt que masquées.

Le projet est aujourd'hui en production sur skill-swap.fr.

1.1. Présentation du candidat

Mon parcours vers le développement est celui d'une reconversion. Après un BTS Management Commercial Opérationnel et un Bachelor Responsable Manager, j'ai exercé six ans en commerce et en management : conseiller client, puis responsable du développement de la livraison à domicile pour une enseigne de grande distribution, enfin commercial dans l'assurance. Ces années m'ont donné une lecture métier des problèmes techniques, l'habitude de chercher le besoin réel avant la solution, et le sens des priorités d'une activité.

C'est cette même exigence de compréhension du besoin qui m'a conduit vers l'ingénierie logicielle. J'ai intégré la formation Concepteur Développeur d'Applications d'O'clock pour acquérir des fondations solides : architecture, conception de bases de données, développement full-stack et mise en production. L'apothéose SkillSwap constitue l'aboutissement de ce cursus — un projet d'équipe mené de bout en bout, du cadrage au déploiement.

2. Compétences du référentiel CDA couvertes

2.1. AT1 — Développer une application sécurisée

2.1.1. CP1 — Installer et configurer son environnement de travail

L'équipe s'est dotée d'un outillage qualité (ESLint, Prettier, Husky avec hook de pre-commit), mis en place par un coéquipier. J'ai contribué à la configuration Docker de développement (`backend/Dockerfile.dev`, `docker-compose.dev.yml`) et porté les correctifs de setup (PR `fix-setup-docker`). → §5.1.

2.1.2. CP2 — Développer des interfaces utilisateur

J'ai développé les interfaces de la page d'accueil (89 %), de la recherche (`SearchPage`, 80 %), de la consultation de profil public (`ProfileFull`, `ProfileTeaser`, 100 %) et de la navigation (`DesktopNav` 100 %, `MobileNav` 92 %), dans une architecture Atomic Design. Les dialogues d'édition de profil et le formulaire d'authentification ont été portés par des coéquipiers. → §6.3, annexes.

2.1.3. CP3 — Développer des composants métier

J'ai développé plusieurs hooks métier React : la gestion d'état de formulaire (`useFormState`, 100 %), la consommation de l'API de recherche avec debounce, pagination et annulation par `AbortController` (`useSearch`, 88 %), et la récupération des membres suivis (`useFollowedUsers`, 86 %). L'orchestration de la messagerie temps réel (`useMessaging` et les hooks socket) a été portée par un coéquipier. → §6, annexes.

2.1.4. CP4 — Contribuer à la gestion d'un projet informatique

J'ai travaillé en flux de Pull Requests sur branches de feature — six branches portées (SEO, page profil, recherche, setup Docker), treize Pull Requests intégrées, deuxième contributeur en volume de commits — au sein d'une organisation Scrum d'équipe (quatre sprints, rituels agiles, backlog Trello). → §5.

2.2. AT2 — Concevoir et développer une application sécurisée organisée en couches

2.2.1. CP5 — Analyser les besoins et maquetter une application

J'ai formalisé les cas d'utilisation et le parcours utilisateur de la plateforme en sprint 0 (`docs/uml/user/use-cases.puml`, `user-flow.puml`), et j'ai conduit une démarche d'analyse de besoin écrite pour la fonctionnalité de profil public : contexte, problématique, stratégie et spécification fonctionnelle formalisés avant l'implémentation (`frontend/src/.code-review/profil-teaser-strategy.md`, 601 lignes). Cette analyse débouche directement sur le composant d'accès aux données décrit en CP8, ce qui matérialise une chaîne complète besoin → spécification → implémentation. → détail §6.3, §6.6.

2.2.2. CP6 — Définir l'architecture logicielle d'une application

J'ai conçu la chaîne de déploiement de production (reverse proxy Nginx, `docker-compose.prod`, `Dockerfile.prod` multi-étapes) — `devops/` à 76 %, dont `nginx/prod.conf` et `Dockerfile.prod` quasi intégralement. Côté applicatif, j'ai contribué à l'implémentation de l'architecture frontend en Atomic Design. Concernant l'architecture backend en couches (router → controller → service → ORM), je ne l'ai pas implémentée, mais je l'ai analysée et outillée : j'ai vérifié mécaniquement le respect du sens de dépendance (`dependency-cruiser`, 4 règles, 0 violation) et documenté les cinq accès Prisma hors couche service — dont le principal, `realtime/socket.ts`, qui persiste les messages directement via `prisma.message.create` (`socket.ts:245`) au lieu de passer par `message.service.ts`, dupliquant la logique du chemin REST ; j'ai aussi relevé que cette persistance est un `Promise.all` (`socket.ts:244`) et non une transaction. → détail §6.2, §7.

2.2.3. CP7 — Concevoir et mettre en place une base de données relationnelle

J'ai produit le modèle entité-relation initial de la plateforme en sprint 0 (`docs/uml/erd.puml`), antérieur à l'implémentation du schéma, et j'ai contribué directement au schéma en ajoutant le champ `slug` à l'entité `Category` (migration `add_category_slug`). Au-delà de cette contribution d'écriture, je maîtrise l'intégralité de la modélisation, que j'ai reconstruite et vérifiée de façon déterministe depuis la base de production : MCD Merise (notation Mocodo), MLD (14 tables, DBML), MPD (SQL des six migrations) et ERD. Cette modélisation couvre la normalisation jusqu'en 3NF — avec une dénormalisation assumée de `message.receiver_id` pour la diffusion temps réel —, les clés composites des tables de jonction, les contraintes d'unicité par paire (`follow`, `evaluation`), et la distinction entre contraintes d'intégrité (FK NOT NULL) et contraintes applicatives (la règle des deux participants d'une conversation, désormais (0,2)). → détail §6.4 et §6.5.

2.2.4. CP8 — Développer des composants d'accès aux données SQL et NoSQL

J'ai développé le composant d'accès aux données du profil public (`getPublicProfileService`, `profile.service.ts:20-88`, ainsi que la tranche verticale route → service → rendu SSR → composant, 100 %). Ce composant met en œuvre une projection maîtrisée : un `select` racine qui énumère explicitement les colonnes remontées (`:31, :33`), et un chargement ciblé des relations — compétences avec leur catégorie via un `include` imbriqué (`:40-44`), évaluations restreintes au seul champ `score` (`:45-49`). S'y ajoutent une agrégation applicative (moyenne des avis arrondie au dixième, `:61-65`), et une minimisation des données exposées (description tronquée `:69-70`, nom réduit à l'initiale `:78`). J'ai également développé le composant frontend de consommation de l'API de recherche (`hooks/useSearch.ts`, 88 % : debounce, pagination, annulation par `AbortController`). L'accès aux données NoSQL (Meilisearch) a été porté par le Lead Back. → détail §8.3.

2.3. AT3 — Préparer le déploiement d'une application sécurisée

2.3.1. CP9 — Préparer et exécuter les plans de tests d'une application

L'équipe a mis en place sept suites de tests d'intégration backend exécutées par le Node Test Runner natif, écrites par trois coéquipiers. Le frontend, dont j'étais le principal contributeur, n'a pas été couvert par des tests automatisés — c'est la première dette que j'identifie, et la stratégie de test frontend (Vitest, Playwright) reste à l'état de décision documentée. → §10.

2.3.2. CP10 — Préparer et documenter le déploiement d'une application

J'ai conçu la chaîne de déploiement de production : images Docker multi-étapes (`Dockerfile.prod` front 100 %, back 86 %), orchestration `docker-compose.prod` (82 %), variables d'environnement (`.env.prod.example`, 100 %), le tout documenté dans `devops/README.md` (92 %, 323 lignes). → §7, dépôt `devops/`.

2.3.3. CP11 — Contribuer à la mise en production dans une démarche DevOps

J'ai conçu la chaîne de mise en production : reverse proxy Nginx avec TLS, HSTS et en-têtes de sécurité (`nginx/prod.conf`, 99 %), orchestration Docker Compose et images multi-étapes. L'automatisation du déploiement continu (pipeline CD) et l'exécution de la mise en production réelle ont été portées par deux coéquipiers. → §7.

3. Cahier des charges

Le cadrage du projet a été formalisé dans un brief d'équipe, complété d'un tableur de user stories et d'un journal des questions de conception¹. La présente section en restitue le contenu.

3.1. Présentation du projet

SkillSwap est une plateforme web d'échange de compétences entre particuliers. Chaque utilisateur peut proposer ses savoir-faire et bénéficier de ceux des autres, favorisant l'entraide et le partage de connaissances. La plateforme permet de créer un profil proposant ses services grâce à ses compétences, de rechercher d'autres profils ayant une compétence souhaitée, et une mise en relation via une messagerie intégrée. Elle repose sur une architecture moderne avec frontend et backend séparés, conteneurisée avec Docker pour garantir portabilité, sécurité et scalabilité.

3.2. Public cible

Des particuliers âgés de 18 à 60 ans, à l'aise avec les outils numériques, intéressés par l'entraide et l'économie collaborative. Ils cherchent à acquérir de nouvelles compétences ou à résoudre des besoins ponctuels (bricolage, informatique, jardinage, cuisine) sans passer par des services professionnels payants ; en échange, ils sont prêts à partager leurs propres compétences. Exemples : un développeur ayant besoin d'une réparation automobile, un jardinier souhaitant rénover sa salle de bain, un bricoleur désireux d'apprendre à cuisiner.

3.3. Besoins fonctionnels — le MVP

- Landing page avec la présentation de SkillSwap et quelques profils.
- Système d'inscription et de connexion.
- Gestion du profil utilisateur détaillé avec ses compétences, intérêts et disponibilités.
- Moteur de recherche par compétences afin de trouver de potentiels profils correspondants.
- Possibilité de suivre / ne plus suivre un profil.
- Possibilité de rentrer en contact avec un profil pour entamer un échange sur les compétences mutuelles.
- Possibilité d'évaluer un partenaire après échange de compétences.

3.4. Évolutions envisagées et périmètre effectivement livré

Le cadrage identifiait plusieurs évolutions possibles au-delà du MVP : la messagerie instantanée via WebSockets, la recherche avancée avec Meilisearch, le blocage d'un profil, la suggestion automatique de profils compatibles et la création de groupes ou communautés. Deux d'entre elles ont finalement été livrées dans le périmètre du projet : la messagerie temps réel (Socket.IO) et le moteur de recherche Meilisearch. Les autres

¹Documents produits hors dépôt Git, comme le journal de bord mentionné en section 5. Leur production est attestée par la checklist de conception validée par l'équipe pédagogique, qui coche notamment « user stories » et « dictionnaire de données ».

— blocage de profil, suggestions automatiques, groupes — n'ont pas été implémentées et restent des perspectives d'évolution.

3.5. Périmètre fonctionnel détaillé — user stories

Acteur	« Je souhaite pouvoir ... afin de ... »
Visiteur	Accéder à la landing page, afin de prévisualiser le fonctionnement du site.
Visiteur	Accéder au formulaire de création de compte.
Visiteur	Accéder au formulaire de connexion.
Membre	Accéder à la page de recherche des profils.
Membre	Accéder à un profil particulier.
Membre	Contacter un autre membre via la page de profil, afin d'initier une conversation.
Membre	Changer mon avatar, gérer ma description, mes compétences, mes intérêts et mes disponibilités, afin de mettre à jour mon profil.
Membre	Consulter les disponibilités des autres membres.
Membre	Accéder à ma messagerie.
Membre	Envoyer un message.
Membre	Supprimer un message.
Membre	Modifier un message. <i>Absent du cadrage initial — fonctionnalité livrée au-delà du périmètre prévu</i> (PATCH /conversations/:id/message/:messageId , conv.router.ts:72-79).
Membre	Clore une conversation.
Membre	Suivre / ne plus suivre un membre.
Membre	Évaluer un autre membre à la fermeture d'une conversation.
Membre	Effectuer une recherche et filtrer par catégorie.
Membre	Me déconnecter.
Membre	Supprimer mon compte.

3.6. Règles de gestion arbitrées en cadrage

Plusieurs règles ont été tranchées collectivement : la mise en relation n'est ouverte qu'entre membres liés par un suivi ; l'évaluation ne devient disponible qu'à la clôture d'une conversation, chaque conversation portant un statut ; les disponibilités sont saisies par créneaux prédéfinis (jour × plage) plutôt que par calendrier libre. La fonctionnalité « souhaite apprendre » envisagée initialement a été retirée du périmètre.

3.7. Contraintes techniques et de compatibilité

Le cadrage fixait la pile technique et sa justification : Docker (cohérence des environnements), Node.js/Express (performances asynchrones, framework léger), Next.js (SEO

via SSR/SSG), shadcn/ui + Tailwind (composants accessibles), PostgreSQL (base relationnelle robuste), Nginx (reverse proxy, SSL), Prisma (ORM type-safe), Meilisearch (recherche tolérante aux fautes), Socket.IO (temps réel). La compatibilité visée couvrait Chrome, Firefox, Edge et Safari, en desktop et mobile, dans leurs versions récentes.

Les choix de cette pile et leur mise en œuvre effective sont détaillés en section 7.

4. Présentation du commanditaire et de l'équipe

4.1. O'clock — école de développement à distance synchrone

Le projet a été réalisé dans le cadre de l'apothéose finale du Titre Professionnel Concepteur Développeur d'Applications (niveau 6, inscrit au RNCP), formation dispensée par O'clock — école de développement web et logiciel structurée autour d'un modèle pédagogique de téléprésentiel : les cours sont assurés en visio en temps réel par des formateurs. Notre promotion a été nommée Dublin.

4.2. L'apothéose — dispositif simulant l'entreprise

L'apothéose constitue la mise en situation professionnelle finale du cursus. Elle réunit un groupe d'apprenants pendant quatre semaines pour livrer un projet d'envergure depuis le cadrage jusqu'à la mise en production, en appliquant la méthodologie agile et l'ensemble des compétences techniques acquises. O'clock joue ici le rôle de commanditaire pédagogique : l'école définit le périmètre général — un projet web complet dont la fonctionnalité principale doit être d'envergure —, valide les choix techniques, et accompagne l'équipe via des points hebdomadaires avec un formateur référent.

4.3. L'équipe SkillSwap

Cinq développeurs apprenants, sur des rôles formalisés en sprint 0 et conservés tout au long du projet :

Membre	Rôle
Sébastien	Product Owner
Loïc	Scrum Master
Yorgan	Lead Back
Jérémy Soriano	Lead Front
Antoine	Frontend

La répartition détaillée des contributions, les rituels tenus et les outils de collaboration — Discord pour les sessions synchrones, Slack, Trello et un Drive partagé — sont décrits en section 5.

4.4. Le projet

L'équipe a porté SkillSwap, plateforme web d'échange de compétences entre particuliers. Le périmètre fonctionnel cible et les contraintes techniques structurantes sont détaillés au section 3. Le projet est aujourd'hui en production sous le nom de domaine skill-swap.fr.

5. Gestion de projet

5.1. Cadre et organisation de l'équipe

Le projet a été conduit par une équipe de cinq personnes en distanciel synchrone, avec des rôles distribués dès le sprint de cadrage : Sébastien (Product Owner), Loïc (Scrum Master), Yorgan (Lead Back), moi-même (Lead Front) et Antoine (Frontend). La coordination quotidienne s'est faite par salons vocaux Discord, avec des sessions de pair-programming ponctuelles.

5.2. Méthodologie : Scrum adapté à quatre semaines

Le projet s'est déroulé du 6 au 29 janvier 2026, soit quatre semaines découpées en quatre sprints d'une semaine. Le sprint 0 a été entièrement consacré au cadrage et à la conception ; les trois suivants au développement incrémental. Les rituels ont été adaptés au format distanciel : un point d'équipe chaque matin pour débriefer l'avancement et répartir la journée, puis une disponibilité continue en salon vocal permettant l'entraide et les décisions au fil de l'eau.

5.3. Le sprint 0 : cadrage et conception

La première semaine a produit l'ensemble des artefacts de conception avant toute ligne de code applicatif : présentation et clarification du cahier des charges, planification et attribution des rôles (lundi) ; user stories, ERD, MCD, dictionnaire de données et charte graphique (mardi) ; MPD, cas d'utilisation, diagramme de séquence, diagramme d'architecture et contrat d'endpoints (mercredi). La réalisation des maquettes m'a été attribuée² ; Sébastien et Yorgan ont pris les endpoints et la matrice RBAC, Loïc l'initialisation Docker. La semaine s'est close par un débrief et la planification du sprint 1.

5.4. Déroulé des sprints

Sprint	Semaine	Commits	PR	Contenu principal
0	6–10 jan.	35	10	Cadrage, conception, initialisation
1	12–18 jan.	148	49	Authentification, landing page, Docker, pages profil et conversation
2	19–25 jan.	129	41	Follow/unfollow, édition de profil, recherche Meiliseach, Socket.IO
3	26–29 jan.	54	30	Messagerie temps réel, mise en production, documentation

Commits hors *merge* et Pull Requests mergées, comptés par semaine ISO sur le dépôt d'équipe. Le travail s'est poursuivi certains week-ends : les bornes de la colonne « Semaine » couvrent la semaine complète, week-end inclus.

²Répartition consignée dans le document de planification du sprint 0 de l'équipe (tableur partagé, hors dépôt Git), colonne du jeudi 8 janvier 2026.

5.5. Ma contribution et sa progression

J'ai porté six branches de feature et treize Pull Requests intégrées, en deuxième position sur le volume de commits de l'équipe. Ma contribution suit une montée en charge nette : quatrième contributeur au sprint 0 (3 commits, la semaine étant consacrée à la conception), je deviens deuxième au sprint 1 (41) puis premier contributeur au sprint 2 (54 commits). Le journal de bord tenu par l'équipe pendant le projet³ retrace ce parcours : mise en place de l'architecture UI et des composants atomiques, landing page complète, configuration Docker de développement et de production, page profil, feature follow/unfollow, revue et refactorisation du frontend, puis conception et implémentation de la page de recherche en cohérence avec le backend Meilisearch.

5.6. Outils et flux de travail

Quatre outils ont structuré la collaboration : **Discord** pour les sessions synchrones et le pair-programming, **Slack** pour les échanges écrits, **Trello** pour le backlog⁴ et un **Drive partagé** pour les documents de cadrage — brief projet, tableur des user stories, journal de bord.

Le flux Git reposait sur une stratégie `main` / `dev` / branches de feature, avec intégration par Pull Request et branches courtes. La qualité de code était outillée par ESLint, Prettier et un hook de *pre-commit* Husky.

5.7. Anticipation des risques

Une analyse de risques a été conduite, identifiant douze risques répartis en cinq familles — techniques, développement, organisationnels, UX/sécurité et déploiement — chacun assorti d'une mitigation. J'ai par ailleurs formalisé une matrice de risques dans la documentation Arc42 du projet (délais, périmètre, performance de la recherche, sécurité des jetons), avec probabilité, impact, mitigation et plan de contingence.

³Tableur partagé, hors dépôt Git — comme les autres documents de cadrage (cahier des charges, user stories, dictionnaire de données). Le fichier `docs/carnet-de-bord.md` du dépôt est resté à l'état de gabarit.

⁴trello.com/b/536OvoxI/skillswap

6. Spécifications fonctionnelles

La présente section décline les choix fonctionnels du projet et leur traduction concrète : les contraintes ayant cadré la conception, l'architecture retenue, les maquettes d'interfaces, le modèle de données relationnel, le script SQL associé, et enfin la formalisation des cas d'utilisation — notamment celui qui sera approfondi techniquement en section 8.

Ces sept sous-sections forment un fil continu qui part de l'expression du besoin (6.1) jusqu'au comportement détaillé d'une fonctionnalité représentative (6.7), en passant par les différentes vues structurelles du système.

6.1. Contraintes du projet et livrables attendus

6.1.1. Contraintes

Le projet a été réalisé dans le cadre de l'apothéose finale de la formation O'clock — Promotion Dublin. Cette modalité simule un environnement professionnel et impose un ensemble de contraintes proches de celles d'une mission en entreprise.

Type de contrainte	Description
Temporelle	Apothéose conduite sur quatre semaines réparties en sprints courts (méthode Scrum), avec une livraison incrémentale et une démonstration en fin d'apothéose.
Humaine	Équipe de cinq personnes ⁵ avec rôles distribués (Lead Front, Lead Back, contributeurs full-stack), travaillant en distanciel synchrone via Discord (salons vocaux dédiés au pair-programming).
Technique — stack	Choix d'une stack moderne TypeScript de bout en bout : Next.js (App Router) côté front, Express côté back, PostgreSQL pour les données relationnelles, Meilisearch pour la recherche full-text, Socket.IO pour le temps réel. Choix documentés dans dix ADRs formalisés pendant le projet (cf. section 7.2).
Technique — qualité	Performance perçue inférieure à 500 ms (mesure Lighthouse), couverture OWASP Top 10 sur la sécurité, conformité visée RGAA 4.1 / WCAG 2.1 AA ⁶ , conventions de code partagées (Prettier, ESLint).
Réglementaire	Conformité RGPD : minimisation des données collectées, consentement explicite, droits d'accès et de suppression. Mentions légales et politique de confidentialité requises avant mise en production.

⁵Composition de l'équipe Dublin : Sébastien (Product Owner), Loïc (Scrum Master), Yorgan (Lead Back), Jérémy Soriano (Lead Front) et Antoine (Frontend).

⁶L'audit accessibilité formel via `axe-core` et Lighthouse est planifié mais non réalisé à la date de rédaction. Cette dette qualité est documentée en section 13.

Fonctionnelle	Périmètre MVP imposé : authentification, profil et compétences, recherche de membres, messagerie temps réel, système de notation et suivi entre membres. Hors-scope : paiement, application mobile native, vidéoconférence, géolocalisation avancée.
---------------	--

6.1.2. Livrables attendus

L'apothéose impose une chaîne complète de livrables, depuis la conception jusqu'à la mise en production effective :

Livrable	Description	État
Application en production	Plateforme accessible publiquement à skill-swap.fr , intégrant l'ensemble des fonctionnalités MVP.	✓ Livré
Base de données	Schéma PostgreSQL en six migrations Prisma successives, avec seed de développement peuplant 41 utilisateurs de démonstration ; le seed de production n'initialise que les référentiels (rôle, catégories, compétences, créneaux).	✓ Livré
Documentation technique	Livrable d'équipe mergé sur <code>main</code> : stratégie de documentation (<code>docs/documentation-strategy/</code>), diagrammes UML (<code>docs/uml/</code>) et référence des endpoints (<code>docs/endpoints/</code>). S'y ajoutent, produits pendant le projet sur la branche <code>Documentation</code> mais jamais mergés sur <code>main</code> , les dix ADRs et une spécification OpenAPI 3.0, initiée le 21 janvier 2026 avec trente endpoints et portée à trente-sept à la fin du projet . Le guide utilisateur Diátaxis et la mise en ligne de l'ensemble sur Vercel sont, eux, une extension post-projet.	✓ Livré ⁷
Tests automatisés	Tests d'intégration backend uniquement : sept fichiers <code>*.spec.test.ts</code> écrits pour le Node Test Runner natif, visant les contrôleurs et les events Socket.IO. Cinq des sept suites échouent sur un bug de fixture identifié et documenté — la donnée de rôle n'est pas créée avant les utilisateurs de test. Dette assumée, non corrigée avant la soutenance.	✓ Partiel ⁸
Conteneurisation	Stack Docker Compose à six services en production (frontend, backend, postgres, meilisearch, nginx, certbot) avec configurations dev et prod distinctes.	✓ Livré

⁷Le statut « Livré » porte sur la documentation d'équipe et sur les artefacts produits pendant le projet. La publication en ligne et le guide Diátaxis sont postérieurs à la soutenance.

⁸Deux limites cumulées : cinq des sept suites backend en échec sur fixture, et aucun test automatisé côté frontend dans le livrable (ni unitaire, ni E2E, ni catalogue de composants). Couverture jamais mesurée. Cf. section 10.

Pipeline de déploiement continu (CD)	Déploiement automatisé vers le VPS de production sur push <code>main</code> (<code>.github/workflows/deploy-prod.yml</code>) : connexion SSH, mise à jour du dépôt, reconstruction et redémarrage des conteneurs. Aucune étape d'intégration continue — le workflow ne lance ni test, ni lint, ni build de vérification avant de déployer.	✓ Livré
--------------------------------------	---	---------

6.2. Architecture logicielle du projet

L'architecture retenue est celle d'un **monolithe modulaire conteneurisé** : les responsabilités sont strictement séparées entre frontend et backend (deux applications distinctes, deux images Docker, deux processus de build), mais le backend reste un seul processus Node.js qui sert à la fois les routes REST et le serveur Socket.IO. Cette approche évite la complexité opérationnelle des microservices tout en autorisant un découpage clair des concerns. La pile de production en compte six au total (Fig. 2 à Fig. 4) ; les deux images applicatives en sont le coeur.

L'index Meilisearch est alimenté par une réindexation lancée manuellement (script `search:reindex`) : il n'y a pas de réindexation automatique au démarrage du serveur.

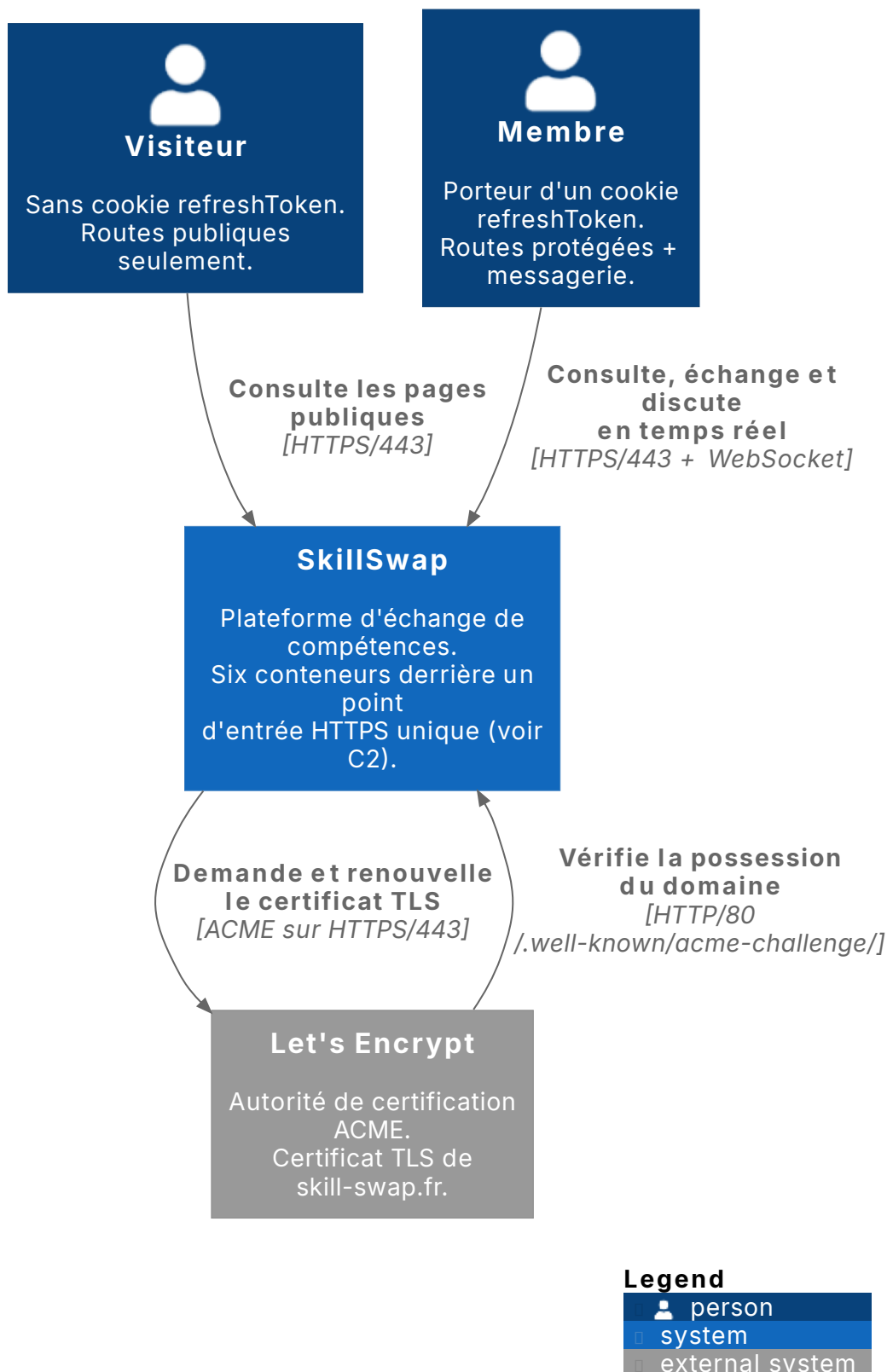
Diagramme de contexte (C4 niveau 1) — SkillSwap

Fig. 1. – **C4 niveau 1 — diagramme de contexte.** Le système vu de l'extérieur : deux acteurs, distingués par la seule présence du cookie `refreshToken` (`frontend/src/middleware.ts:15-16`), et un unique système tiers, Let's Encrypt. Le détail interne est réparti sur les trois vues du niveau C2 qui suivent.

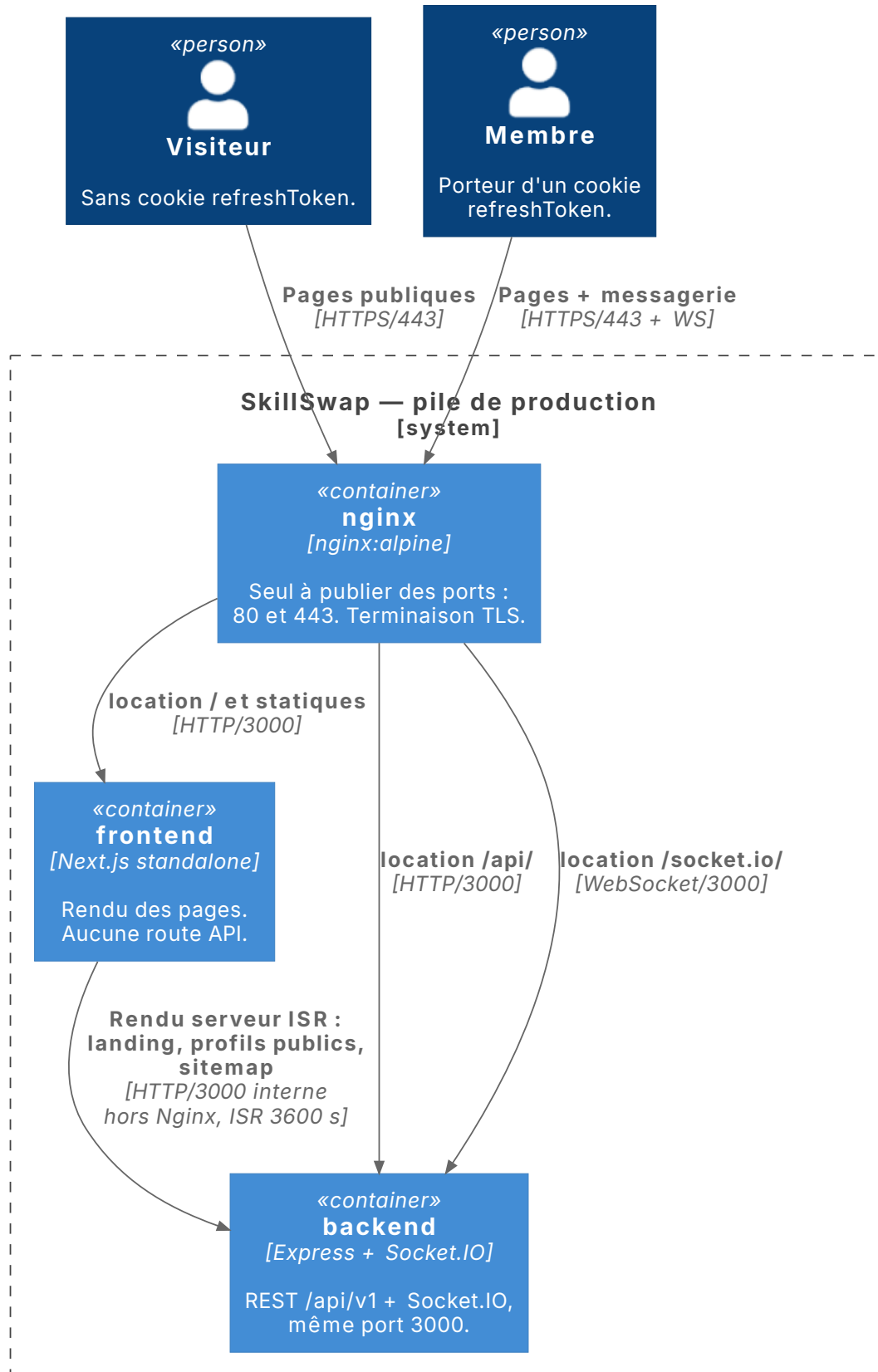
Conteneurs (C4 niveau 2, vue 1/3) — entrée et routage

Fig. 2. – **C4 niveau 2, vue 1/3 — entrée et routage.** Le chemin d'une requête depuis le navigateur. Nginx est le seul conteneur à publier des ports ; le frontend et le backend ne sont joignables que depuis le réseau Compose. Coutures : le conteneur *backend* est repris en Fig. 3, le conteneur *nginx* en Fig. 4.

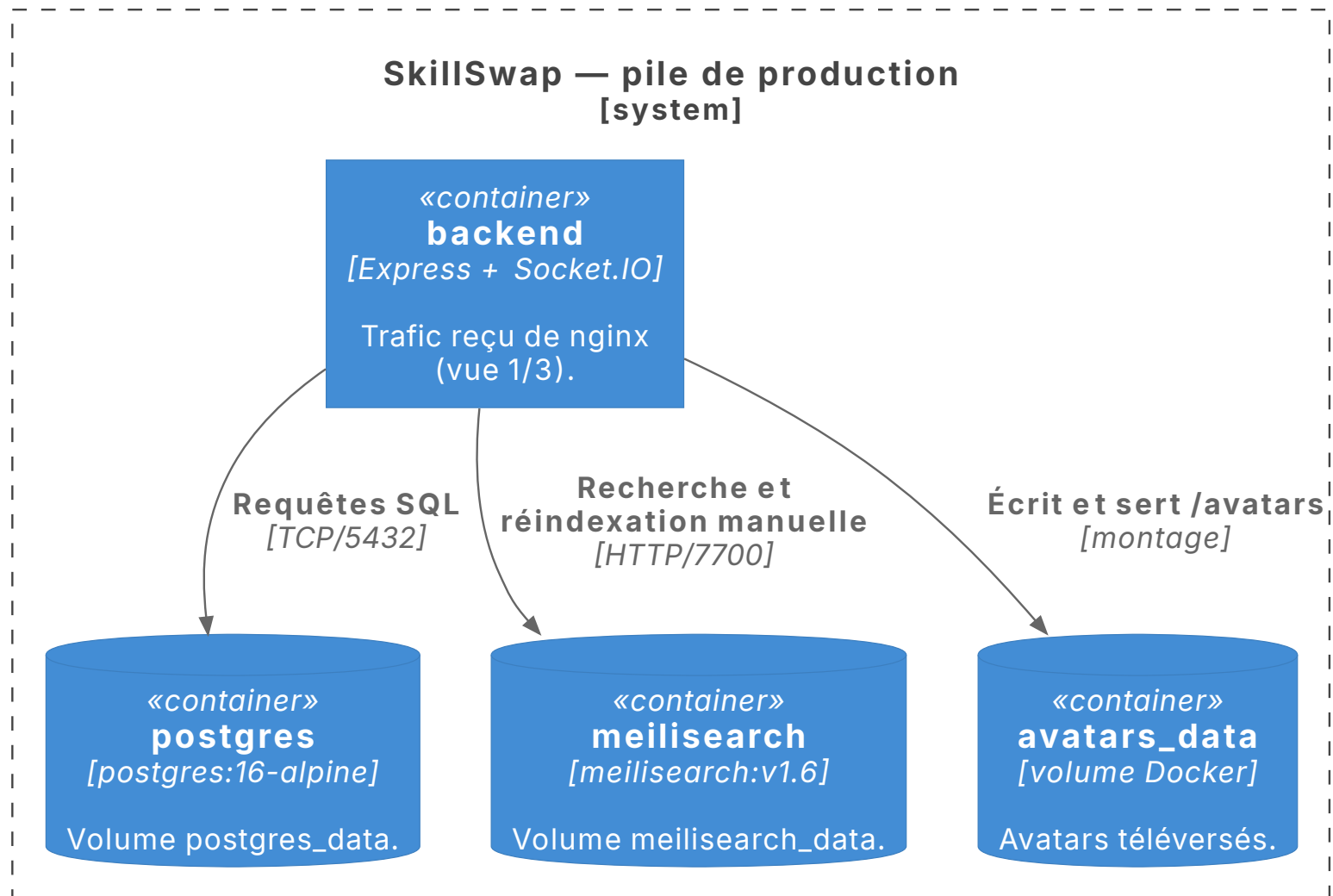
Conteneurs (C4 niveau 2, vue 2/3) — persistance du backend

Fig. 3. – **C4 niveau 2, vue 2/3 — persistance du backend.** Couture avec Fig. 2 : le conteneur *backend*, qui y reçoit le trafic de Nginx. Chaque arête porte son protocole et son port réels.

Conteneurs (C4 niveau 2, vue 3/3) — chaîne de renouvellement TLS

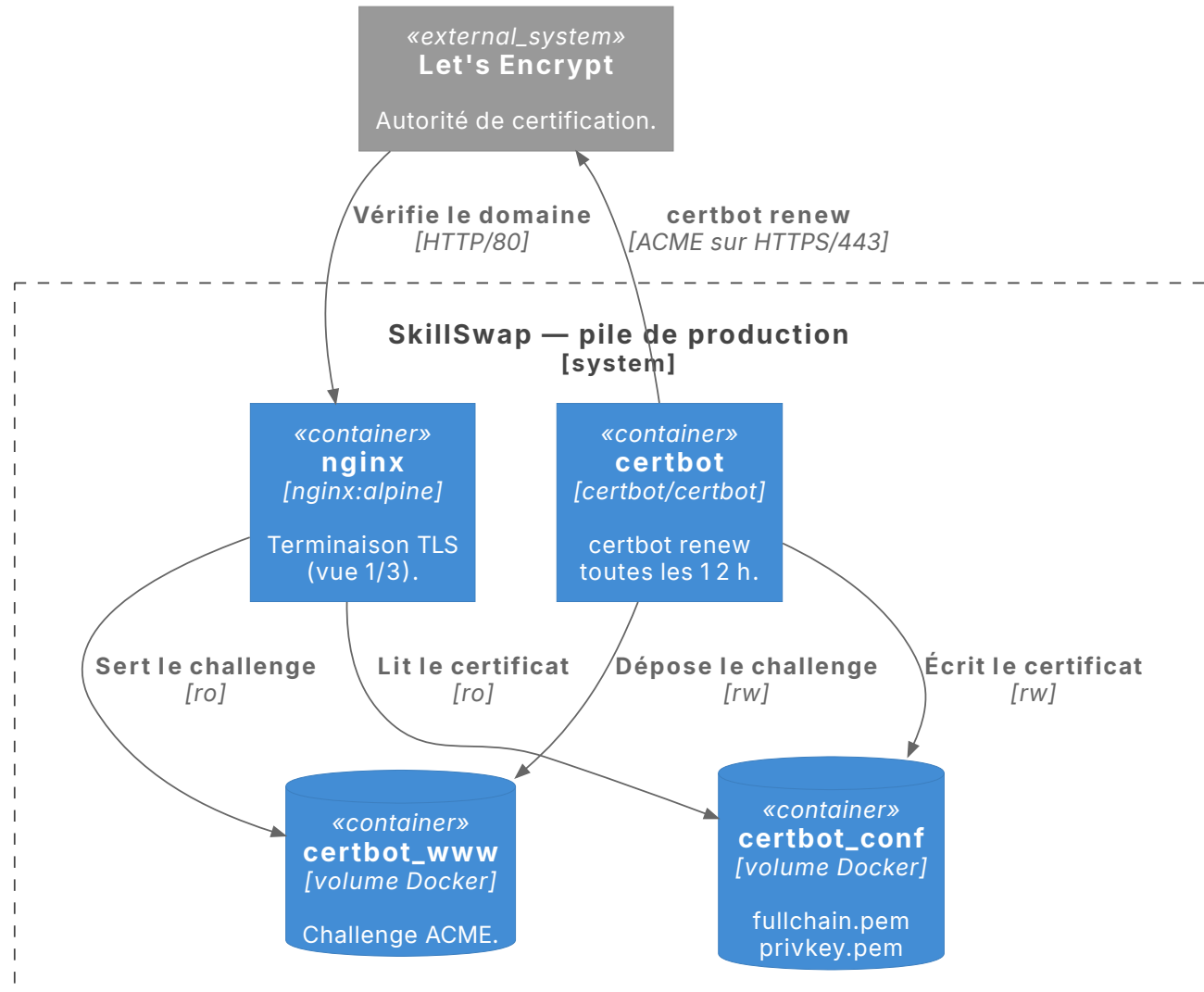


Fig. 4. – **C4 niveau 2, vue 3/3 — chaîne de renouvellement TLS.** Couture avec Fig. 2 : le conteneur *nginx*, qui assure la terminaison TLS. Les deux volumes *certbot* sont montés deux fois — en écriture côté certbot, en lecture seule côté nginx.

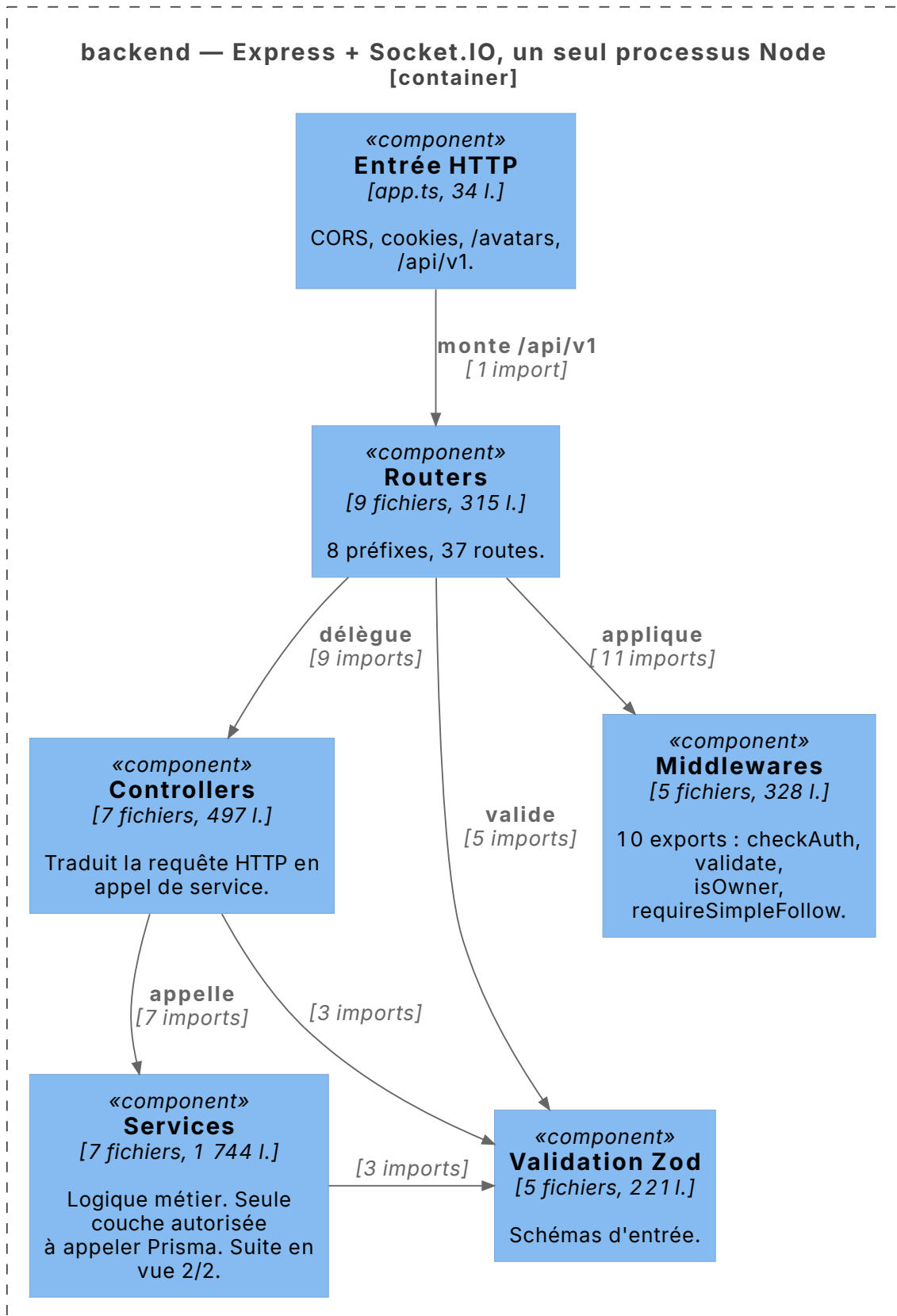
Composants du backend (C4 niveau 3, vue 1/2) — chaîne de traitement REST

Fig. 5. – **C4 niveau 3, backend vue 1/2 — chaîne de traitement d’une requête REST.** Regroupement par couche, non par fichier. Les nombres portés sur les arêtes sont des nombres d’imports, mesurés et non estimés. Couture : la couche *Services* est reprise en Fig. 6.

Composants du backend (C4 niveau 3, vue 2/2) — accès aux données
En rouge : les cinq modules qui atteignent Prisma hors de la couche service

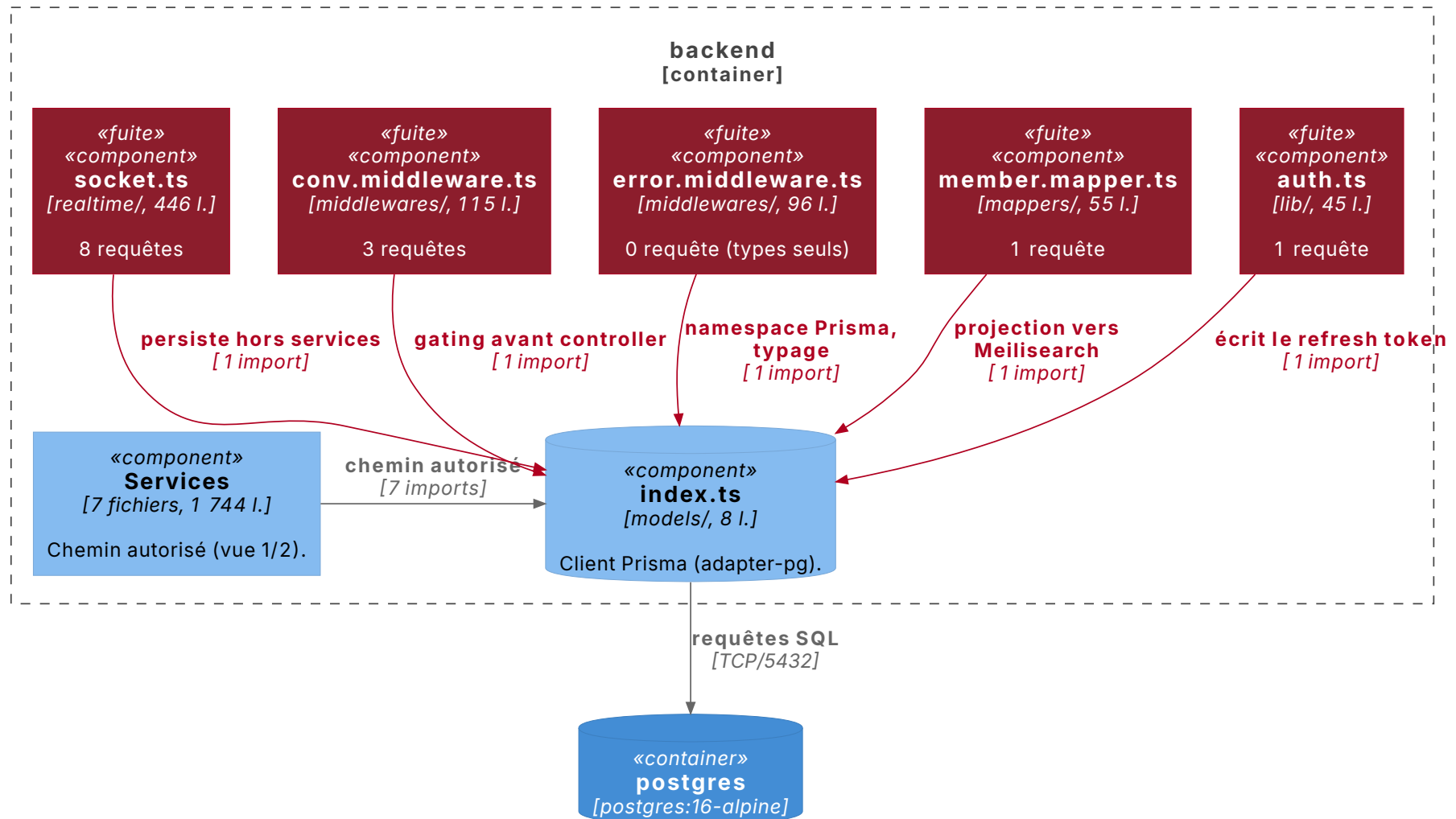


Fig. 6. – **C4 niveau 3, backend vue 2/2 — accès aux données**. En rouge, les cinq modules atteignant Prisma hors de la couche service, avec leur nombre de requêtes. Analyse `dependency-cruiser` sur **72 modules et 169 dépendances**, **zéro violation** des quatre règles de sens de dépendance. Détail en sous-section 7.3.

Les figures de conception de cette section ont été reconstruites en août 2026 à partir du code livré et de la base de données, selon une démarche déterministe : chaque élément représenté est adossé à un fichier source, une route ou une entrée du catalogue PostgreSQL. Les artefacts d'origine produits en sprint 0 sont conservés dans le dépôt d'équipe (`docs/uml/`).

6.2.1. Écarts relevés lors de la reconstruction

La reconstruction a été confrontée à la figure d'architecture d'origine, en comparant non pas deux fichiers source mais la source PlantUML **embarquée dans le PNG livré** — donc le texte qui a réellement produit l'image. Onze écarts de fait en ressortent, dont quatre structurants, reproduits ci-dessous. Le relevé complet, élément par élément et relation par relation, est dans `docs/uml/c4/TRACABILITE.md`.

Écart	Ce que montrait la figure	Ce que montre le code
Canal temps réel	Une liaison WebSocket directe entre le module de chat du frontend et le serveur Socket.IO, hors du proxy.	Tout le WebSocket transite par Nginx : <code>nginx/prod.conf:85-94</code> route <code>/socket.io/</code> vers l'upstream <code>backend:3000</code> . Le backend ne publie aucun port — <code>docker-compose.prod.yml:9-10</code> déclare <code>expose</code> et non <code>ports</code> : il est injoignable depuis l'extérieur.
Terminaison TLS	Aucune. Ni HTTPS, ni port, ni certificat n'apparaissent sur la figure.	<code>nginx/prod.conf:33</code> — <code>listen 443 ssl http2</code> ; certificats en <code>prod.conf:37-38</code> ; redirection 80 vers 443 en <code>prod.conf:26-28</code> ; ports publiés <code>80:80</code> et <code>443:443</code> en <code>docker-compose.prod.yml:85-87</code> .
Limitation de débit	Le composant de passerelle porte la mention « Rate limiting » parmi ses responsabilités.	Aucune directive de limitation dans la configuration : <code>grep -nE "limit_req limit_conn limit_rate"</code> sur <code>devops/nginx/prod.conf</code> ne retourne aucun résultat.
Statut de <code>/profil</code>	La page de profil est placée derrière un composant <code>[Private]</code> , donc réservée aux membres connectés.	La route est publique, et délibérément : <code>frontend/src/middleware.ts:6</code> ne protège que <code>/recherche</code> , <code>/conversation</code> et <code>/mon-profil</code> ; la ligne 5 documente le choix — « <code>/profil</code> est PUBLIC pour le SEO ».

6.2.2. Communications inter-composants

Deux chemins distincts atteignent l'API. Le code client exécuté dans le navigateur appelle `/api/v1/*` et `/socket.io/` en HTTPS à travers le reverse proxy Nginx, seul composant de la pile à publier des ports. Le conteneur frontend, lui, appelle directement `http://backend:3000` sur le réseau Docker interne, hors proxy, pour le rendu serveur ISR de la page d'accueil, des profils publics et du sitemap (`page.tsx:60` et `:82`, `profil/[id]/page.tsx:64`, `sitemap.ts:34`). Le backend ne publiant aucun port (`docker-compose.prod.yml:9-10` déclare `expose` et non `ports`), ce second chemin n'est joignable que depuis l'intérieur du réseau Compose.

Sur l'un comme sur l'autre, deux canaux complémentaires coexistent : **REST** (`/api/v1/*`) pour les requêtes synchrones (authentification, CRUD, listing initial des messages) et **WebSocket** via Socket.IO sur le même serveur HTTP pour les flux temps réel (envoi/réception de messages, notifications de nouvelles conversations). Ce choix d'architecture hybride est documenté dans l'ADR-011 — formalisé a posteriori pour ce dossier — et détaillé techniquement en sous-section 8.4.

6.3. Maquettes et enchaînement des maquettes

Les maquettes ont été produites en amont du développement à l'aide de **Figma**, sur la base d'un atelier de cadrage produit avec l'équipe au début de l'apothéose. Elles définissent la charte graphique, la hiérarchie des écrans et les principaux états interactifs.

6.3.1. Arborescence de l'application

Arborescence des routes — frontend Next.js App Router

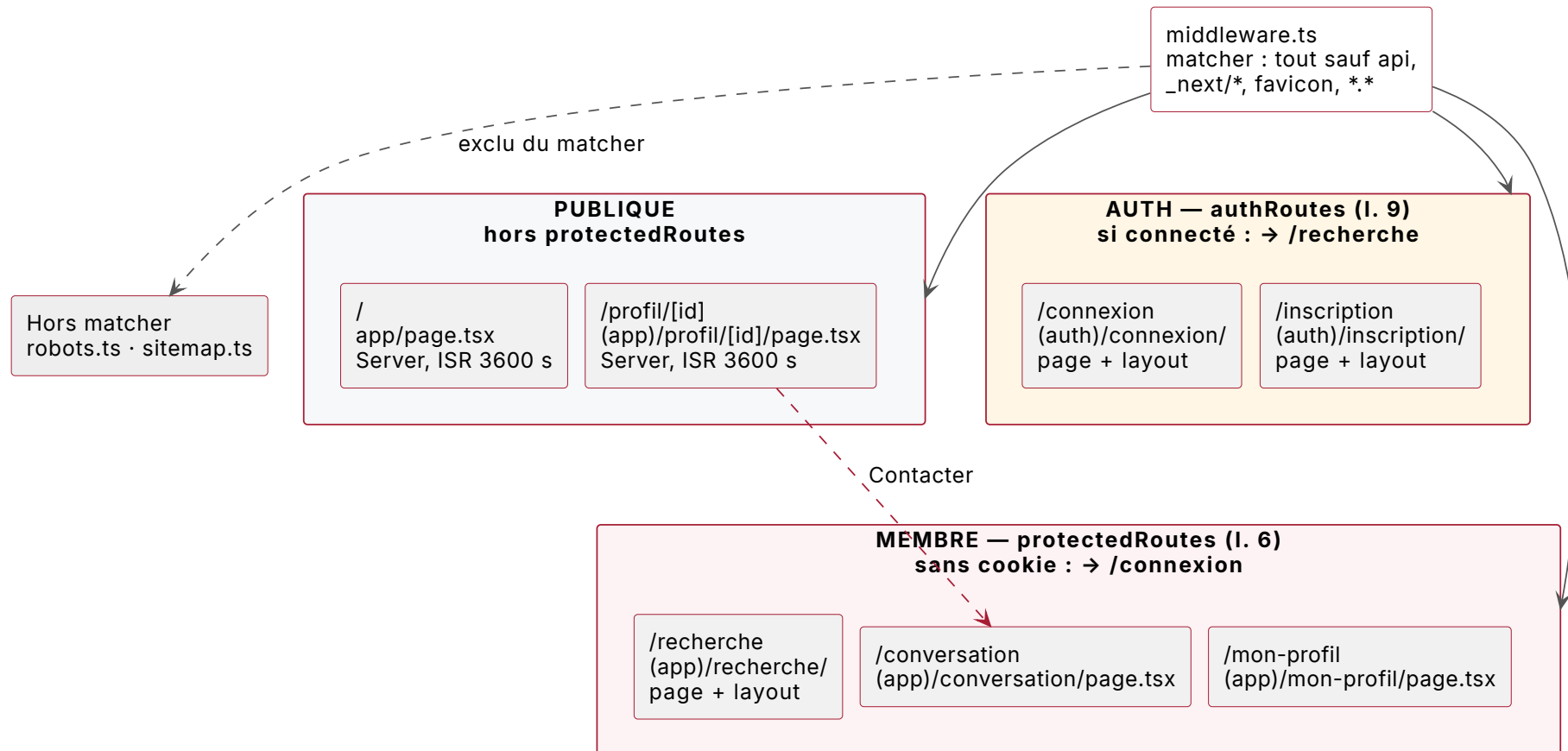


Fig. 7. – **Arborescence des routes du frontend.** Les sept routes rendues par l'App Router, réparties selon ce que `middleware.ts` protège **réellement** et non selon leur rangement. Les parenthèses `(app)` et `(auth)` sont des groupes de routes : elles n'apparaissent pas dans l'URL et **ne déterminent pas la protection**. `/profil/[id]` en est la démonstration : rangé sous `(app)`, il reste public, parce qu'absent de `protectedRoutes` — choix documenté en clair dans le code pour l'indexation. Les trois routes protégées sont `/recherche`, `/conversation` et `/mon-profil` ; les deux routes d'authentification redirigent vers `/recherche` si un cookie de session est déjà présent.

L'arborescence reflète le principe de **progressive disclosure** : la zone publique reste librement accessible, et l'authentification n'est requise qu'au moment d'une action transactionnelle (envoi de message, ajout de compétence, suivi de membre).

6.3.2. Parcours utilisateur principal

Parcours utilisateur — de la landing à la première conversation

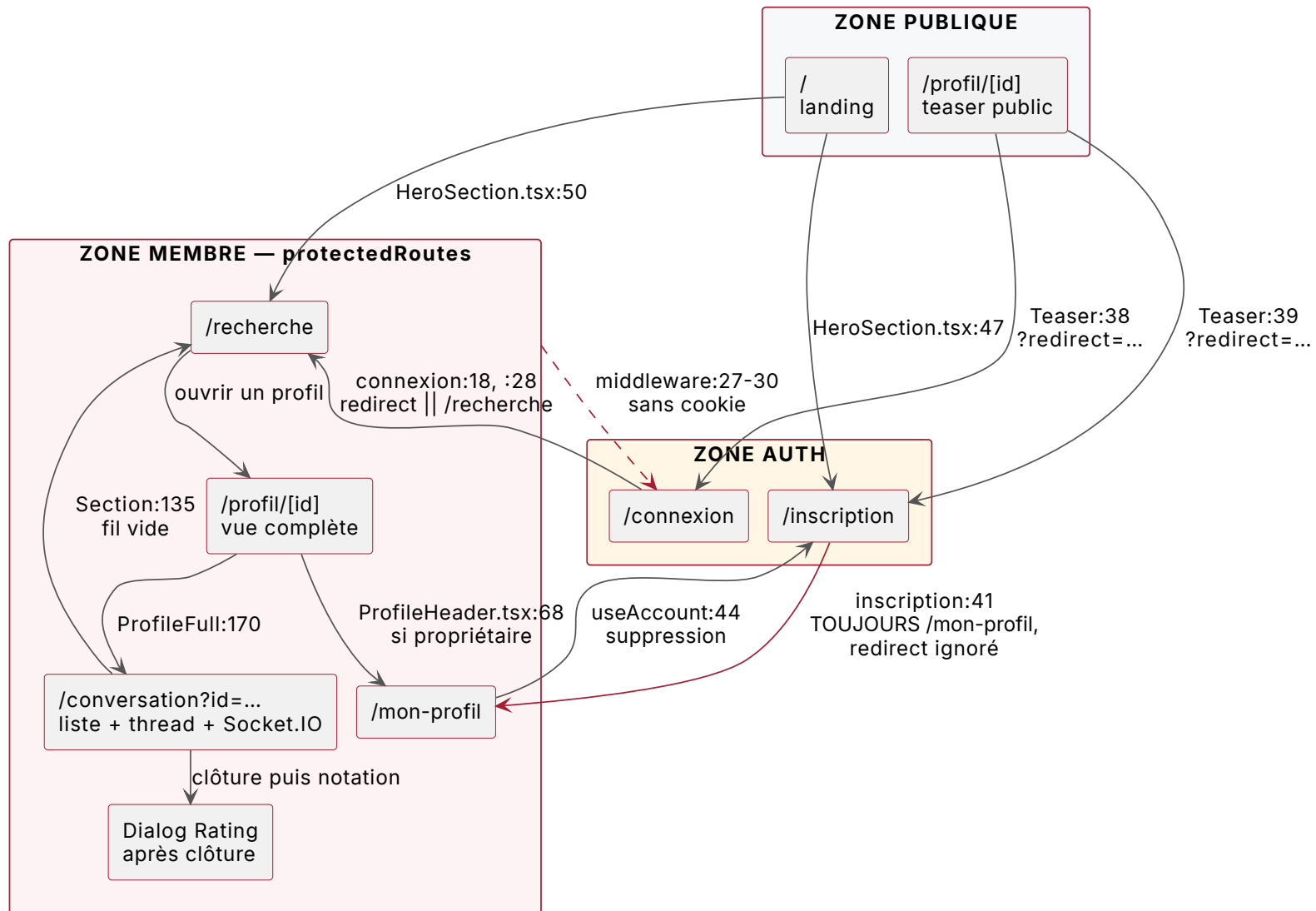


Fig. 8. – **Parcours utilisateur — de la landing à la première conversation.** Établi depuis les routes rendues et les redirections réellement écrites : chaque transition porte son fichier et sa ligne.

Les maquettes des quatre écrans principaux — accueil, profil, recherche et messagerie — sont reproduites en annexe F : Fig. 28, Fig. 29, Fig. 30 et Fig. 31.

6.3.3. Écarts relevés lors de la reconstruction

Dix transitions sur onze étaient exactes. Le parcours d'origine nommait correctement les routes françaises, les groupes `(auth)` et `(app)`, et la garde du middleware. Trois points seulement le séparent du code.

Écart	Ce que montrait la figure	Ce que montre le code
Redirection après inscription	« succès → <code>redirect=/recherche</code> », symétrique de la connexion.	inscription/page.tsx:41 pose <code>window.location.href = '/mon-profil'</code> en dur. Le paramètre <code>redirect</code> n'y est jamais lu, alors que la connexion le consomme — connexion/page.tsx:18 puis :28. Or ProfileTeaser.tsx:39 le produit : un visiteur venu d'un profil public perd sa destination.
Transitions absentes	Trois retours de bascule de session ne figurent pas.	Déconnexion vers <code>/</code> — Header/index.tsx:34 ; suppression de compte vers <code>/inscription</code> — useAccount.ts:44 ; conversation créée vers <code>/conversation?id=...</code> — ProfileFull.tsx:170, l'identifiant transitant par la query string et non par un segment d'URL.
Fidélité d'ensemble	—	C'est la plus fidèle des six figures reconstruites : contrairement à l'arborescence, elle ne portait aucune route inventée, et son seul écart de fond tient à une ligne.

6.4. Modèle entités-associations et modèle physique de la base de données

La modélisation suit les trois niveaux Merise. Le **MCD** a été formalisé en notation Mocodo à partir des règles de gestion ; le **MLD** et le **MPD** ont été reconstruits de façon déterministe depuis une base PostgreSQL montée à partir des six migrations réelles, en interrogeant le catalogue `information_schema` — aucune inférence depuis le code applicatif. Le fichier `schema.prisma` reste la source de vérité opérationnelle qui génère les migrations, ce qui garantit la cohérence stricte entre le schéma documenté et le schéma physique appliqué en production.

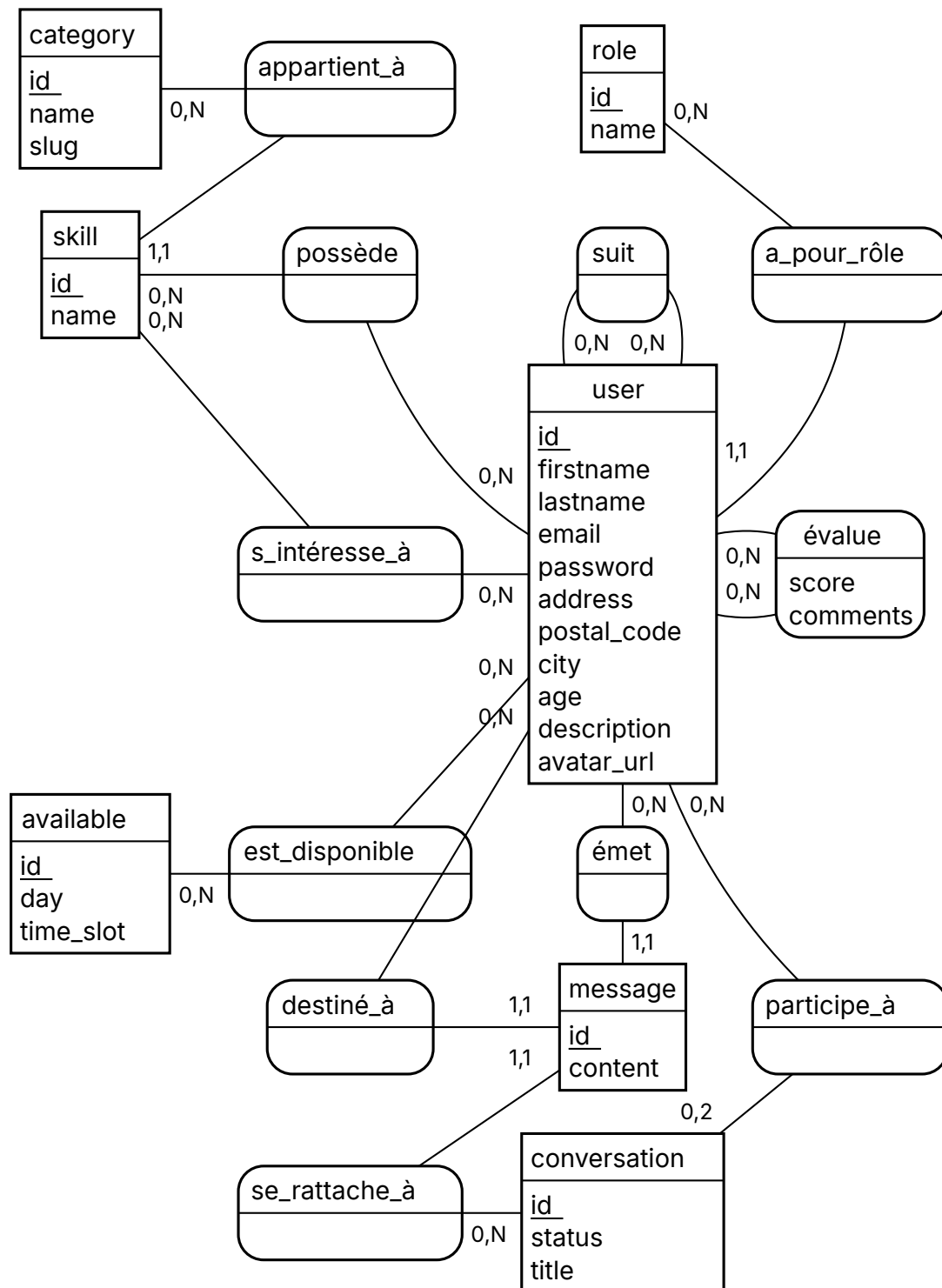
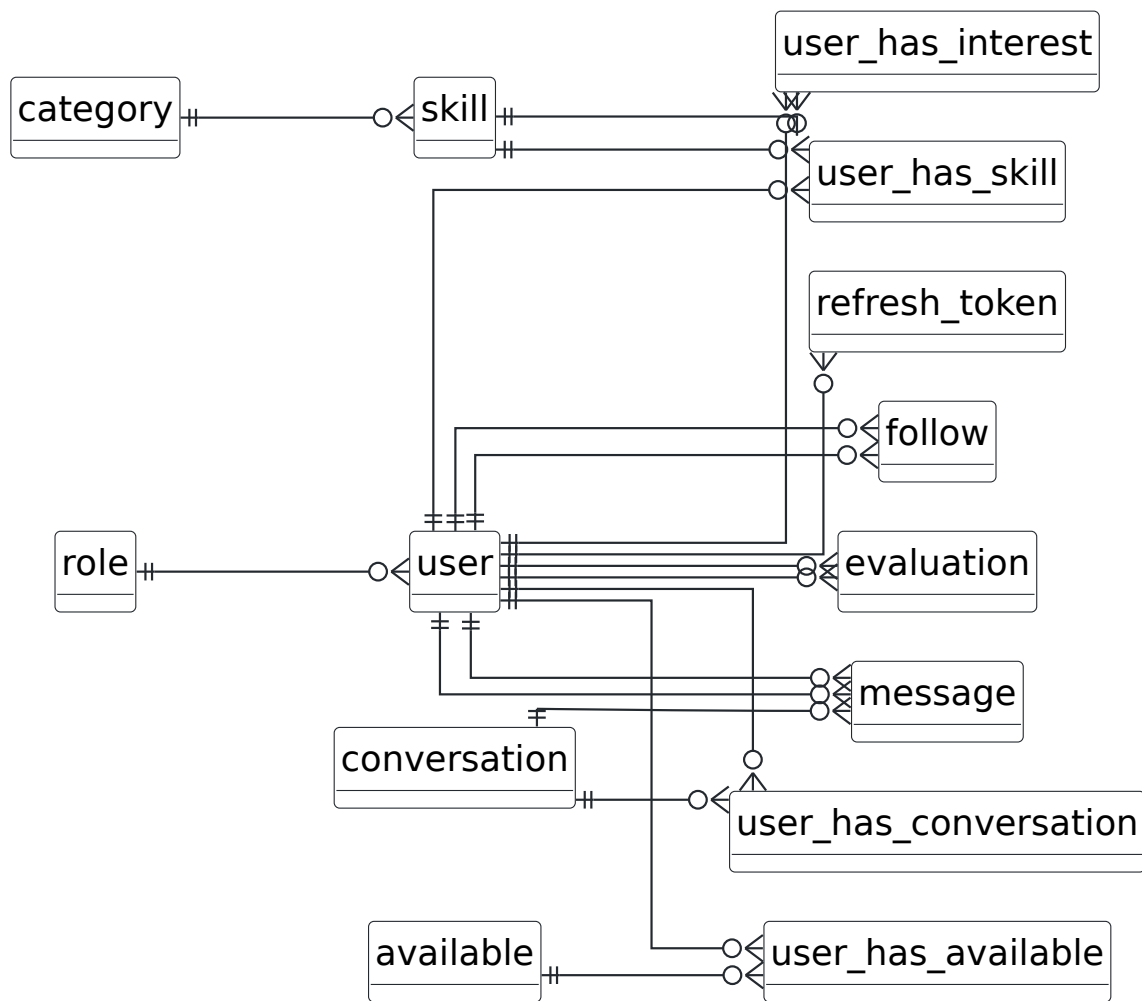


Fig. 9. – Modèle conceptuel de données (Merise, notation Mocodo) — sept entités métier et onze associations, dont une association porteuse **évalue** (attributs **score** et **comments**). L'entité technique **refresh_token** est volontairement exclue du niveau conceptuel. Les entités sont **groupées par domaine fonctionnel** autour de **user** — compétences au nord-ouest, identité et social au nord, disponibilités à l'ouest, échange au sud. **user** porte **onze des vingt-deux pattes d'association** — neuf associations, dont deux réflexives (**suit** et **évalue**) qui comptent double ; l'agencement les répartit sur les quatre côtés de l'entité au lieu de six sur un seul flanc. Contenu identique à la dérivation d'origine : réagencement seul, vérifié par diff sémantique automatisé.

ERD SkillSwap — vue d'ensemble (14 tables, 18 FK)



Pied-de-corbeau : || = exactement 1, o{ = 0 à n.
 18 liens = les 18 clés étrangères du catalogue.
 Arêtes doubles vers user : message (sender_id, receiver_id), follow (follower_id, followed_id), evaluation (evaluator_id, evaluated_id).
 Colonnes : voir les cinq sous-modèles.

Fig. 10. – Vue d'ensemble du modèle physique — quatorze tables et dix-huit clés étrangères, regroupées en cinq domaines fonctionnels. Chaque domaine est détaillé, colonnes complètes, en annexe G.

6.4.1. Domaines fonctionnels du modèle

Les quatorze modèles se regroupent en cinq domaines cohérents :

Domaine	Modèles
Identité	User, Role, RefreshToken — gestion des comptes, des rôles et des sessions par rotation de tokens. Détail : Fig. 32.

Compétences	<code>Skill</code> , <code>Category</code> , <code>UserHasSkill</code> , <code>UserHasInterest</code> — référentiel des compétences disponibles sur la plateforme et expression des compétences offertes / recherchées par chaque membre. Détail : Fig. 33.
Disponibilités	<code>Available</code> , <code>UserHasAvailable</code> — créneaux horaires standardisés associés à chaque membre. Détail : Fig. 34.
Échange	<code>Conversation</code> , <code>Message</code> , <code>UserHasConversation</code> — coeur de l'échange, traité en détail en section 8. Détail : Fig. 35.
Social	<code>Follow</code> , <code>Rating</code> — graphe de suivi entre membres et système de notation. Détail : Fig. 36.

La différence de comptage entre les deux niveaux est normale et voulue : le MCD recense **sept entités métier** (`user`, `role`, `category`, `skill`, `available`, `conversation`, `message`), tandis que le MLD en compte **quatorze** après transformation. Les sept tables supplémentaires sont la matérialisation des associations : les quatre tables de jonction issues des relations N-N, les deux associations réflexives `follow` et `evaluation`, et l'entité technique `refresh_token`, exclue du niveau conceptuel parce qu'elle relève de l'authentification et non du métier.

Les contraintes d'unicité critiques sont matérialisées au niveau base : `@@unique([followedId, followerId])` sur `Follow` (graphe orienté, non-auto-suivi appliqué côté middleware) et `@@unique([evaluatorId, evaluatedId])` sur `Rating` (mappée `evaluation` en BDD), qui empêche tout doublon de notation entre une même paire de membres.

6.5. Script de création et de modification de la base de données

L'évolution du schéma est gérée par six migrations Prisma successives, chacune représentant une étape datée de la conception. La traçabilité SQL de cette évolution est intégrale et auditable.

Date	Migration	Objet
2026-01-12	init_db	Création initiale du schéma : 13 tables, contraintes de clés étrangères, index primaires, enums <code>RoleOfUser</code> , <code>StatusOfConversation</code> et <code>dayInAWeek</code> .
2026-01-14	add_category_slug	Ajout du champ <code>slug</code> sur <code>Category</code> pour les URLs SEO-friendly des pages catégorie.
2026-01-16	create_relation_table_user_available	Refonte du modèle de disponibilités : suppression des colonnes <code>start</code> / <code>end</code> / <code>user_id</code> de <code>available</code> , introduction de l'enum <code>Time</code> (<code>Morning</code> / <code>Afternoon</code>) et création de la table de jonction N-N <code>user_has_available</code> .
2026-01-17	fix_snake_case	Renommage de la colonne <code>avatarUrl</code> → <code>avatar_url</code> sur la table <code>user</code> (réalignement camelCase → snake_case oublié à la migration initiale).

2026-01-18	add_unique_constraint	Ajout de contraintes d'unicité manquantes (<code>follow</code> sur <code>(followed_id, follower_id)</code> , <code>evaluation</code> sur <code>(evaluator_id, evaluated_id)</code>).
2026-01-20	make_the_comment_field_in_the_rating_table_optional	Passage du champ <code>comments</code> de <code>evaluation</code> en facultatif (UX : l'évaluateur peut donner une note sans être obligé de commenter).

L'extrait ci-dessous est issu du schéma physique réel, obtenu par `pg_dump --schema-only` sur une base montée à partir des six migrations — il reflète donc l'état effectivement appliqué, et non une reconstitution :

```
CREATE TYPE public."RoleOfUser" AS ENUM (
  'Membre'
);

CREATE TABLE public."user" (
  id integer NOT NULL,
  firstname text NOT NULL,
  lastname text NOT NULL,
  email text NOT NULL,
  password text NOT NULL,
  address text,
  postal_code integer,
  city text,
  age integer,
  description text,
  role_id integer NOT NULL,
  created_at timestamp(3) without time zone DEFAULT CURRENT_TIMESTAMP NOT NULL,
  updated_at timestamp(3) without time zone DEFAULT CURRENT_TIMESTAMP NOT NULL,
  avatar_url text
);
```

L'ordre des colonnes porte la trace des migrations successives : `avatar_url` figure en dernière position parce qu'elle a été recrée par la migration `fix_snake_case`, postérieure à la création initiale de la table.

Les extraits SQL générés par Prisma — création de la table `user` dans `init_db` et matérialisation des contraintes d'unicité dans `add_unique_constraint` — sont reproduits intégralement en **annexe A**. Prisma génère le SQL au format conventionnel : type `TEXT` sans contrainte de longueur (les bornes sont appliquées en amont par les schémas Zod côté serveur), unicités via `CREATE UNIQUE INDEX` séparés, et FK déclarées en fin de migration via `ALTER TABLE`. Les contraintes d'unicité de `Follow` et `evaluation` sont appliquées au niveau base — défense en profondeur garantissant l'intégrité même en cas de bug applicatif.

6.6. Diagramme du comportement des fonctionnalités — cas d'utilisation

Les cas d'utilisation sont établis à partir des **37 routes réellement montées** sur `/api/v1` et de leurs chaînes de middlewares ; les acteurs sont au nombre de deux, l'énumération

`RoleOfUser` ne comportant que `Membre`. Chaque cas porte sa méthode HTTP, son chemin et, le cas échéant, le middleware qui en restreint l'accès. Trente-sept cas ne tenant pas sur une page lisible, la vue est découpée par domaine, sur les préfixes de routeur.

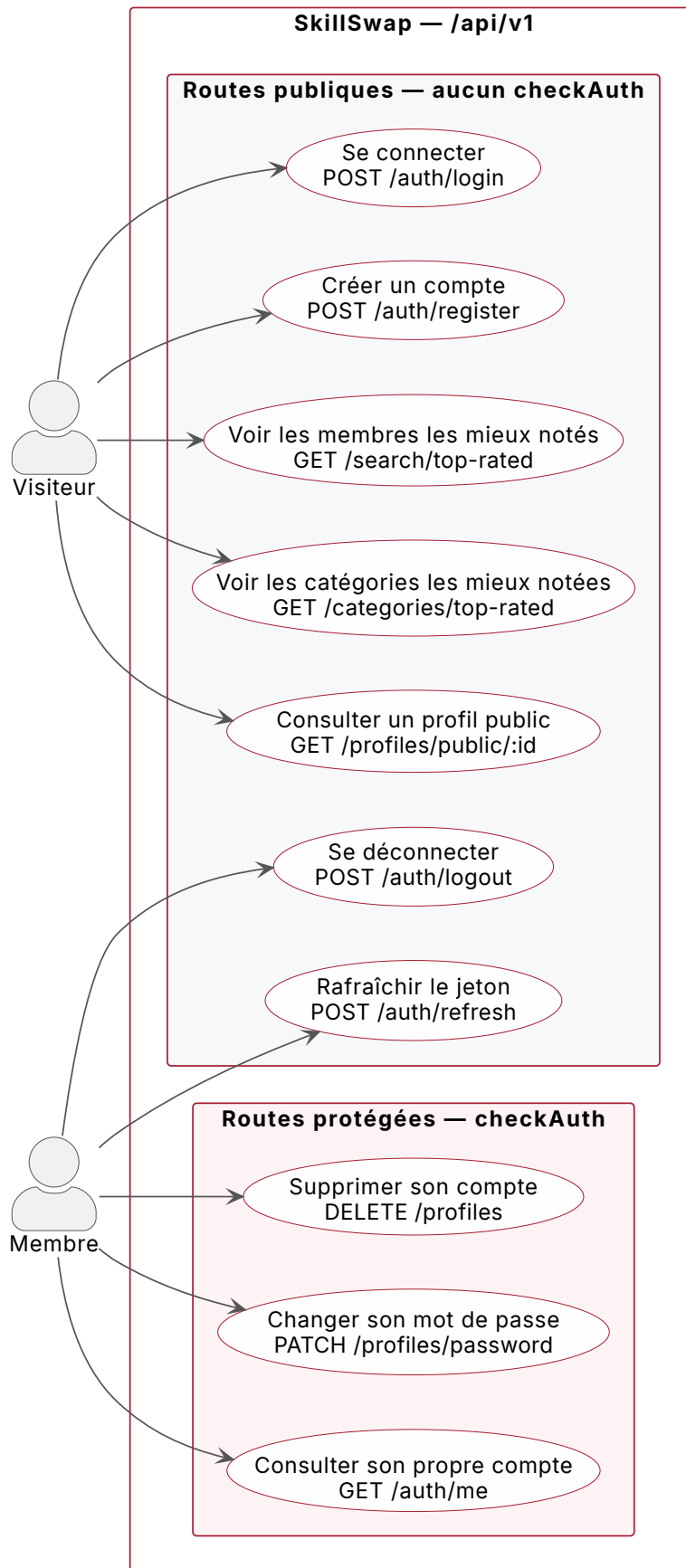
Cas d'utilisation (vue 1/5) — accès public, authentification, compte

Fig. 11. – **Vue 1/5 — accès public, authentification et compte : 10 routes.** Les sept routes publiques sont celles qui n'appliquent aucun `checkAuth`.

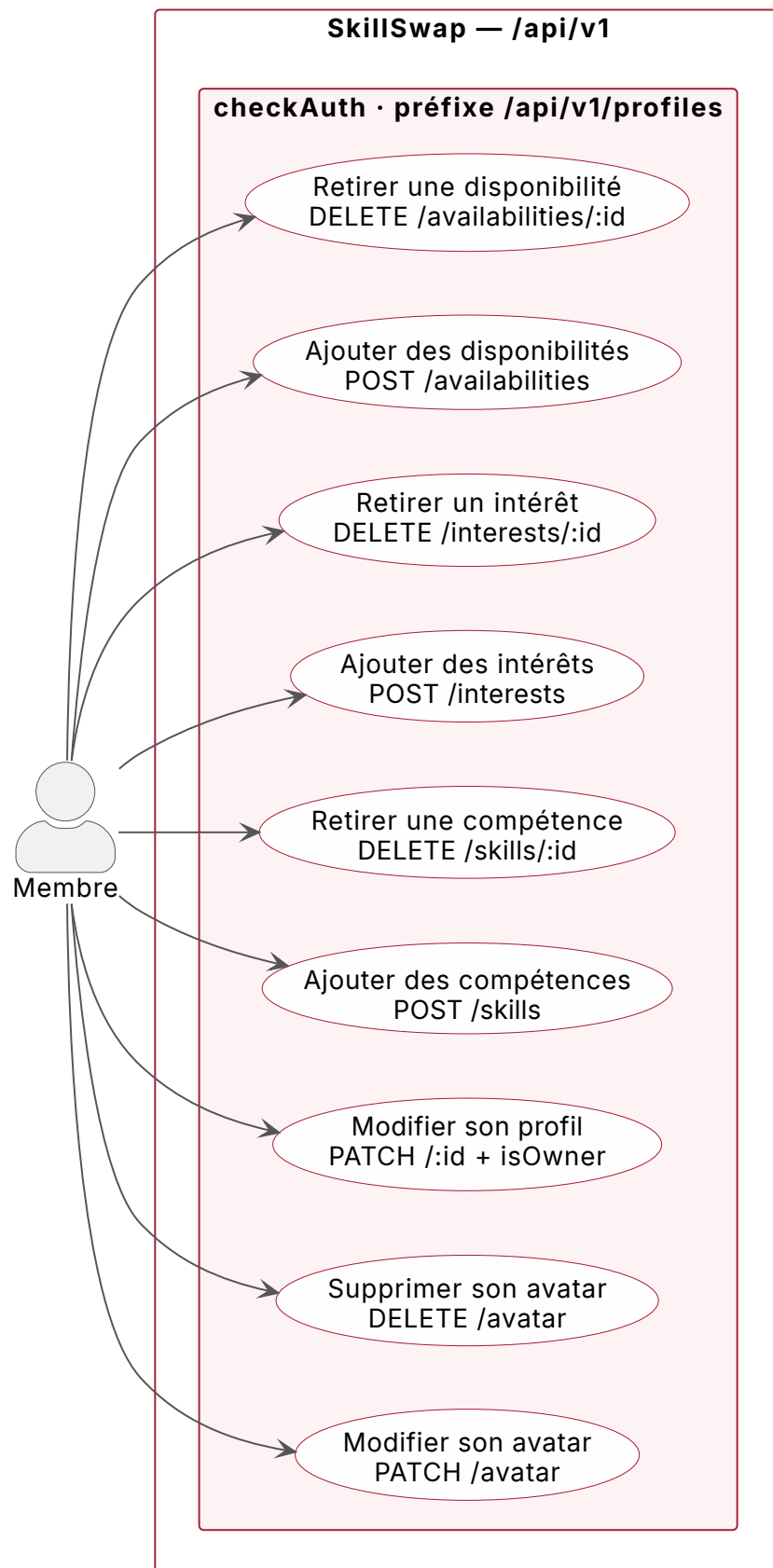
Cas d'utilisation (vue 2/5) — gestion de son profil

Fig. 12. – **Vue 2/5 — gestion de son profil : 9 routes.** Toutes sous `checkAuth`. Une seule porte une contrainte supplémentaire : `PATCH /profiles/:id` applique `isOwner`.

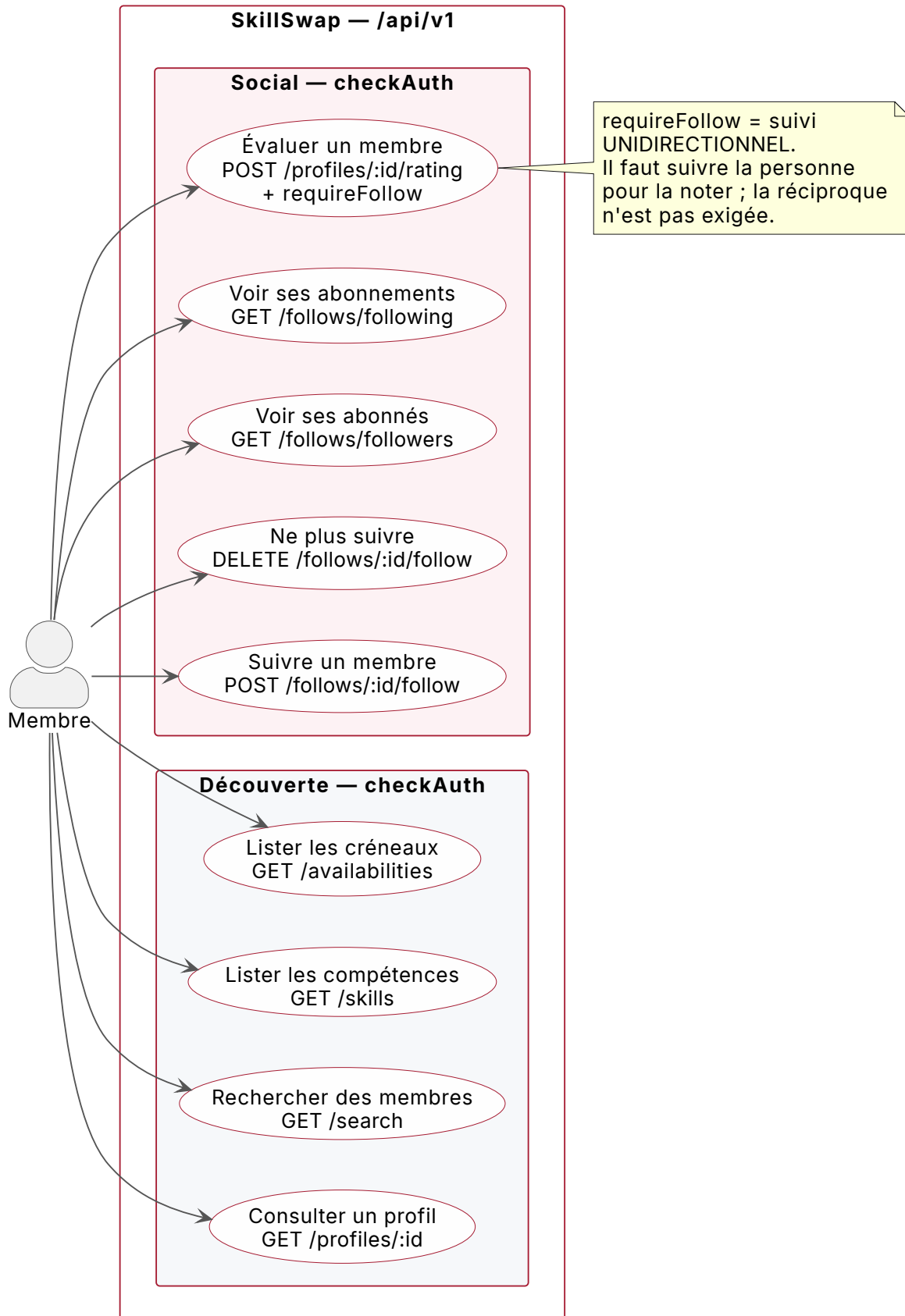
Cas d'utilisation (vue 3/5) — découverte et social

Fig. 13. – **Vue 3/5 — découverte et social : 9 routes.** L'évaluation est la seule à porter une contrainte de lien social, et ce lien est unidirectionnel — voir sous-section 6.6.1.

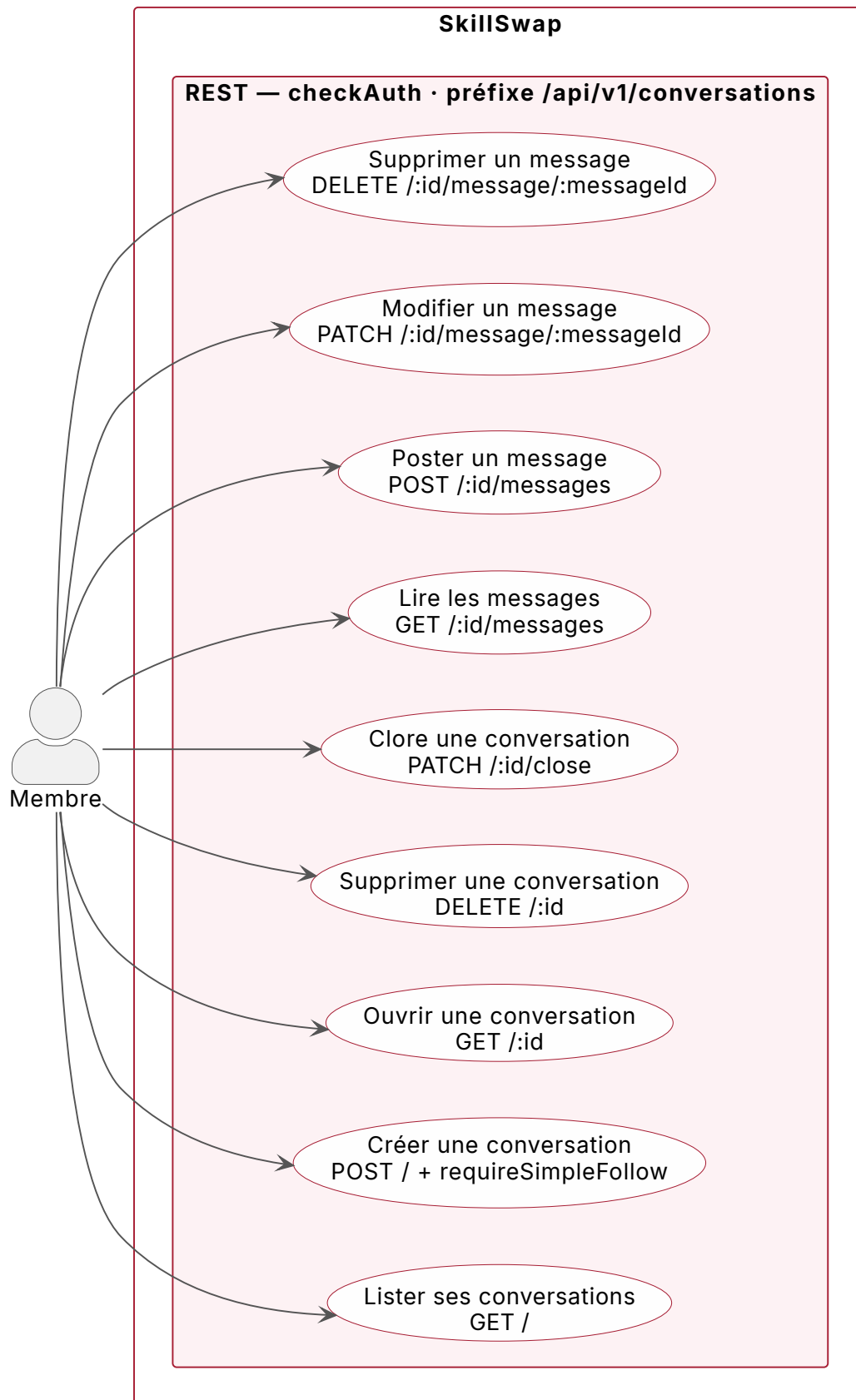
Cas d'utilisation (vue 4/5) — messagerie REST

Fig. 14. – **Vue 4/5 — messagerie, canal REST : 9 routes.** La création de conversation est la seule route du produit à porter `requireSimpleFollow` : c'est le gating métier détaillé en section 8.

Cas d'utilisation (vue 5/5) — messagerie temps réel

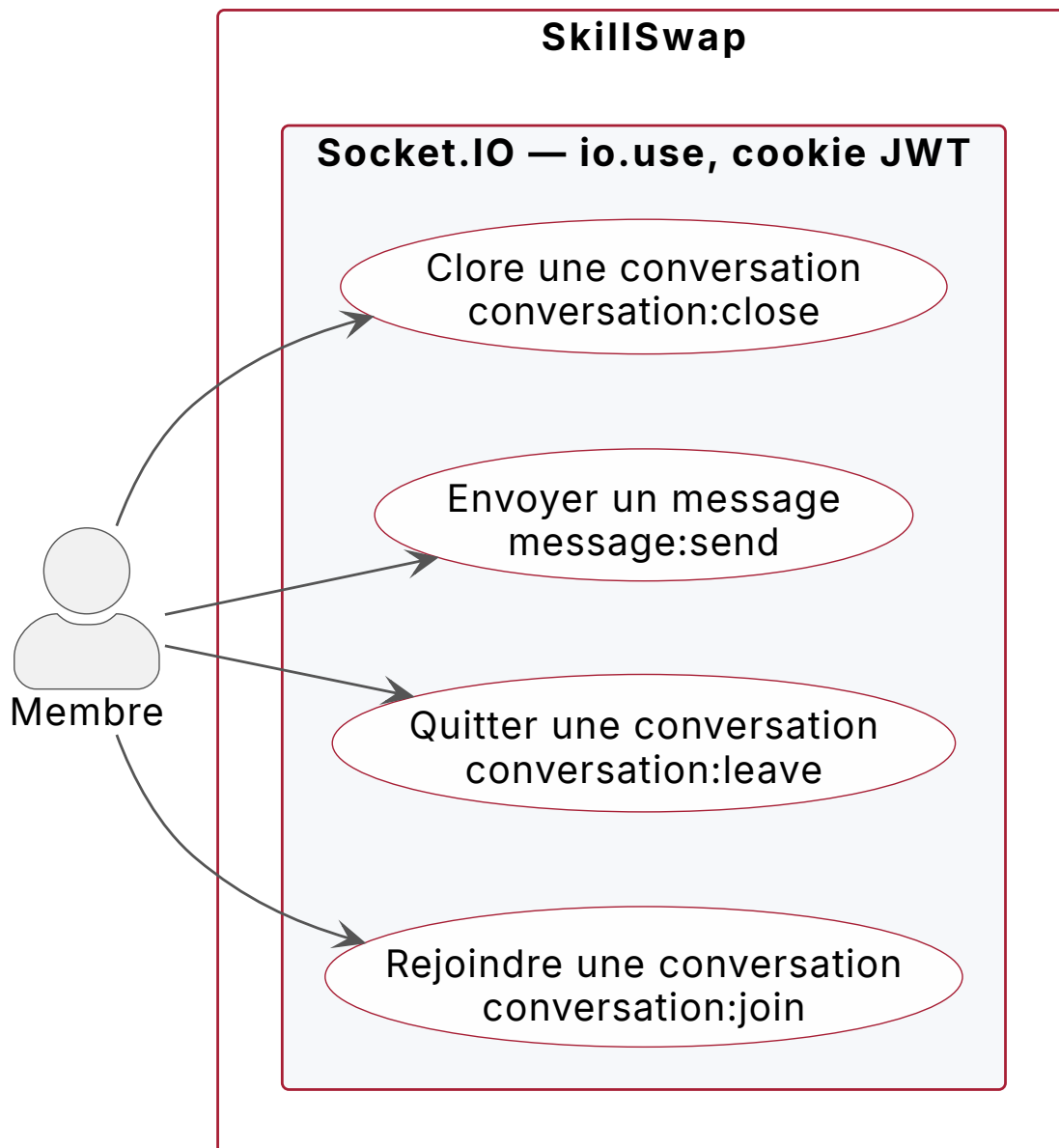


Fig. 15. – **Vue 5/5 — messagerie, canal temps réel : 4 events.** Ces quatre events ne sont pas des routes REST et ne sont pas gardés par `checkAuth` mais par le middleware de handshake `io.use` — d'où leur zone distincte.

6.6.1. Écarts relevés lors de la reconstruction

La figure d'origine a été confrontée à sa source PlantUML embarquée — 232 lignes, directive `@startuml UC_SkillSwap_Complet` — c'est-à-dire au texte qui a réellement produit l'image. Elle déclarait **49 cas d'utilisation pour 37 routes**, sans jamais indiquer de méthode, de chemin ni de contrainte d'accès.

Écart	Ce que montrait la figure	Ce que montre le code
Contrainte d'évaluation	« Évaluer un membre (suivi mutuel requis) ».	<code>profile.router.ts:92</code> monte <code>requireFollow</code> , qui interroge

		<code>followerId: connectedUser,</code> <code>followedId: targetId</code> — <code>conv.middleware.ts:32-37</code> . Un seul sens : il suffit de suivre la personne pour la noter, la réciproque n'est pas exigée. <code>requireMutualFollow</code> existe bien, <code>conv.middleware.ts:48-76</code> , mais n'est monté sur aucune route .
Acceptation des CGU	Cas d'usage « Accepter les CGU », rattaché à la création de compte.	Aucune trace : ni champ dans <code>validation/auth.validation.ts</code> , ni route, ni page. Le schéma d'inscription ne comporte aucun consentement.
Niveau de compétence	Cas d'usage « Définir le niveau », rattaché à l'ajout d'une compétence.	<code>model UserHasSkill</code> — <code>schema.prisma:75-85</code> — ne porte que <code>userId</code> et <code>skillId</code> . Aucune occurrence de <code>level</code> , <code>niveau</code> ou <code>proficiency</code> dans tout le schéma.
Routes sans cas d'usage	Six routes réelles n'apparaissent nulle part dans la figure.	<code>POST /auth/refresh,</code> <code>GET /auth/me,</code> <code>DELETE /profiles/avatar,</code> <code>PATCH /conversations/:id/message/:messageId,</code> <code>GET /skills,</code> <code>GET /availabilities.</code>

Neuf autres cas de la figure sont des sous-étapes d'interface reliées en `<<include>>` — « Saisir un mot-clé », « Voir les résultats », « Donner une note » — qui décrivent un écran et non un service. Ils ne sont pas des écarts de fond, mais ils expliquent l'essentiel du surnombre : 49 contre 37.

6.6.2. Acteurs et cas d'utilisation principaux

Visiteur non-authentifié — peut consulter les profils publics, parcourir les catégories, et bien sûr s'inscrire ou se connecter. Aucune action transactionnelle (message, suivi, évaluation) ne lui est accessible.

Membre authentifié — accède à l'ensemble des fonctionnalités MVP : édition complète de son profil (compétences, intérêts, disponibilités, avatar), recherche d'autres membres avec filtre par catégorie, suivi / désuivi de membres, ouverture de conversations avec les membres suivis, envoi et clôture de conversations, évaluation des membres qu'il suit et qu'il n'a pas déjà évalués.

Aucun rôle d'administration n'est implémenté. La table `role` existe et chaque utilisateur y est rattaché par une clé étrangère `role_id` non nulle, mais l'énumération `RoleOfUser` ne contient qu'une seule valeur, `Membre` — vérifiable en base par

`SELECT enumLabel FROM pg_enum`. Le `roleId` est bien embarqué dans le JWT et exposé sur la requête par le middleware `checkAuth`, mais il n'est lu par aucun middleware ni contrôleur pour prendre une décision d'autorisation.

La table `role` constitue donc un **point d'extension du modèle de données**, prêt à accueillir d'autres valeurs, et non un mécanisme actif : la modération est une dette assumée, reportée en V2. L'autorisation réellement en vigueur en V1 est binaire (visiteur / membre) et repose sur trois mécanismes indépendants du rôle : l'authentification (`checkAuth`), la propriété de la ressource (`isOwner`) et la relation sociale (`requireFollow`).

Le périmètre MVP couvre ainsi l'essentiel des cas d'utilisation visiteur et membre, l'effort de la promotion ayant été concentré sur le coeur produit.

6.7. Diagramme du détail du cas d'utilisation le plus significatif

Le cas d'utilisation le plus significatif retenu est **l'envoi d'un message à un membre suivi** : il combine l'ensemble des prérequis fonctionnels de la plateforme (authentification, suivi, conversation), mobilise les deux canaux de communication (REST pour la création de conversation, Socket.IO pour l'envoi et la réception), et illustre les patterns techniques les plus défendables du projet (optimistic UI, cloisonnement par rooms, gating métier). Ce choix résulte d'un audit comparatif documenté qui l'identifie comme la seule fonctionnalité couvrant exhaustivement les axes du référentiel⁹.

⁹Audit interne : `docs/audits/feature-inventory-cda.md`. Ce cas d'utilisation est repris en section 8 sous l'angle technique fin et en section 11 sous l'angle comportemental observable.

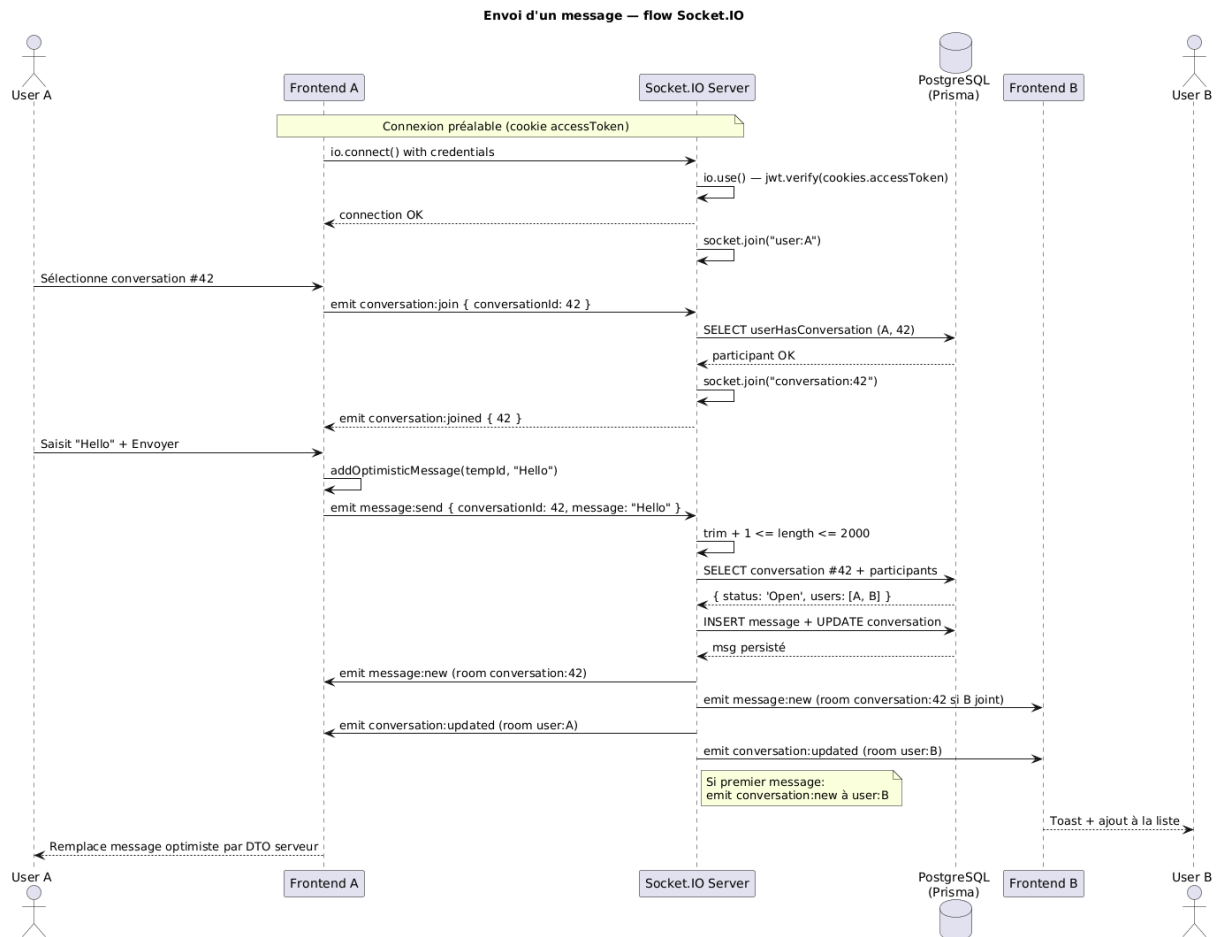


Fig. 16. – Diagramme de séquence — envoi d'un message d'Alice vers Bob. Six lignes de vie : Alice et son frontend, le serveur Socket.IO, la base PostgreSQL accédée via Prisma, le frontend de Bob et Bob. Les flèches en pointillés représentent les retours synchrones ; les flèches pleines, les events asynchrones poussés par le serveur.

6.7.1. Lecture du diagramme

Le diagramme illustre le pipeline complet en huit étapes :

1. **Connexion préalable** : à l'authentification, le client Socket.IO se connecte avec le cookie `accessToken` ; le serveur vérifie le JWT et inscrit la socket dans la room personnelle `user:Alice`.
2. **Sélection d'une conversation** : Alice sélectionne la conversation 42 dans sa liste. Le frontend émet `conversation:join` ; le serveur valide qu'Alice est bien participante de cette conversation et l'inscrit dans la room `conversation:42`.
3. **Saisie et envoi optimistic** : Alice rédige un message et clique « Envoyer ». Le frontend ajoute immédiatement le message dans la liste locale avec un identifiant temporaire négatif, puis émet `message:send` au serveur sans attendre.
4. **Validation serveur** : le serveur vérifie la longueur du message (1 à 2000 caractères), le statut de la conversation (refusé si `Close`), et la qualité de participant de l'émetteur.
5. **Persistance** : un `Promise.all` Prisma crée le message en table `message` et met à jour le `updated_at` de la `conversation` parent.
6. **Diffusion ciblée** : le serveur émet `message:new` à la room `conversation:42` (Alice et Bob s'ils sont tous deux dans la room), et `conversation:updated` aux rooms `user:Alice` et `user:Bob`.

7. **Cas particulier — premier message** : si c'est le premier message de la conversation, le serveur émet en plus `conversation:new` à la room `user:Bob`, déclenchant un toast d'apparition côté Bob.
8. **Réconciliation côté Alice** : Alice reçoit `message:new` ; le filtre côté hook ignore le retour serveur lorsque `sender.id === user.id` (l'optimistic local reste en place comme affichage final).

Le détail technique de chaque étape — code des handlers, structure des events, gestion d'erreurs — est traité en sous-section 8.4 ; le présent diagramme se contente d'en donner la vue comportementale macro.

7. Spécifications techniques

La présente section couvre les choix techniques transversaux du projet — stack, architecture, patterns et sécurité de niveau infrastructure. Les déclinaisons fonctionnelles spécifiques à la messagerie (auth Socket, gating métier, cloisonnement par rooms) sont traitées en section 8.

7.1. Stack technique

Le projet adopte une stack **TypeScript de bout en bout**, de la base de données au navigateur, ce qui réduit la surface d’erreurs d’intégration et autorise une seule boîte à outils mentale pour toute l’équipe.

Composant	Technologie	Version
Frontend	Next.js (App Router) + TypeScript + Tailwind CSS + shadcn/ui	16.1.1
Backend API	Express + TypeScript + Zod	5.2.1
ORM	Prisma Client	7.2.0
Base de données	PostgreSQL	16
Recherche full-text	Meilisearch	server 1.6 / client 0.55.0
Temps réel	Socket.IO (intégré au serveur HTTP Express)	4.8.3
Infrastructure	Docker Compose + Nginx (reverse proxy + TLS)	—
Tests backend	Node Test Runner natif (sans framework tiers)	—
Tests frontend	Aucun outil de test dans le livrable (Vitest et Playwright planifiés — cf. ADR-010)	—

7.2. Choix architecturaux — synthèse des ADRs

Dix décisions d’architecture ont été formalisées en ADRs (Architecture Decision Records) pendant le projet. La décision d’adopter Socket.IO pour le temps réel a été formalisée a posteriori dans un onzième ADR rédigé pour ce dossier¹⁰. Synthèse :

N°	Sujet	Décision retenue
001	Framework frontend	Next.js (App Router) — SSR pour le SEO des profils publics, écosystème React maîtrisé, configuration de production simple.

¹⁰Les dix ADRs du projet — 001-nextjs.md à 010-testing-strategy.md — se trouvent dans docs/documentation-implementation/arc42/09-decisions/ du dépôt d’équipe (branche Documentation). Le onzième, 011-socket-io.md, a été rédigé le 8 mai 2026 pour ce dossier : il formalise après coup un choix technique réellement en production depuis janvier 2026. Chaque ADR documente le contexte, les options envisagées, la décision retenue et ses conséquences.

002	Styling frontend	Tailwind CSS + shadcn/ui — utility-first, cohérence visuelle, composants accessibles par défaut.
003	Accès aux données	Prisma — ORM type-safe, migrations versionnées, génération automatique du client TypeScript.
004	Gestion d'état serveur	TanStack Query avait été envisagé (ADR-004) mais n'a finalement pas été intégré ; la gestion des données côté client s'appuie sur une composition de hooks React natifs.
005	Validation des entrées	Zod — schémas type-safe côté serveur et côté client, volontairement dupliqués faute de package partagé ; messages d'erreur explicites via un middleware Express maison de six lignes.
006	Architecture des composants UI	Atomic Design — atoms / molécules / organismes / pages, lisibilité accrue.
007	Authentification	JWT + cookies httpOnly + refresh token rotatif — pas d'exposition côté client, rotation automatique à chaque refresh.
008	Recherche full-text	Meilisearch — performance et simplicité d'API supérieures à la recherche full-text native de PostgreSQL, ressources opérationnelles maîtrisées.
009	Stratégie de bascule	Mock → API — développement front sur mocks réalistes en début d'apothéose, puis migration progressive vers l'API réelle jusqu'au retrait complet des mocks.
010	Stratégie de tests	Pyramide diversifiée décidée, chaque outil sur son étage : TypeScript (types), Storybook (composants UI), Vitest (hooks et utilitaires), Playwright (E2E), TypeDoc pour la documentation d'API. Hors l'étage TypeScript, acquis de fait, aucun de ces quatre outils n'a été intégré au livrable ; les tests effectivement écrits sont côté backend — sept specs d'intégration au Node Test Runner natif.
011	Communications temps réel	Socket.IO — auth par cookie partagé, modèle de rooms (<code>user:X</code> + <code>conversation:Y</code>), serveur unique avec l'API REST.

7.3. Patterns transversaux

Le code est structuré selon trois patterns récurrents qui assurent homogénéité et testabilité.

Backend en couches. Chaque domaine fonctionnel est décliné dans la même séquence : un **router** expose les endpoints, des **middlewares** appliquent l'authentification (`checkAuth`) et les règles métier (`requireSimpleFollow`, `isOwner`, `parseNumericParams`), un **controller** traduit la requête HTTP en appel de service, le **service** contient la logique

métier et appelle Prisma. Cette régularité permet de localiser n'importe quelle fonctionnalité backend par convention plutôt que par documentation.

Le sens des dépendances a été vérifié mécaniquement avec `dependency-cruiser` — 72 modules et 169 dépendances analysés : aucun *controller* ni *router* n'importe Prisma, et aucun service ne dépend de la couche présentation — zéro violation. Cinq modules font néanmoins exception en accédant au client Prisma hors de la couche service. Le nombre d'appels `prisma.<modèle>.<opération>` que chacun émet mesure l'ampleur réelle de l'écart : `realtime/socket.ts` — **8 requêtes**, persistance des messages temps réel sans repasser par les services REST ; `middlewares/conv.middleware.ts` — **3 requêtes**, la vérification du lien de suivi précède le controller ; `mappers/member.mapper.ts` — **1 requête**, projection vers Meilisearch ; `lib/auth.ts` — **1 requête**, écriture du refresh token à l'émission. Le cinquième, `middlewares/error.middleware.ts`, en émet **zéro** : il importe le namespace `Prisma` — majuscule initiale, les types générés — et non l'instance `prisma` du client. La règle de dépendance le signale au même titre que les autres, mais aucune requête ne part de ce fichier. Ces cinq points sont la dette d'architecture identifiée du backend ; leur cartographie est en Fig. 6.

Atomic Design côté frontend. Les composants sont rangés en quatre niveaux — atoms (boutons, inputs, badges issus de shadcn/ui), molécules (items composés réutilisables), organismes (sections de page), et pages (routes Next.js). Le périmètre messagerie illustre cette discipline : neuf organismes dans `ConversationPage/` composent les molécules `MessageBubble`, `ConversationItem` et les atoms `Button` / `Input`.

Le compte mérite d'être posé, car `components/` contient **cinq répertoires** alors que l'énumération ci-dessus n'annonce que quatre niveaux — et les deux nombres sont justes.

Trois des quatre niveaux sont bien des répertoires de `components/` : `atoms/`, `molécules/` et `organismes/`. Le quatrième, les **pages**, n'est pas dans `components/` : il est tenu par les routes de l'App Router, dans `app/`. Les deux répertoires restants, `layouts/` et `providers/` — un fichier chacun, `MainLayout.tsx` et `AuthProvider.tsx` — **ne sont pas des niveaux Atomic Design** ; ce sont des utilitaires de composition rangés au même endroit. Enfin, le niveau *templates* de la nomenclature d'origine **est absent** du projet : aucun répertoire, aucun fichier.

Soit : 3 niveaux dans `components/` + 1 hors de `components/` = les 4 niveaux ; et 3 niveaux + 2 répertoires hors nomenclature = les 5 répertoires. Ces deux derniers sont portés en gris sur la figure ci-après, précisément pour qu'on ne les compte pas comme des niveaux.

Composants du frontend (C4 niveau 3, vue 1/2) — composition de l'interface

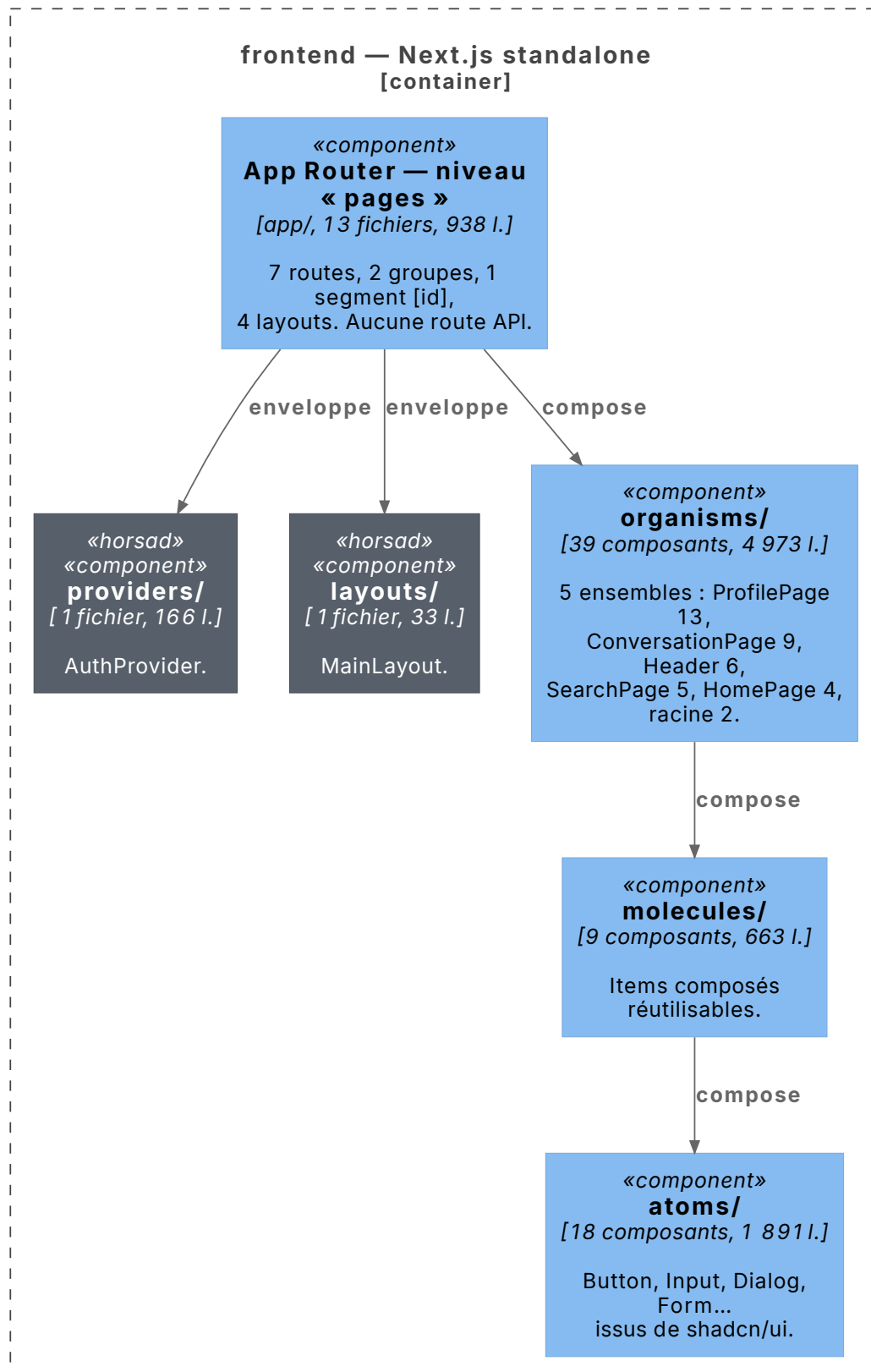


Fig. 17. – **C4 niveau 3, frontend vue 1/2 — composition de l'interface.** Chaque niveau porte son nombre de composants, barils `index.ts` exclus, selon la même convention de comptage que le lexique 15. En gris, les deux répertoires de `components/` qui ne sont pas des niveaux Atomic Design. Couture : l'App Router et les organismes sont repris en Fig. 18.

Frontend (C4 niveau 3, vue 2/2) — garde de routes et chemins de données

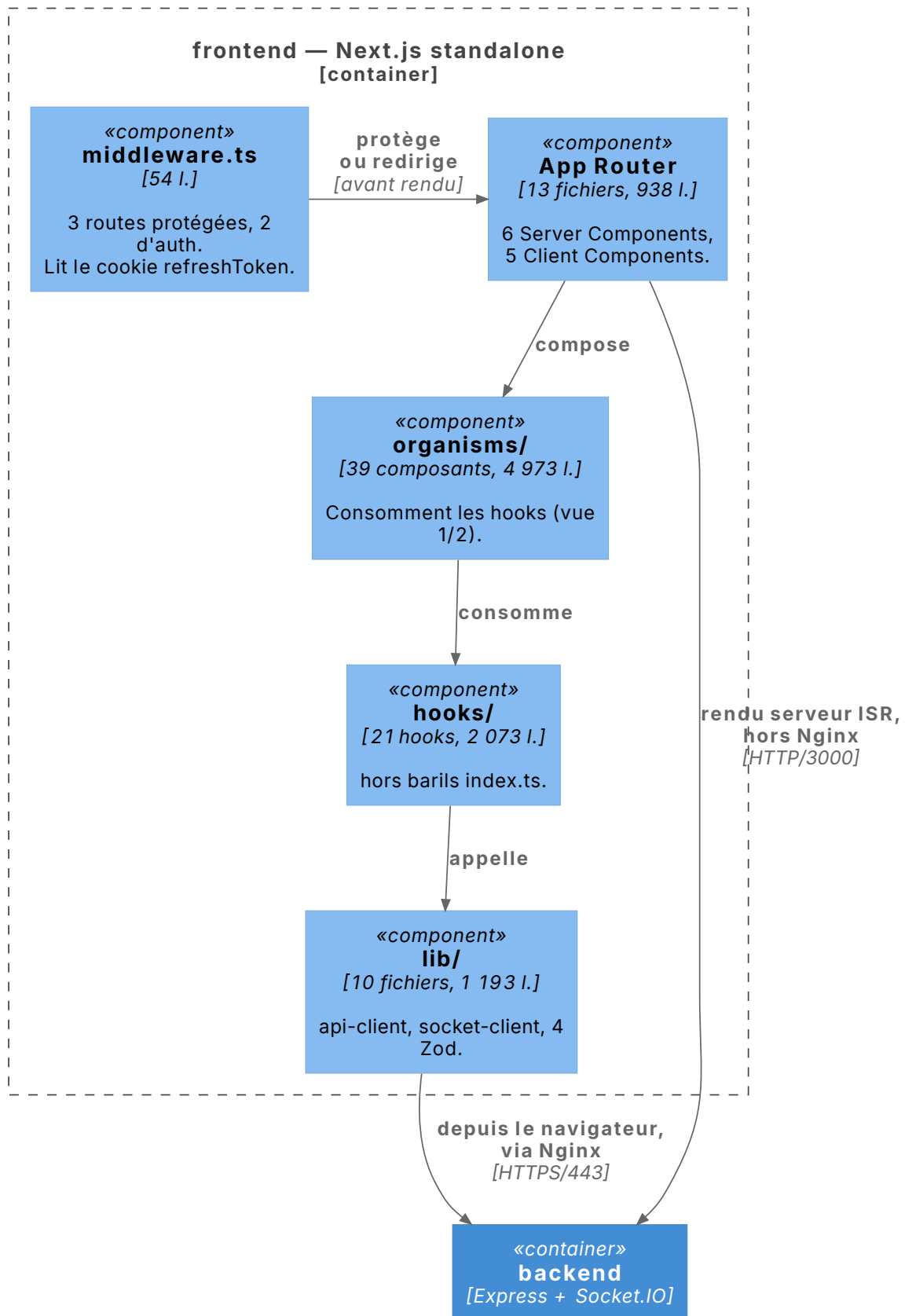


Fig. 18. – C4 niveau 3, frontend vue 2/2 — garde de routes et chemins de données.
Les deux chemins d'accès à l'API établis au niveau conteneurs : le code client passe par Nginx en HTTPS, le conteneur frontend appelle le backend directement sur le réseau Docker interne pour le rendu serveur ISR.

Type-safety end-to-end. Les types TypeScript se propagent de la base de données (générés par Prisma) jusqu'à l'UI (consommés par les hooks React). La validation des entrées s'appuie sur Zod des deux côtés, mais les schémas ne sont **pas** mutualisés : cinq schémas serveur (`backend/src/validation/`) et quatre schémas client (`frontend/src/lib/validation/`) coexistent en duplication assumée. Extraire un package partagé imposait un build TypeScript commun et une publication interne, effort écarté avant la soutenance ; la cohérence contractuelle repose donc sur une discipline de relecture, et constitue la dette technique la plus visible du projet.

7.4. Sécurité transversale

Cinq mécanismes de sécurité couvrent les routes et les échanges, à des degrés que le tableau ci-dessous précise, complétés par les contrôles spécifiques à la messagerie détaillés en section 8.

Domaine	Mesure et implémentation
Hashing mots de passe	<code>argon2</code> (paramètres par défaut de la lib <code>argon2</code> Node) appliqué dans <code>auth.service.ts:21</code> avant insertion en base. Pas de hashage côté client.
Sessions	JWT signés (<code>jsonwebtoken</code>) en cookies <code>httpOnly + secure + sameSite='strict'</code> en production (<code>auth.controller.ts:63-94</code>). Refresh token rotatif renouvelé à chaque appel <code>/api/v1/auth/refresh</code> (<code>auth.service.ts:103-108</code>), invalidation de l'ancien.
Validation des entrées	Schémas Zod appliqués par middleware déclaratif sur 18 des 37 routes applicatives (<code>body</code> , <code>query</code> , <code>params</code>) — <code>register</code> , <code>login</code> , <code>profil</code> , <code>conversations</code> , <code>recherche</code> , <code>catégories</code> . Les routeurs <code>follow</code> , <code>skill</code> et <code>availability</code> n'ont pas de schéma : <code>follow</code> ne valide que des identifiants numériques, convertis par <code>parseNumericParams</code> (<code>auth.middleware.ts:38-45</code>), tandis que <code>skill</code> et <code>availability</code> n'exposent qu'une lecture sans paramètre — écart assumé. En cas d'échec, <code>parseAsync</code> lève et le gestionnaire d'erreurs répond <code>422 Unprocessable Entity</code> avec la liste des messages de validation (<code>error.middleware.ts:32-33</code>).
Transport	HTTPS forcé en production via Nginx + certificats Let's Encrypt renouvelés automatiquement. Redirection HTTP → HTTPS systématique.
CORS	Une origine autorisée, configurée par variable d'environnement, appliquée à Express (<code>app.ts:12-17</code>) comme au handshake Socket.IO (<code>socket.ts:81-82</code>) ; rejet des requêtes hors-domaine. En production, le serveur refuse de démarrer si <code>ALLOWED_ORIGIN</code> est absente (<code>config.ts:29-38</code>).

Quatre zones de fragilité sont assumées et reportées en V2 : **Helmet installé mais non monté côté Express** (la dépendance `helmet@8.1.0` est présente dans `backend/package.json` mais n'est jamais appliquée — les en-têtes de sécurité courants `X-Frame-Options`, `X-Content-Type-Options`, `X-XSS-Protection` et HSTS sont néanmoins posés par Nginx en façade en production, ce qui couvre l'essentiel) ; **Content Security Policy stricte absente** (ni côté Express, ni côté Nginx) ; **audit accessibilité formel non réalisé** (RGAA 4.1 / WCAG 2.1 AA visés mais non mesurés via `axe-core` ou Lighthouse) ; **rate-limiting absent** (aucune protection applicative contre l'abus en volume — ni `express-rate-limit`, ni limit côté Nginx). Ces dettes ne remettent pas en cause la stabilité observée en production mais constituent les premiers chantiers de durcissement à programmer.

8. Réalisations — Messagerie temps réel

La présente section porte sur la **fonctionnalité la plus représentative** du projet, au sens du référentiel : la messagerie temps réel entre membres. Ce choix n'est pas anodin — il résulte d'un audit fonctionnel comparatif que j'ai conduit pour la préparation de ce dossier¹¹, et qui a évalué neuf fonctionnalités principales du projet selon cinq critères (cœur métier, couverture technique, visibilité du travail Lead Front, enjeux de sécurité, richesse pour la démonstration). La messagerie obtient un score de 25 sur 25, seule à couvrir l'ensemble des axes en un périmètre unique.

Plus fondamentalement, la messagerie est le moment où SkillSwap cesse d'être un annuaire de profils et devient une plateforme d'échange. Sans elle, deux membres aux compétences complémentaires ne peuvent jamais entrer en relation. Le code traduit cette centralité métier : la création d'une conversation est le seul cas du projet gardé par une règle métier non triviale (le middleware `requireSimpleFollow` impose qu'un membre suive l'autre avant toute prise de contact), tandis que la table `Conversation` est l'un des hubs du modèle relationnel, reliée à `User`, `Message`, et `Rating` via la règle de clôture.

Cette section présente les réalisations techniques selon la décomposition imposée par le référentiel : captures d'écran d'interfaces utilisateur, composants métier, composants d'accès aux données, et autres composants significatifs (handlers Socket.IO, middleware applicatif).

Le périmètre couvert dans la suite représente, en chiffres :

Élément	Volume
Schémas Prisma	3 modèles
Routes REST conv./messages	9 routes
Events Socket.IO bidirectionnels	10 events (4 entrants, 6 sortants)
Schémas de validation Zod	7 schémas (<code>conversation.validation.ts</code>)
Middlewares métier dédiés	1 fichier, 3 helpers
Composants React (<code>ConversationPage/</code>)	9 composants UI + 2 hooks locaux
Hooks React composés	8 hooks (1 façade <code>useMessaging</code> + 7 spécialisés)
Fichiers de tests dédiés	3 spec files (Node Test Runner)

Tous les extraits de code présentés ci-dessous proviennent du dépôt de production¹² et sont rendus dans un thème clair pour faciliter leur lecture par le jury.

¹¹Analyse rédigée après la période projet, dans le dépôt de documentation ; elle ne fait pas partie du livrable d'équipe.

¹²Repo : github.com/O-clock-Dublin/projet-skillswap. Ce dépôt est figé pour la soutenance ; les éventuelles dettes techniques mentionnées sont assumées comme telles et reportées en V2.

8.1. Captures d'écran d'interfaces utilisateur

Les captures qui suivent ont été réalisées en environnement de développement local, dont le seed reproduit fidèlement les états observables en production¹³. Elles couvrent les principales vues du parcours messagerie côté utilisateur connecté.

8.1.1. Liste des conversations — vue desktop

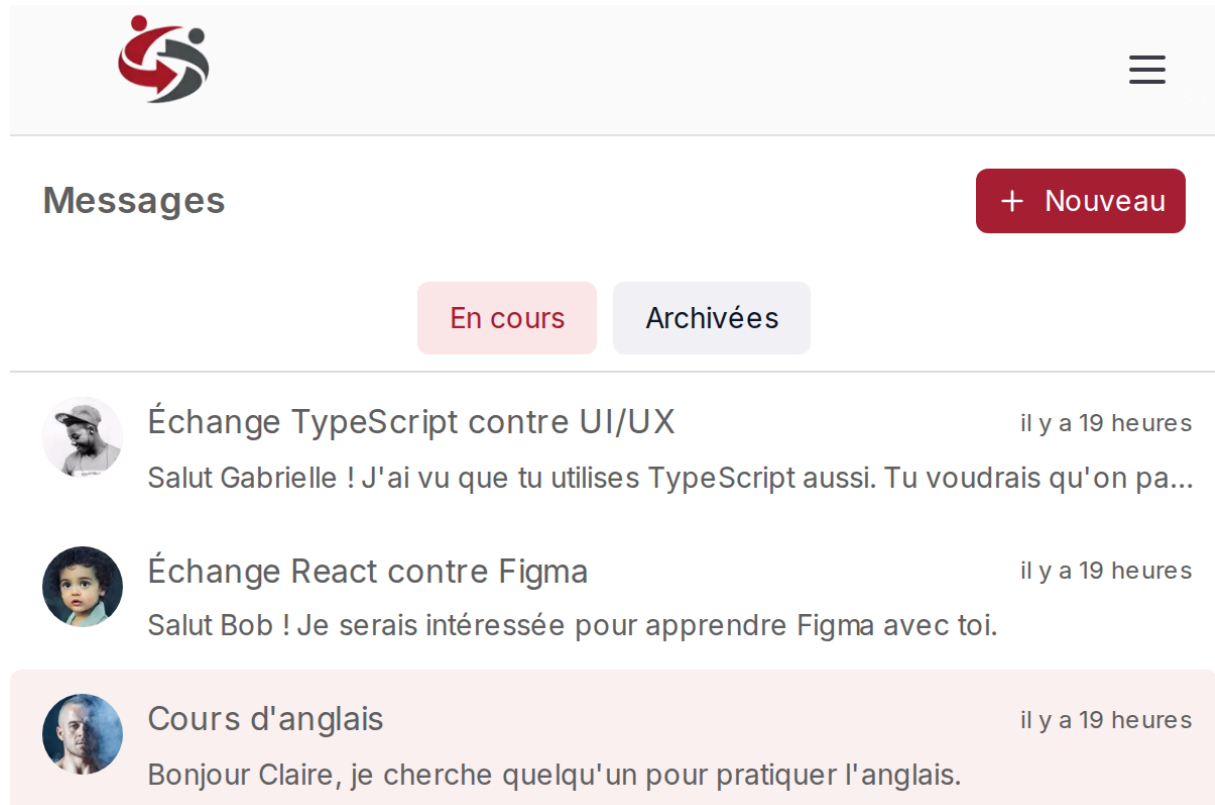


Fig. 19. – Liste des conversations de l'utilisateur connecté. Composant racine : `ConversationSection.tsx`. Chaque entrée est rendue par `ConversationItem.tsx` (molécule), avec avatar, nom du correspondant, dernier message et horodatage relatif.

L'interface présente les conversations triées par date du dernier message descendant. La sélection d'une entrée déclenche l'émission d'un event Socket.IO `conversation:join` qui inscrit le client dans la room dédiée à la conversation. L'état « vu / non vu » et le badge de message non lu sont gérés côté client via le hook `useConversationList`.

¹³Le seed dev (`backend/src/models/seeding.dev.ts`) initialise 41 utilisateurs, des relations de suivi, des conversations et des messages représentatifs. Compte de démonstration : `alice.dupont@example.com` / `password123`. Les avatars et données affichés sont fictifs.

8.1.2. Thread de message ouvert — vue desktop

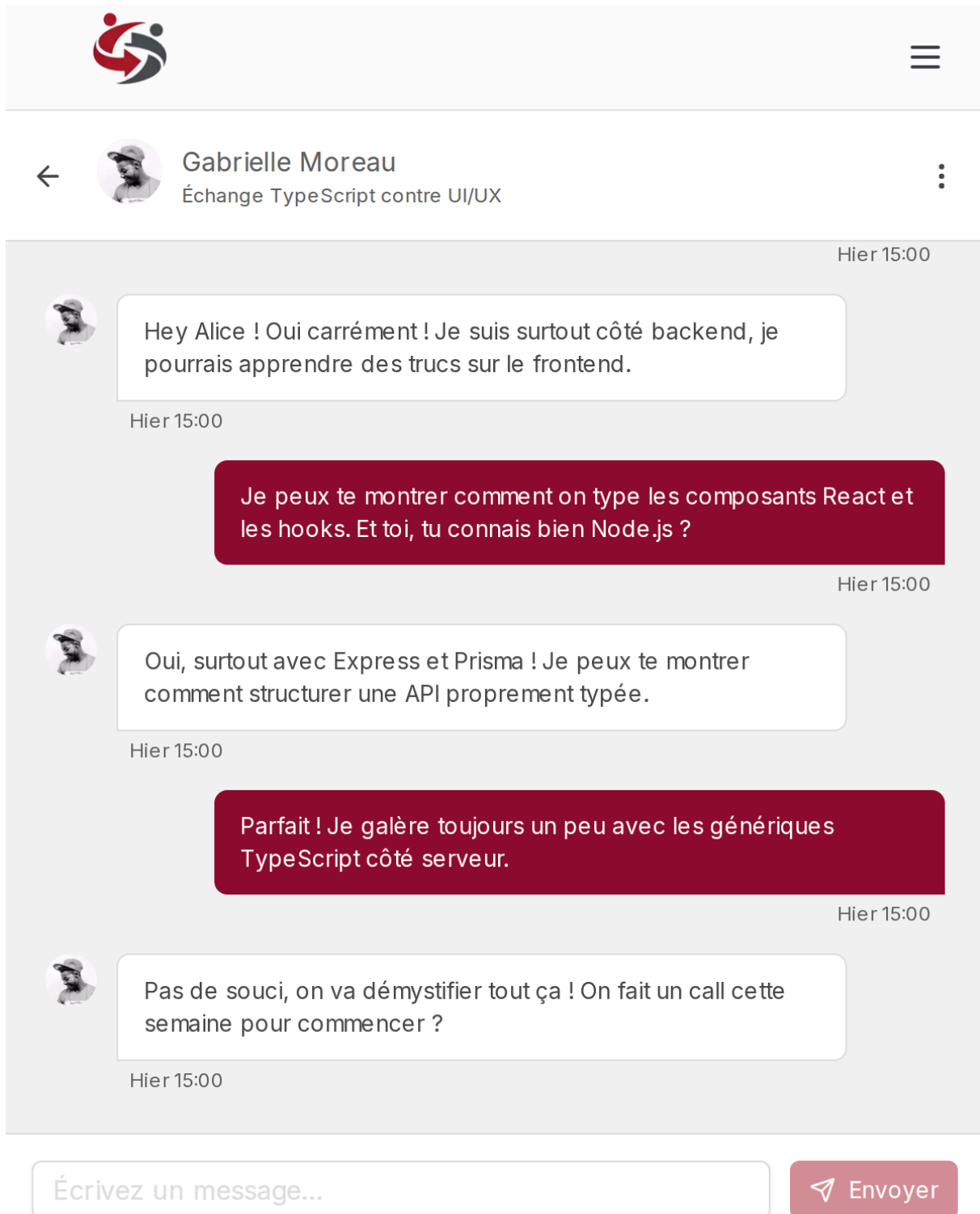


Fig. 20. – Thread d’une conversation ouverte. Composants imbriqués : `MessageThread/index.tsx` encapsule `ThreadHeader`, `MessageList` (avec composant atomique `MessageBubble` pour chaque message) et `MessageInput`. La pagination cursor-based charge les messages plus anciens au scroll vers le haut.

La liste de messages est virtualisée en pagination ascendante (cursor-based) afin de supporter des conversations longues sans charger l’intégralité de l’historique. La saisie utilisateur transite par `MessageInput.tsx`, qui émet l’événement `Socket.IO message:send` plutôt qu’une requête REST — choix architectural justifié dans la sous-section 8.4.

8.1.3. Dialogue d'évaluation après clôture

Laisser un avis ✕

Évaluez votre expérience avec Gabrielle

VOTRE NOTE *

☆ ☆ ☆ ☆ ☆

Votre commentaire

Partagez votre expérience en quelques mots...

Optionnel - Votre avis aide la communauté

Annuler Publier l'avis

Parfait ! Je galère toujours un peu avec les génériques TypeScript côté serveur.

Hier 15:00

Pas de souci, on va démystifier tout ça ! On fait un call cette semaine pour commencer ?

Hier 15:00

Écrivez un message... Envoyer

Fig. 21. – Dialogue d'évaluation accessible après clôture d'une conversation, ou depuis le menu contextuel d'un membre suivi non encore évalué (`RatingDialog.tsx`). Note de 1 à 5 et commentaire facultatif. Bloqué côté serveur si l'utilisateur a déjà évalué le membre cible (contrainte d'unicité Prisma `@@unique([evaluatorId, evaluatedId])`).

L'évaluation reste conditionnée à une relation établie : l'événement `conversation:closed` déclenche l'apparition du dialogue ; un second point d'entrée existe via le menu contextuel, gardé par la double condition `isFollowing && !isRated`. L'adaptation mobile (responsive Tailwind, navigation stack-based au lieu de split-view sur écran étroit) et le dialogue de

création (`NewConversationDialog.tsx`), avec gating front+back (`requireSimpleFollow`) sont visibles directement sur skill-swap.fr et présentés en démo orale.

8.2. Composants métier — orchestrateur `useMessaging` et optimistic UI

L'orchestration de la messagerie côté frontend repose sur la **composition de hooks React natifs** plutôt que sur une bibliothèque tierce de gestion d'état serveur. TanStack Query avait été envisagé (ADR-004¹⁴) mais n'a finalement pas été intégré : la composition de hooks natifs a été retenue à l'usage, pour trois raisons — la maîtrise fine du cycle de vie des sockets et des effets, la réduction de la surface de dépendances externes, et l'apprentissage approfondi des hooks React natifs (`useState`, `useEffect`, `useCallback`, `useRef`, `AbortController`).

8.2.1. Architecture des hooks — le pattern de composition

Le hook racine `useMessaging.ts` (139 LOC) compose sept hooks spécialisés, chacun responsable d'une dimension fonctionnelle isolée :

Hook	Responsabilité	LOC approx.
<code>useConversationList</code>	Charge la liste, expose les mutateurs (<code>addConversation</code> , <code>updateConversationLastMessage</code> , etc.)	95
<code>useSelectedConversation</code>	Mémoire l'identifiant sélectionné, dérive l'objet conversation depuis la liste	64
<code>useConversationMessages</code>	Pagination cursor-based, optimistic UI (ajout temporaire avant DTO serveur)	163
<code>useFollowedUsers</code>	Liste des utilisateurs suivis (pour la création de nouvelle conversation)	30
<code>useGlobalSocket</code>	Listeners globaux : <code>conversation:updated</code> , <code>conversation:closed</code> , <code>conversation:new</code>	94
<code>useConversationActions</code>	Handlers UI ; instancie <code>useSocket(selectedConvId)</code> pour <code>message:send</code> et <code>conversation:close</code>	184
<code>useMessagingScroll</code>	Synchronisation du scroll auto sur réception de message + scroll inverse au load-more	111

Chaque hook est testable indépendamment et maintient un périmètre de responsabilités strict. Cette discipline permet d'isoler les régressions lors d'une évolution future et de raisonner localement sur chaque mécanisme — typiquement, un bug d'optimistic UI se cherche dans `useConversationMessages` sans devoir comprendre la pagination ou la navigation.

¹⁴ `docs/documentation-implementation/arc42/09-decisions/004-tanstack-query.md` — dépôt d'équipe, branche `Documentation`.

8.2.2. Hook racine — composition

```
// frontend/src/hooks/useMessaging.ts (extrait simplifié – 139 LOC au total)
export function useMessaging() {
  const { user } = useAuth();

  const {
    conversations,
    addConversation,
    updateConversationLastMessage,
    updateConversationStatus,
    removeConversation,
  } = useConversationList();

  const {
    selectedConvId,
    setSelectedConvId,
    selectedConv,
    clearSelection,
  } = useSelectedConversation(conversations);

  const {
    messages,
    hasMore,
    loadMore,
    addOptimisticMessage,
  } = useConversationMessages({
    conversationId: selectedConvId,
    limit: 30,
  });

  const { followedUsers, fetchFollowedUsers } = useFollowedUsers();

  const {
    onConversationUpdate,
    onConversationClosed,
    onConversationNew,
  } = useGlobalSocket();

  // Réagit à un nouveau message reçu sur n'importe quelle conversation
  useEffect(() => {
    onConversationUpdate((conversationId, lastMessage) => {
      updateConversationLastMessage(conversationId, lastMessage);
    });
  }, [onConversationUpdate, updateConversationLastMessage]);

  // Réagit à la clôture d'une conversation par l'autre participant
  useEffect(() => {
    onConversationClosed((conversationId, closedBy) => {
      updateConversationStatus(conversationId, "Close");
      if (closedBy && closedBy.id !== user?.id) {
        toast.info(`${closedBy.firstname} à clôturer un échange`);
      }
      if (conversationId === selectedConvId) {
        clearSelection();
      }
    });
  }, [onConversationClosed, updateConversationStatus,
```



```

    selectedConvId, clearSelection, user]);

// Réagit à une nouvelle conversation reçue (premier message d'un autre membre)
useEffect(() => {
  onConversationNew((conversation) => {
    addConversation(conversation);
    toast.info(
      `${conversation.participant?.firstname} a démarré un nouvel échange`,
    );
  });
}, [onConversationNew, addConversation]);

// Composition complète des actions (signature réelle ; props omises ici)
const actions = useConversationActions({
  selectedConvId,
  selectedConv: selectedConvWithMessages,
  setSelectedConvId,
  addConversation,
  updateConversation,
  removeConversation,
  clearSelection,
  fetchFollowedUsers,
  addMessage,
  addOptimisticMessage,
});

return {
  conversations,
  selectedConv,
  messages,
  hasMore,
  loadMore,
  followedUsers,
  fetchFollowedUsers,
  ...actions,
};
}

```

L'effet de bord crucial est l'enregistrement des trois listeners globaux (`onConversationUpdate`, `onConversationClosed`, `onConversationNew`) qui maintiennent la liste de conversations synchronisée même quand l'utilisateur n'est pas en train de regarder la conversation concernée. Cette logique exploite le modèle de rooms Socket.IO : chaque utilisateur connecté rejoint automatiquement sa room personnelle `user:${userId}` (cf. sous-section 8.4), ce qui permet au serveur de pousser les notifications de mise à jour ciblées.

8.2.3. Optimistic UI — gestion du `tempId` négatif

```

// frontend/src/hooks/messaging/useConversationActions.ts (extrait, l.100-118)
const handleSendMessage = useCallback(
  async (content: string) => {
    if (!selectedConvId || !content.trim()) return;

    // tempId négatif : marqueur d'un message en attente de réponse serveur
    const tempId = -Date.now();

```

```

const sender = user
? {
  id: user.id,
  firstname: user.firstname,
  lastname: user.lastname,
  avatarUrl: user.avatarUrl,
}
: undefined;

addOptimisticMessage(tempId, content, sender);
socketSendMessage(content); // wrapper de socket.emit("message:send", ...)
},
[selectedConvId, socketSendMessage, addOptimisticMessage, user],
);

```

Le pattern `optimistic UI` consiste à insérer immédiatement le message dans la liste locale avec un identifiant négatif (`-Date.now()`), puis à laisser l'événement `Socket.IO message:new` émis par le serveur ajouter le DTO définitif (avec identifiant entier positif assigné par PostgreSQL). La déduplication exploite un filtre côté listener `onMessage` du hook `useConversationActions` : si `newMessage.sender?.id === user?.id`, le message est **ignoré** lors du retour serveur — l'optimistic local reste en place et sert d'affichage final.

```

// useConversationActions.ts:57-65 - déduplication des messages propres
useEffect(() => {
  onMessage((newMessage) => {
    if (newMessage.sender?.id === user?.id) {
      return; // déjà ajouté en optimistic, on ignore le retour serveur
    }
    addMessage(newMessage);
  });
}, [onMessage, addMessage, user?.id]);

```

L'avantage utilisateur est immédiat : la latence perçue de l'envoi est quasi-nulle, ce qui correspond à l'expérience attendue d'une messagerie moderne. Le compromis assumé : en cas d'échec réseau silencieux, le message optimistic reste affiché avec son `tempId` négatif (pas de réconciliation automatique vers un état d'erreur dans le code actuel). Une amélioration V2 consiste à introduire un timeout de réconciliation et un état visuel « en cours d'envoi / échec » — point identifié dans la dette technique en fin de section.

8.3. Composants d'accès aux données — services Prisma

L'accès à la base de données passe par Prisma comme ORM type-safe (cf. ADR-003¹⁵). Les services backend exposent les opérations métier en masquant la mécanique des requêtes, ce qui permet aux contrôleurs et handlers `Socket.IO` de se concentrer sur la logique de validation et la diffusion des événements.

8.3.1. Création atomique d'un message + mise à jour de la conversation

L'envoi d'un message implique deux écritures distinctes : insertion en table `Message` et mise à jour du champ `updatedAt` de la `Conversation` associée (pour la conserver en tête

¹⁵ [docs/documentation-implementation/arc42/09-decisions/003-prisma.md](https://github.com/SkillSwap/docs/blob/main/documentation-implementation/arc42/09-decisions/003-prisma.md) — dépôt d'équipe, branche `Documentation`.

de liste). Plutôt qu'une transaction, le code utilise un `Promise.all` qui parallélise les deux requêtes — acceptable car elles ne sont pas mutuellement dépendantes, et la durée de fenêtre d'incohérence est de l'ordre de la milliseconde.

```
// backend/src/realtime/socket.ts (extrait du handler message:send)
const [msg] = await Promise.all([
  prisma.message.create({
    data: {
      conversationId,
      senderId: userId,
      receiverId,
      content,
    },
    select: {
      id: true,
      content: true,
      createdAt: true,
      sender: {
        select: {
          id: true,
          firstname: true,
          lastname: true,
          avatarUrl: true,
        },
      },
    },
  }),
  prisma.conversation.update({
    where: { id: conversationId },
    data: { updatedAt: new Date() },
  }),
]);
```

Le `select` explicite est un pattern récurrent dans le code prod : il limite les champs retournés au strict nécessaire pour le DTO réseau, ce qui évite à la fois le sur-fetch (gain de performance) et la fuite involontaire de champs sensibles (par exemple `password`, `email`, ou les champs liés au système de tokens). Cette discipline transversale relève autant de la performance que de la sécurité.

8.3.2. Pagination cursor-based des messages

```
// backend/src/services/message.service.ts (extrait -
getConversationMessagesService)
const limit = Math.min(Math.max(params.limit ?? 50, 1), 100);

const messages = await prisma.message.findMany({
  where: {
    conversationId,
    // Cursor : récupère uniquement les messages plus anciens que le cursor
    ...(params.cursor ? { id: { lt: params.cursor } } : {}),
  },
  orderBy: { id: 'desc' },
  take: limit,
  select: {
    id: true,
    content: true,
```

```

    createdAt: true,
    sender: {
      select: { id: true, firstname: true, lastname: true, avatarUrl: true },
    },
  },
});

// Reverse pour ordre chronologique côté front (oldest → newest)
const data = messages.slice().reverse().map(/* ... mapping DTO ... */);

// nextCursor null = plus de messages à charger
const nextCursor =
  messages.length > 0 ? messages[messages.length - 1].id : null;

```

Le choix d’une pagination **cursor-based** (par identifiant de message, clause Prisma `id: { lt: cursor }`) plutôt qu’**offset-based** (par numéro de page) est motivé par deux considérations : la stabilité face à l’insertion de nouveaux messages pendant la navigation (un offset sauterait des messages, un cursor reste stable), et la performance sur des tables longues (l’offset force PostgreSQL à scanner les lignes ignorées, le cursor permet un index seek direct via la clé primaire). Pour une messagerie où les messages s’accumulent en continu, le cursor est l’approche correcte. La clause `limit` est bornée à `[1, 100]` côté serveur (`Math.min(Math.max(limit ?? 50, 1), 100)`) pour empêcher tout abus côté client.

8.3.3. Schéma Prisma — extrait des modèles principaux

Le schéma Prisma intégral des trois modèles directement liés à la messagerie (`Conversation`, `UserHasConversation`, `Message`) plus `Follow` (gating) est reproduit en **annexe A**. Les choix de modélisation traduisent plusieurs décisions architecturales : la table de jonction `UserHasConversation` permet une relation N-N entre utilisateurs et conversations (donc support natif de conversations de groupe en V2 sans changement de schéma), la cascade `onDelete: Cascade` garantit l’absence de messages orphelins en cas de suppression d’utilisateur, le statut `StatusOfConversation` (`Open` / `Close`) matérialise le cycle de vie de l’échange jusqu’à sa clôture (qui déclenche le dialogue d’évaluation), et la pagination cursor-based exploite la clé primaire `id` (autoincrément) comme curseur naturel — sans nécessiter d’index composite supplémentaire. La limite de longueur de message (2000 caractères) est appliquée côté serveur dans le handler Socket.IO et dans le schéma Zod `CreateMessageSchema`, sans contrainte SQL explicite — trade-off de simplicité assumé.

8.4. Autres composants — Socket.IO server et middleware métier

Le chemin d’exécution complet de `message:send`, depuis la saisie dans l’interface jusqu’aux trois diffusions, est représenté en Chapitre 16.9 — trois vues, chaque nœud adossé à un fichier et une ligne.

Cette dernière sous-section regroupe les composants qui ne relèvent ni strictement de l’UI, ni des services d’accès aux données : le serveur Socket.IO et les middlewares applicatifs. Ils incarnent l’épine dorsale temps réel de la messagerie et la garde métier qui en conditionne l’accès.

8.4.1. Authentification Socket.IO par cookie JWT

L'authentification du client Socket.IO réutilise le même cookie HTTP-only `accessToken` que les routes REST, plutôt qu'un mécanisme distinct (token en query string par exemple). Ce choix élimine une surface d'attaque (le token n'est jamais exposé dans une URL ou loggué par un proxy) et simplifie la session côté frontend (un seul flux de rafraîchissement à gérer).

```
// backend/src/realtime/socket.ts (extrait, 1.88-122)
io.use((socket, next) => {
  try {
    const cookieHeader = socket.handshake.headers.cookie ?? "";
    const cookies = cookie.parse(cookieHeader);
    const token = cookies.accessToken;
    if (!token) return next(new Error("Unauthorized"));

    const decoded = jwt.verify(token, config.jwtSecret) as jwt.JwtPayload;
    const userId = Number(decoded.id ?? decoded.userId ?? decoded.sub);

    if (!Number.isInteger(userId) || userId <= 0) {
      return next(new Error("Unauthorized"));
    }

    socket.data.userId = userId;
    next();
  } catch {
    next(new Error("Unauthorized"));
  }
});

io.on("connection", (socket) => {
  // Inscription automatique à la room personnelle
  socket.join(`user:${socket.data.userId}`);

  socket.on("conversation:join", async (payload) => {
    /* validation + inscription room conversation */
  });

  socket.on("message:send", async (payload) => {
    /* validation + persistance + diffusion */
  });

  socket.on("conversation:close", async (payload) => {
    /* validation + update status + diffusion */
  });

  socket.on("conversation:leave", (payload) => {
    /* leave room conversation (silencieux) */
  });
});
```

À la connexion, chaque socket rejoint automatiquement sa room personnelle `user:${userId}`. Cette room est utilisée par le serveur pour pousser des notifications (nouvelle conversation reçue, mise à jour de la liste, fermeture par l'autre participant) qui doivent atteindre l'utilisateur indépendamment de la conversation qu'il est en train de visualiser.

8.4.2. Middleware métier `requireSimpleFollow` — gating de la création de conversation

```
// backend/src/middlewares/conv.middleware.ts:78-115
export const requireSimpleFollow = async (
  req: Request,
  _res: Response,
  next: NextFunction,
) => {
  try {
    const receiverId = Number(req.body.receiverId) || Number(req.paramsId);
    const senderId = req.userId;

    if (!Number.isInteger(receiverId) || receiverId <= 0) {
      return next(
        new BadRequestError('receiverId is required and must be a valid id.'),
      );
    }

    if (receiverId === senderId) {
      return next(
        new BadRequestError('You cannot start a conversation with yourself.'),
      );
    }

    const follow = await prisma.follow.findFirst({
      where: { followerId: senderId, followedId: receiverId },
      select: { followerId: true, followedId: true },
    });

    if (!follow) {
      return next(new ForbiddenError('You can only message users you follow.'));
    }

    next();
  } catch (e) {
    next(e);
  }
};
```

Ce middleware applique trois contrôles successifs : la validité du `receiverId` (entier positif), l'absence d'auto-conversation (impossible de se contacter soi-même), et la présence d'une relation de suivi `Follow` du `sender` vers le `receiver`. Cette règle est le verrou métier d'entrée du produit : aucune conversation ne peut naître vers un membre qu'on ne suit pas, et le contournement d'un front compromis est bloqué au niveau backend. Le contrôle est en revanche ponctuel : il s'exécute à la création de la conversation, et n'est rejoué ni par le service REST d'envoi (`message.service.ts:114-171`) ni par le handler Socket (`socket.ts:167-347`). Un désabonnement postérieur laisse donc la conversation pleinement fonctionnelle. Rendre la garantie permanente supposerait de révérifier le lien `Follow` à chaque envoi — dette identifiée, reportée en V2.

8.4.3. Modèle de rooms Socket.IO

Le serveur exploite deux types de rooms simultanément, chacune avec un rôle bien défini :

Room	Membres	Usage
<code>user:\${userId}</code>	Toutes les sockets du même utilisateur (multi-onglets)	Notifications globales : <code>conversation:updated</code> , <code>conversation:closed</code> , <code>conversation:new</code>
<code>conversation:\${id}</code>	Participants ayant émis <code>conversation:join</code>	Diffusion ciblée : <code>message:new</code> , <code>conversation:joined</code> , <code>conversation:closed</code>

Le cloisonnement par rooms réalise un contrôle d'accès au niveau du transport : un message émis dans la room `conversation:42` n'est diffusé qu'aux participants ayant explicitement rejoint cette room, à condition qu'ils soient déjà passés par la validation participant du handler `conversation:join`. Un membre tiers ne peut pas écouter passivement les messages d'autrui.

8.4.4. Diagramme de séquence — envoi d'un message

Le flux complet d'un envoi de message, depuis la saisie utilisateur jusqu'à la confirmation chez les deux participants, est décrit dans le diagramme de séquence ci-après. Je l'ai produit avec PlantUML en août 2026 pour ce dossier, en le décalquant du handler `message:send`¹⁶.

¹⁶Le dépôt d'équipe comporte bien un fichier `docs/uml/sequence/conversation.puml`, mais il documente le chargement REST de la page de messagerie, non le flux d'envoi temps réel. Source de la présente figure : `backend/src/realtime/socket.ts:167-347`.

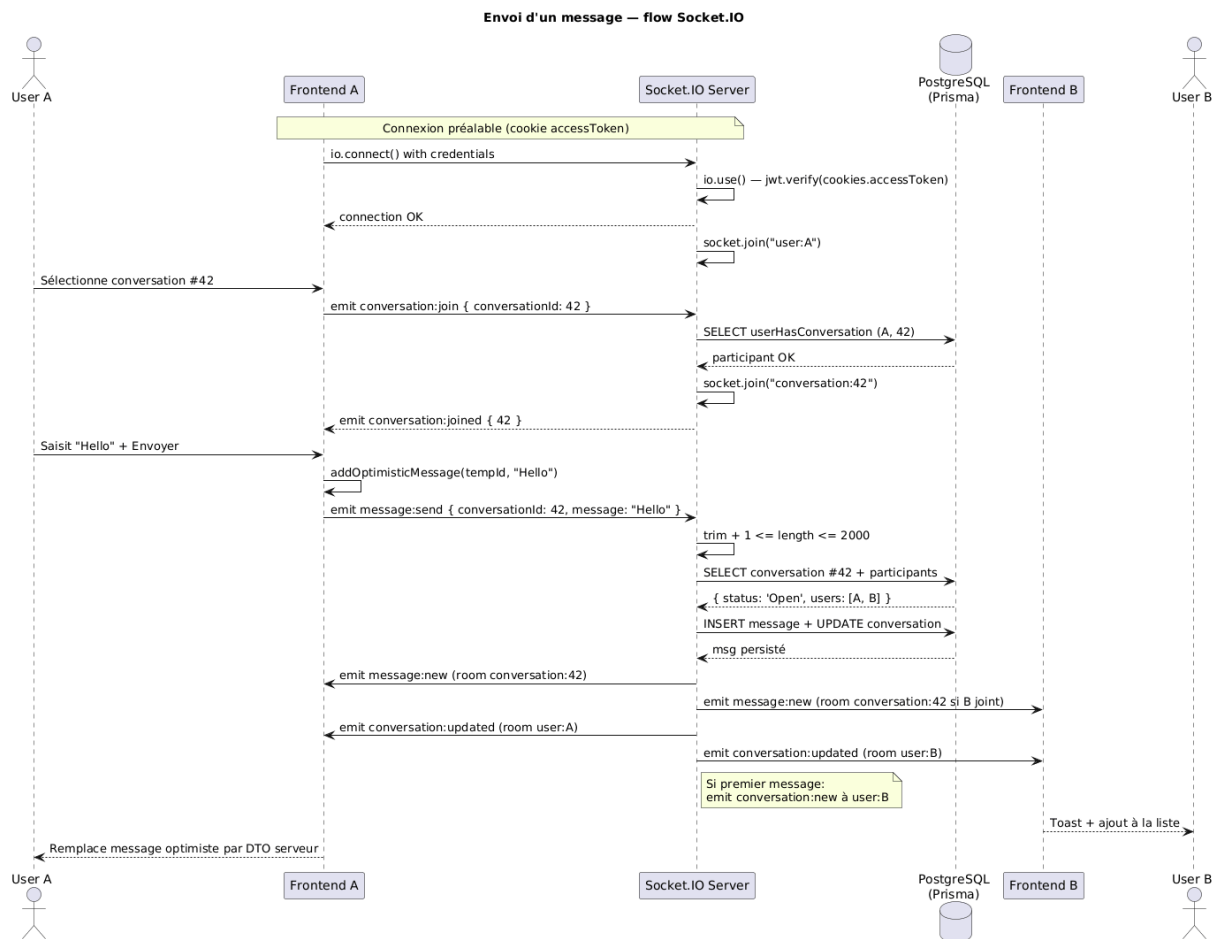


Fig. 22. – Diagramme de séquence de l'envoi d'un message — flux complet : authentification socket, optimistic UI, validation serveur, persistance Prisma, diffusion multi-rooms.

8.5. Bilan technique de la section

Cette section a couvert en profondeur la fonctionnalité messagerie selon les quatre axes imposés par le référentiel — interfaces utilisateur, composants métier, accès aux données, autres composants. Les compétences professionnelles mobilisées dans la réalisation se cartographient ainsi :

Compétence CDA	Mise en œuvre
CP2 — Développer des interfaces utilisateur	9 composants React (<code>ConversationPage/</code>) avec composition Atomic Design (atoms via <code>shadcn/ui</code> , molécules <code>ConversationItem</code> et <code>MessageBubble</code> , organisme <code>MessageThread</code> sous-composé en 5 fragments), responsive Tailwind, accessibilité ARIA.
CP3 — Développer des composants métier	Composition de 8 hooks React (orchestrateur + 7 spécialisés), pattern optimistic UI avec <code>tempId</code> négatif, gestion fine du cycle de vie des sockets via <code>useSocket(conversationId)</code> .
CP6 — Définir l'architecture logicielle	Architecture hybride REST + WebSocket assumée et documentée (ADR-011, formalisé a posteriori pour ce dossier), séparation claire des responsabilités entre

	handlers Socket et services métier, modèle de rooms à deux niveaux (room utilisateur + room conversation).
CP7 — appliqué à la messagerie	3 modèles dédiés (<code>Conversation</code> , <code>UserHasConversation</code> , <code>Message</code>), cascade de suppression sécurisée, pagination cursor-based exploitant la PK <code>id</code> , longueur de message bornée côté serveur (2000 caractères) et côté Zod.
CP8 — Développer des composants d'accès aux données	Services Prisma type-safe avec <code>select</code> explicite anti-overfetch, pagination cursor-based, parallélisation <code>Promise.all</code> sur écritures indépendantes.

8.5.1. Dette technique assumée

Les zones de fragilité connues sont assumées et reportées en V2 :

- **Handler** `message:send` **dense** (`socket.ts:167-347`) — fetch, vérifs, persistance et 3 émissions concentrés dans une seule fonction ; refactorisation vers `services/message.service.ts` prévue en V2.
- **Validation Socket non-Zod** — bornes manuelles (`Number.isInteger`, longueur) au lieu de schémas Zod ; trade-off latence assumé, harmonisation possible en V2.
- **Aucun test automatisé côté frontend** — le livrable ne contient ni test unitaire, ni test E2E : le scénario `alice→bob` à deux navigateurs n'est validé que manuellement (cf. section 11). L'outillage correspondant (Vitest, Playwright) est décidé dans l'ADR-010 mais n'a pas été intégré avant la soutenance.
- **Couverture non mesurée** — aucun outil de couverture n'est branché, ni en local ni en CI ; les sept fichiers de tests backend ne sont donc pas quantifiés.

Ces dettes n'ont pas affecté la stabilité observée en production sur skill-swap.fr depuis la mise en service.

9. Éléments de sécurité

Les choix de sécurité transversale (hashing argon2, JWT en cookies `httpOnly`, CORS, HTTPS) ont été présentés en sous-section 7.4. La présente section porte sur les contrôles **spécifiques à la messagerie** — la fonctionnalité retenue comme la plus représentative — qui combinent authentification du canal WebSocket, autorisation contextuelle à chaque event, validation des entrées et cloisonnement par rooms. Ces contrôles forment une chaîne où chaque maillon est nécessaire pour qu'un message émis par un membre parvienne effectivement, et seulement, à son destinataire légitime.

9.1. Authentification du canal Socket.IO

Le serveur Socket.IO réutilise le cookie `accessToken` posé par l'authentification REST plutôt qu'un mécanisme distinct (token en query string ou auth manuelle). Au handshake, un middleware de session (`io.use`) extrait le cookie, vérifie la signature JWT et stocke l'identifiant utilisateur dans `socket.data.userId`. En cas d'échec — cookie absent, signature invalide, identifiant non entier positif — la connexion est rejetée avec `next(new Error('Unauthorized'))` avant qu'aucun event applicatif ne puisse être émis ou reçu.

Ce choix élimine deux surfaces d'attaque par rapport à un token en URL : le token n'est jamais exposé dans une chaîne de connexion ni loggué par un proxy intermédiaire, et la session reste cohérente entre les flux REST et WebSocket (un seul flux de rafraîchissement à gérer côté frontend). Le code complet de ce handshake est reproduit en **annexe B**.

9.2. Contrôles d'accès appliqués à chaque event

Contrôle	Effet	Réf. code
Auth handshake	Refus de connexion si JWT manquant ou invalide.	<code>socket.ts: 88-122</code>
Vérif participant <code>conversation:join</code>	Refus de rejoindre une room conversation si l'utilisateur n'est pas dans <code>UserHasConversation</code> pour cette conversation.	<code>socket.ts: 136-152</code>
Validation bornes message	Refus si <code>content</code> absent, vide après <code>trim()</code> , ou supérieur à 2000 caractères.	<code>socket.ts: 167-186</code>
Refus si conversation <code>Close</code>	Aucun message accepté sur une conversation clôturée — le statut est lu en base avant persistance.	<code>socket.ts: 214-220</code>
Vérif participant <code>message:send</code>	Refus si l'émetteur n'est pas participant de la conversation cible (recheck explicite, indépendant de <code>conversation:join</code>).	<code>socket.ts: 222-230</code>
Cloisonnement par rooms	Diffusion ciblée — un message émis dans <code>conversation:42</code> n'atteint que les sockets ayant rejoint cette room précise. Inscription à la room personnelle à la connexion ;	<code>socket.ts: 128, 155</code>

	inscription à la room conversation seulement après vérification participant.	
--	--	--

À ces six contrôles côté Socket s'ajoute **en amont** le gating métier appliqué au moment de la création de la conversation par REST : le middleware `requireSimpleFollow` (`conv.middleware.ts:78-115`) exige que l'émetteur suive le destinataire avant que la conversation puisse exister. Sans relation de suivi, aucune conversation n'est créée, donc aucun message ne peut transiter — le contrôle Socket de l'étape 5 ne sera jamais sollicité car il n'y aura pas de couple conversation/participant à vérifier. Cette antériorité du gating REST sur le pipeline Socket protège la création de la conversation, pas sa durée de vie : le lien `Follow` n'est revérifié ni par le service REST d'envoi, ni par le handler Socket, une fois la conversation créée — dette détaillée ci-dessous.

9.3. Défense en profondeur — front, back, base

Trois barrières indépendantes appliquent des contrôles complémentaires à des niveaux distincts du système, ce qui garantit que la compromission d'une couche ne suffit pas à contourner les contrôles.

Niveau	Mesures
Frontend	Le dialogue de création de conversation ¹⁷ ne propose que les utilisateurs suivis (filtrage par <code>useFollowedUsers</code>). <code>MessageInput.tsx</code> désactive le champ de saisie quand <code>conversationStatus</code> <code>===</code> <code>'Close'</code> . Ces mesures relèvent de l'UX — elles guident le membre légitime mais ne sont pas des contrôles de sécurité réels (un client compromis peut les contourner).
Backend	Les six contrôles Socket listés ci-dessus + le gating REST <code>requireSimpleFollow</code> forment la vraie barrière de sécurité . Toute requête mal formée, tout payload hors-bornes, tout participant non légitime est refusé avec une erreur explicite avant tout effet de bord.
Base de données	Trois contraintes d'unicité matérialisent au niveau base les règles non contournables : <code>@@unique([followedId, followerId])</code> sur <code>Follow</code> , <code>@@unique([evaluatorId, evaluatedId])</code> sur la table <code>evaluation</code> , clé primaire composite <code>@@id([userId, conversationId])</code> sur <code>UserHasConversation</code> . Ces contraintes prennent le relais en cas de bug applicatif (idempotence garantie sur les insertions concurrentes).

9.4. Dettes de sécurité assumées

Six zones de fragilité ont été identifiées en audit interne et sont documentées plutôt que masquées. Aucune ne remet en cause la stabilité fonctionnelle observée en production, mais toutes constituent des chantiers de durcissement à programmer en V2.

¹⁷ `frontend/src/components/organisms/ConversationPage/NewConversationDialog.tsx`

Dette	Impact et plan V2
Helmet installé non monté	La dépendance <code>helmet@8.1.0</code> est listée dans <code>backend/package.json</code> mais jamais appliquée dans <code>backend/src/app.ts</code> . Mitigation actuelle : Nginx pose en façade <code>X-Frame-Options</code> , <code>X-Content-Type-Options</code> , <code>X-XSS-Protection</code> et HSTS. V2 : ajouter <code>app.use(helmet())</code> dans Express (correction d'une ligne).
Rate-limiting absent	Aucune protection applicative contre l'abus en volume — ni <code>express-rate-limit</code> , ni <code>limit_req</code> Nginx. Risque : brute-force sur l'authentification, abus d'envoi de messages. V2 : <code>express-rate-limit</code> sur les routes <code>/api/v1/auth/{login,register,refresh}</code> (5 tentatives / 15 min par IP).
Content Security Policy absente	Aucune directive <code>Content-Security-Policy</code> posée, ni côté Express ni côté Nginx. Le frontend Next.js n'est donc pas protégé contre l'injection de scripts tiers. V2 : CSP stricte côté Nginx en mode <code>Report-Only</code> d'abord, puis enforcement après recensement des inline-scripts légitimes.
Validation Socket sans Zod	Les events Socket utilisent une validation manuelle (<code>Number.isInteger</code> , bornes de longueur) plutôt que les schémas Zod employés sur les routes REST. Trade-off latence assumé pour le handler <code>message:send</code> . V2 : harmonisation Zod si la latence mesurée le permet.
Garde d'évaluation plus faible qu'annoncée	<code>requireMutualFollow</code> est écrit et exporté — <code>middlewares/conv.middleware.ts:48-76</code> — mais monté sur aucune route . La seule garde effective de l'évaluation est <code>requireFollow</code> (<code>profile.router.ts:92</code>), qui ne vérifie qu'un lien unidirectionnel : <code>followerId: connectedUser, followedId: targetId</code> en <code>conv.middleware.ts:32-37</code> . Impact : il suffit de suivre un membre pour le noter, sans qu'il vous suive en retour — un compte peut donc noter n'importe qui après un simple follow, et le retirer ensuite. La note reste. V2 : soit monter <code>requireMutualFollow</code> sur <code>POST /profiles/:id/rating</code> si la réciprocité est la règle voulue, soit supprimer le middleware mort et aligner la documentation sur le suivi simple.
Garantie anti-spam ponctuelle, pas permanente	Le gating <code>requireSimpleFollow</code> ne s'exécute qu'à la création de la conversation : ni le service REST d'envoi (<code>message.service.ts:114-171</code>) ni le handler Socket (<code>socket.ts:167-347</code>) ne revérifient le lien <code>Follow</code> à l'envoi. Impact : un membre désabonné après coup continue de recevoir des messages sur une conversation déjà ouverte.

V2 : revérifier le lien `Follow` dans `message.service.ts` et dans le handler Socket `message:send` avant chaque envoi.

10. Plan de tests

10.1. Stratégie décidée et stratégie réalisée

L'ADR-010 formalise une pyramide de tests à plusieurs étages : le typage TypeScript comme premier filet, Storybook pour les composants d'interface, Vitest pour les hooks et utilitaires, Playwright pour les parcours de bout en bout, et TypeDoc pour la documentation d'API. L'intention était de faire porter chaque vérification par l'outil le plus adapté plutôt que de tout couvrir avec un seul.

L'écart avec la réalisation est important et doit être énoncé sans détour. Hors le typage TypeScript, acquis de fait par la stack, **aucun de ces quatre outils n'a été intégré au livrable**. Les tests effectivement écrits sont d'une autre nature : des tests d'intégration backend, non prévus comme tels par l'ADR-010, qui exercent l'API par requêtes HTTP réelles contre une base dédiée.

Étage décidé (ADR-010)	Outil prévu	État réel
Typage	TypeScript	Acquis
Composants UI	Storybook	Non intégré
Hooks et utilitaires	Vitest	Non intégré
Parcours de bout en bout	Playwright	Non intégré
Documentation d'API	TypeDoc	Non intégré
(non prévu par l'ADR)	Node Test Runner — intégration backend	Implémenté

10.2. Les tests d'intégration backend

Sept suites ont été écrites, totalisant **soixante tests**. Elles ne testent pas des fonctions isolées mais des **parcours de requête complets** : chaque test émet une requête HTTP sur l'application Express réellement instanciée, et vérifie le code de statut comme le contenu de la réponse.

Suite	Périmètre couvert	Tests
auth.controller	Inscription, connexion, rafraîchissement de jeton	8
conv	Création de conversation (dont refus si pas de suivi, et incrément de titre), consultation, retrait d'un participant, clôture	13
message	Envoi, lecture paginée, modification et suppression de message, avec les cas 404 et 422	13
profile.controller	Compétences, disponibilités, suppression de compte, et un bloc dédié à la visibilité des données	11

<code>follow.controller</code>	Abonnés, abonnements, suivi et désabonnement	8
<code>realtime/socket</code>	Handshake JWT (accepté et refusé), <code>conversation:join</code> participant et non-participant	4
<code>search.controller</code>	Recherche de membres et classement par note	3

Un point mérite d'être relevé : la suite `profile.controller` comporte un bloc explicitement nommé « Security & Data visibility », et la suite `conv` vérifie le refus de création lorsque l'émetteur ne suit pas le destinataire. Les tests ne couvrent donc pas seulement le chemin nominal : ils vérifient aussi que les règles d'accès refusent ce qu'elles doivent refuser.

Le choix de ne pas ajouter de framework tiers. L'exécution repose sur le lanceur natif de Node — `node --test` — sans Jest, Mocha ni Vitest côté backend :

```
node --test --env-file=./src/test/config/.env.test \
  --import ./src/test/config/global-setup.ts \
  --experimental-test-isolation=none ./src/**/*.spec.test.ts
```

Trois raisons justifient ce choix. D'abord **une dépendance en moins** à installer, configurer et maintenir, sur un projet de quatre semaines. Ensuite **l'absence d'étape de transpilation dédiée** : le lanceur natif consomme directement les sources TypeScript. Enfin **la stabilité de l'interface** : le lanceur suit le cycle de vie de Node lui-même, sans risque de rupture liée à une montée de version d'un outil tiers. Le prix à payer est réel — pas d'écosystème de *matchers* riches, pas de rapport de couverture intégré — mais il était acceptable au regard du périmètre.

10.3. État d'exécution réel

Cinq des sept suites échouent. Seules `auth.controller` et `socket` passent, et uniquement sur les parcours qui ne dépendent pas de la donnée en cause.

L'échec est identique partout et se produit au démarrage de la suite, pas dans le code applicatif : `roleId: NaN`, suivi de `Argument 'role' is missing`. La cause est un **ordre d'initialisation dans la fixture globale** : le `beforeAll` ne garantit pas la création du rôle parent avant l'insertion des utilisateurs de test, qui partent alors avec une clé étrangère invalide et font tomber toute la suite.

Le diagnostic est donc établi, et il est important d'en tirer la bonne conclusion : **le défaut est dans le code de test, pas dans le code applicatif**. Le correctif consiste à réordonner l'initialisation de la fixture. Il n'a pas été appliqué avant la fin du projet, la priorité ayant été donnée à la finition fonctionnelle. C'est une dette assumée, et le premier chantier de reprise.

La couverture n'a jamais été mesurée. Aucun outil de couverture n'est branché, ni en local ni en intégration continue. Le dossier ne peut donc avancer aucun pourcentage — et n'en avancera pas.

Il faut enfin noter qu'un script `test:unit` existe dans le `package.json` du backend, mais qu'**aucun fichier `*.unit.test.ts` n'a jamais été écrit** : le script ne sélectionne rien.

10.4. Absence de tests automatisés côté frontend

Le livrable ne contient **aucun** test frontend : ni test unitaire, ni test de composant, ni test de bout en bout, ni catalogue de composants. Le constat est vérifiable en une commande sur le dépôt de production :

```
grep -iE "vite|playwright|jest|testing-library|storybook" frontend/package.json
→ aucun résultat
```

10.5. La validation manuelle, méthode réellement employée

Pendant le projet, les fonctionnalités ont été validées **à la main**, en parcourant l'application dans l'environnement de développement conteneurisé. Cette validation s'appuyait notamment sur une collection de requêtes HTTP annotées (`request.http`) permettant de rejouer les appels d'API hors interface.

Cette démarche n'est pas sans valeur : elle a permis de livrer une application fonctionnelle en production. Mais elle a deux limites structurelles — elle n'est pas **reproductible** à l'identique d'une fois sur l'autre, et elle ne laisse **aucune trace** exploitable après coup.

Le jeu d'essai présenté en section 11 est la formalisation de cette démarche : mêmes gestes, mais scénario écrit à l'avance, résultats attendus définis, données obtenues consignées et vérifiées en base.

10.6. Ce qui manque et ce que je mettrais en place

Corriger la fixture d'abord. Cinq suites déjà écrites ne demandent qu'un réordonnement pour redevenir utiles. C'est le meilleur rapport entre effort et bénéfice de toute cette liste : soixante tests existent, il s'agit de les remettre en état de tourner, pas d'en écrire de nouveaux.

Tests unitaires sur la validation et les utilitaires. Les schémas Zod et les fonctions pures — formatage de dates, calcul de moyenne des évaluations, troncature de description — sont testables sans base ni serveur. Ce sont les tests les moins coûteux à écrire et les plus rapides à exécuter.

Tests de composants. Les composants d'interface porteurs de logique — la liste de conversations, le fil de messages, les formulaires validés — méritent des tests de rendu et d'interaction, indépendamment du choix d'outil.

Tests de bout en bout sur les parcours critiques. Trois parcours justifient l'investissement : inscription et connexion, recherche puis mise en relation, et l'échange de messages en temps réel entre deux sessions simultanées.

Mesure de couverture. Non comme un objectif chiffré à atteindre — un pourcentage élevé ne garantit rien — mais comme un révélateur des zones jamais exercées.

Exécution en intégration continue. Le pipeline actuel ne fait que déployer (cf. section 6). Y ajouter une étape qui lance les tests et bloque le déploiement en cas d'échec est ce qui transforme une suite de tests en filet de sécurité réel. Une suite qu'on n'exécute pas automatiquement finit par ne plus être exécutée du tout — les cinq suites en échec en sont l'illustration.

10.6.1. Ce que cette dette a concrètement coûté

L'argument le plus solide en faveur de tests frontend ne vient pas d'un principe général, mais de ce dossier lui-même. Le jeu d'essai de la section 11 a mis au jour un **défaut réel et reproductible** : à la réception de l'événement `conversation:new`, la nouvelle conversation s'affiche **deux fois** dans la liste du destinataire, alors que la base n'en contient qu'une. La cause est une insertion en tête de liste sans garde sur l'identifiant (`useConversationList.ts:43-45`).

Ce défaut est passé inaperçu pendant les quatre semaines du projet. Il aurait été détecté par un test d'intégration frontend de quelques lignes — émettre deux fois l'événement et vérifier que la liste ne contient qu'une entrée. C'est la mesure concrète du coût de cette dette : non pas un risque théorique, mais un bug présent dans le livrable, trouvé le jour où un scénario a enfin été rejoué de bout en bout de façon méthodique.

11. Jeu d'essai

11.1. Contexte d'exécution

Le scénario retenu est celui de la fonctionnalité représentative : **Alice envoie un premier message à Bob qu'elle suit**. Il active la branche `isFirstMessage` du handler `message:send`, et exerce donc l'ensemble de la chaîne — gating métier, création de conversation, persistance, diffusion multi-rooms et réconciliation de l'UI optimiste.

L'exécution a eu lieu le **23 août 2026** en environnement de **développement local conteneurisé** (Docker Compose : PostgreSQL 16, Meilisearch, Nginx, backend et frontend), sur une **copie du dépôt d'équipe placée hors du dépôt** — le livrable d'origine n'a reçu aucune écriture. Le jeu de données est celui du seed de développement : 41 utilisateurs, dont les deux acteurs.

Paramètre	Valeur
Acteurs	Alice Dupont (<code>id = 1</code>) et Bob Martin (<code>id = 27</code>)
Authentification	<code>alice.dupont@example.com</code> / <code>bob.martin@example.com</code> , mot de passe du seed
Point d'entrée	<code>http://localhost:8888</code> (Nginx → frontend + API)
Conversation créée	« Jeu d'essai CDA » (<code>id = 17</code>)
Pré-requis	Le lien de suivi Alice → Bob existe déjà dans le seed

Les captures d'écran ont été produites en pilotant **deux contextes navigateur simultanés**, afin d'observer les deux côtés de l'échange dans la même seconde¹⁸.

11.2. Étape 1 — Le lien de suivi, pré-requis du gating

Données d'entrée	Données attendues	Données obtenues	Analyse
Alice authentifiée Consultation du profil de Bob (<code>/profil/27</code>)	Lien de suivi actif Ligne présente en table <code>follow</code> Action « Contacter » accessible	Profil affiché, état « suivi » actif <code>follow</code> <code>id = 1</code> , Alice → Bob Créé au seed le 22/08 à 18:37:48	Conforme. Le pré-requis métier est satisfait : sans ce lien, la création de conversation serait refusée par <code>require SimpleFollow</code> .

¹⁸Outil de pilotage hors périmètre du livrable d'équipe : il n'a servi qu'à produire ces illustrations pour le présent dossier, et ne constitue en aucun cas une couverture de test automatisé — cf. section 10.

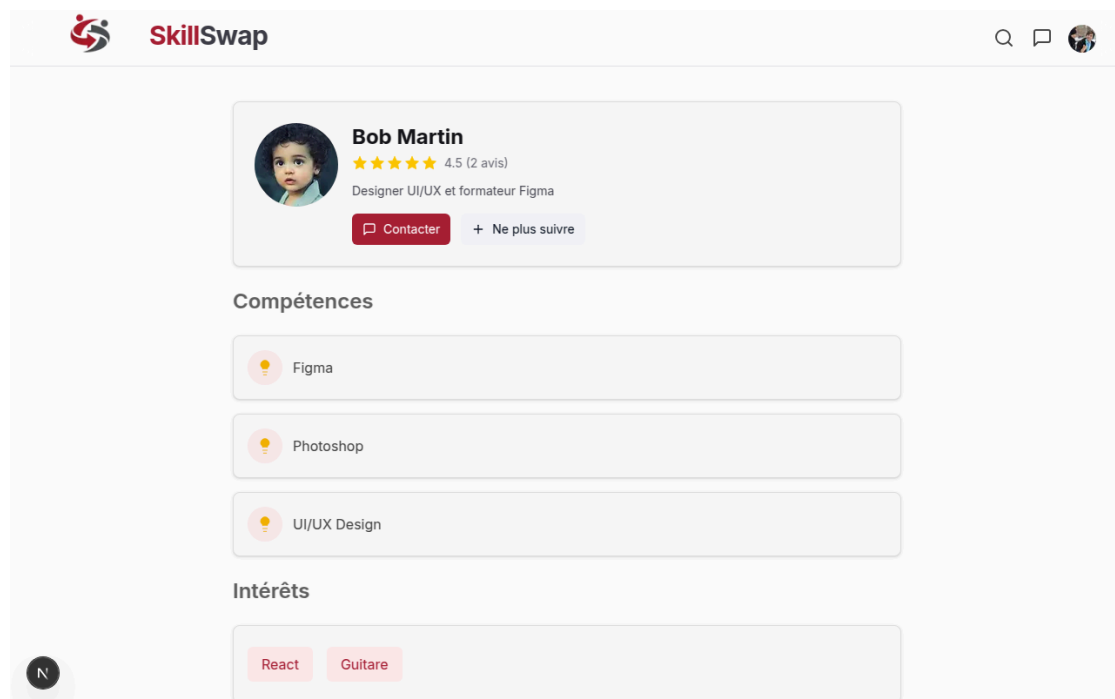


Fig. 23. – Étape 1 — profil public de Bob consulté par Alice, lien de suivi actif.

11.3. Étape 2 — Création de la conversation

Données d'entrée	Données attendues	Données obtenues	Analyse
POST /api/v1/conversations Cookie accessToken (Alice) <pre>{ title: "Jeu d'essai CDA", receiverId: 27 }</pre>	201 Created Conversation au statut Open Exactement 2 participants Aucun message	Conversation id = 17 créée à 16:27:11.755 Statut Open, participants Alice ↔ Bob nb_messages = 0	Conforme. La cardinalité (0,2) est respectée dès la création : les deux lignes user_has_conversation sont insérées en une fois. Le fil est vide, ce qui armera la branche premier message.

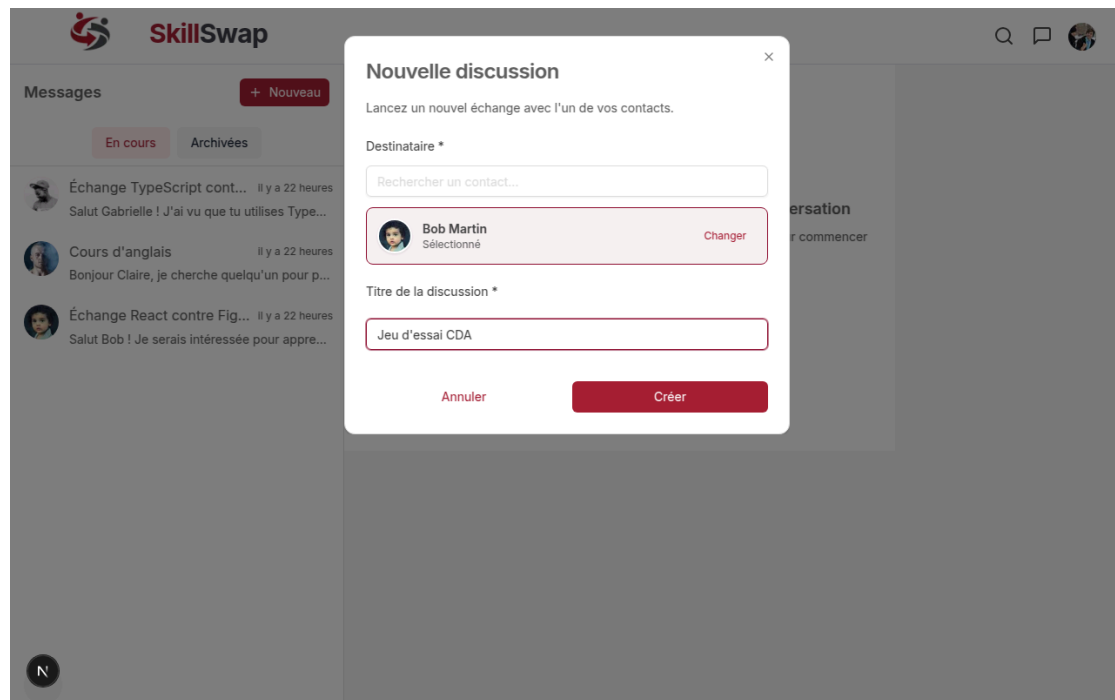


Fig. 24. – Étape 2 — dialogue de création, Bob sélectionné parmi les seuls membres suivis.

11.4. Étape 3 — Envoi du message et affichage optimiste

Données d'entrée	Données attendues	Données obtenues	Analyse
Event <pre>message:send</pre> <pre>{ conversationId: 17, message: "Salut Bob !..." }</pre> Socket authentifié par cookie	Affichage immédiat côté Alice, avant réponse serveur Persistance d'un message <code>updated_at</code> de la conversation modifié	Message affiché instantanément dans le fil d'Alice Message <code>id = 108</code> persisté à 16:27:20.973 <code>updated_at</code> passé à 16:27:20.972	Conforme. L'écart de 9 secondes entre la création de la conversation et le message confirme que le fil était bien vide à l'ouverture. La mise à jour de l'horodatage prouve l'exécution du <code>Promise.all</code> décrit en section 8.

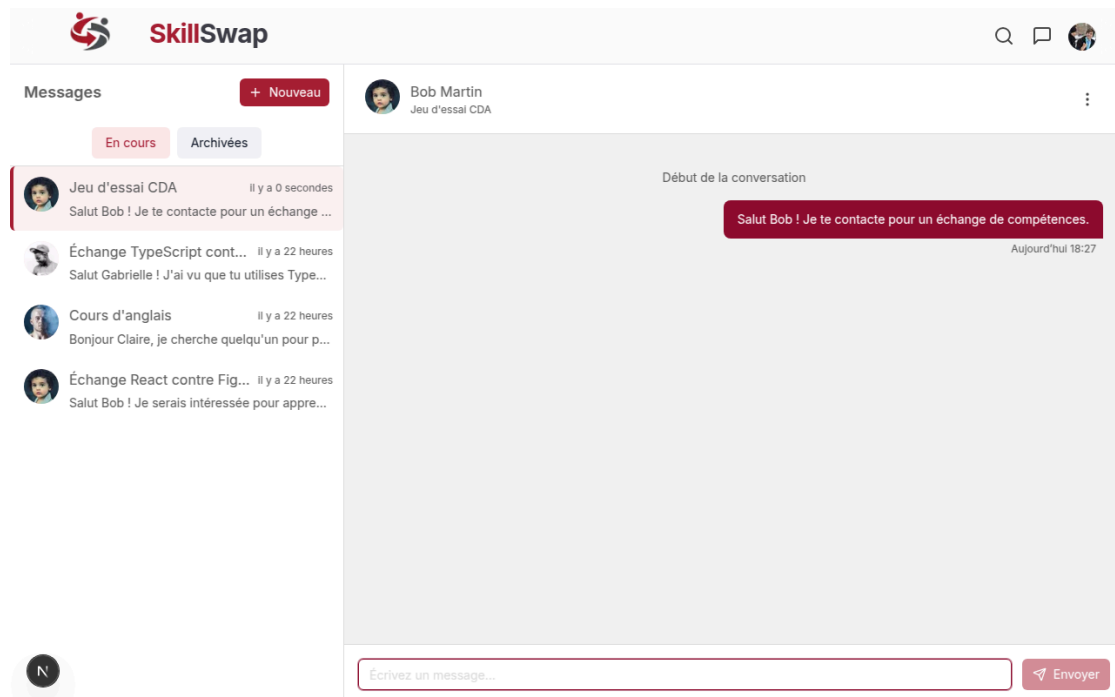


Fig. 25. – Étape 3 — le message apparaît immédiatement côté Alice, avant confirmation du serveur.

11.5. Étape 4 — Réception temps réel par Bob

Données d'entrée	Données attendues	Données obtenues	Analyse
Bob authentifié, page conversations ouverte Aucune conversation sélectionnée Socket inscrit à la room <code>user:27</code>	Réception de <code>conversation:new</code> Notification visuelle Conversation ajoutée en tête de liste	Toast « Alice a démarré un nouvel échange » affiché Conversation visible en tête de liste Mais dupliquée : deux entrées identiques	Partiellement conforme. La diffusion temps réel fonctionne — Bob est notifié sans être dans la conversation, ce qui valide le modèle de rooms à deux niveaux. L'affichage présente en revanche un écart, analysé en sous-section 11.8.

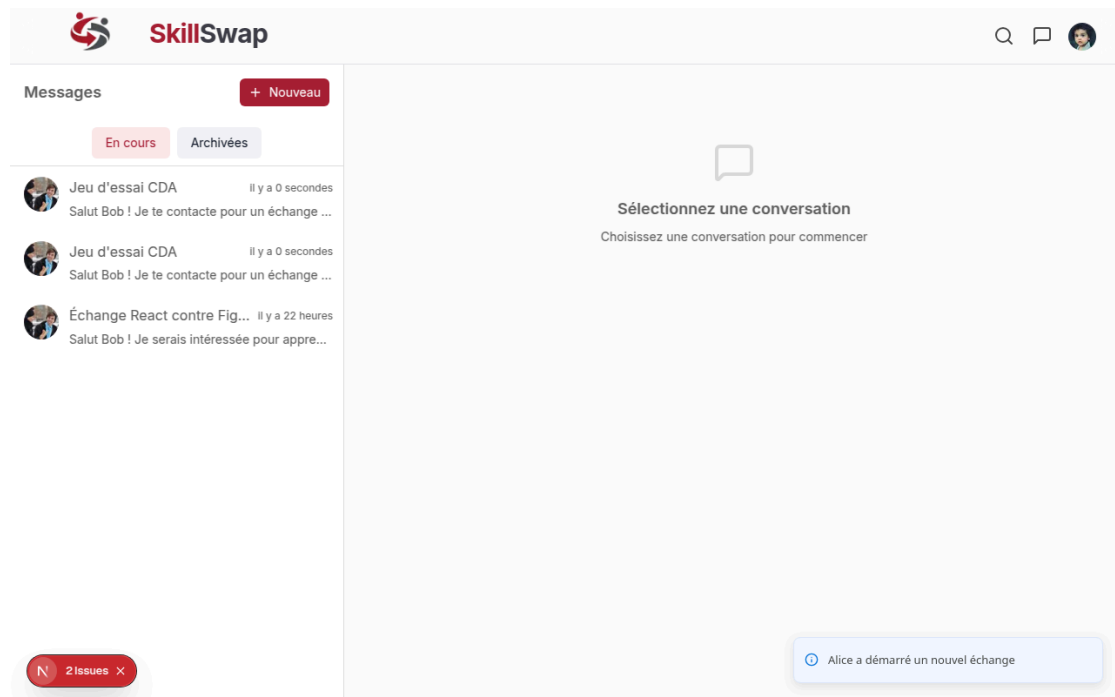


Fig. 26. – Étape 4 — Bob reçoit la notification sans avoir ouvert la conversation. Le doublon d’affichage visible dans la liste est analysé plus bas.

11.6. Étape 5 — Réconciliation côté Alice

Données d’entrée	Données attendues	Données obtenues	Analyse
Alice reçoit <code>message:new</code> en retour de sa propre émission	Le message optimiste reste affiché Aucun doublon dans le fil Un seul message en base	Fil d’Alice inchangé, un seul message affiché <code>COUNT(message) = 1</code> pour la conversation 17	Conforme. Le filtre <code>sender.id === user.id</code> ignore le retour serveur pour l’émetteur : l’affichage optimiste reste l’état final, sans duplication.

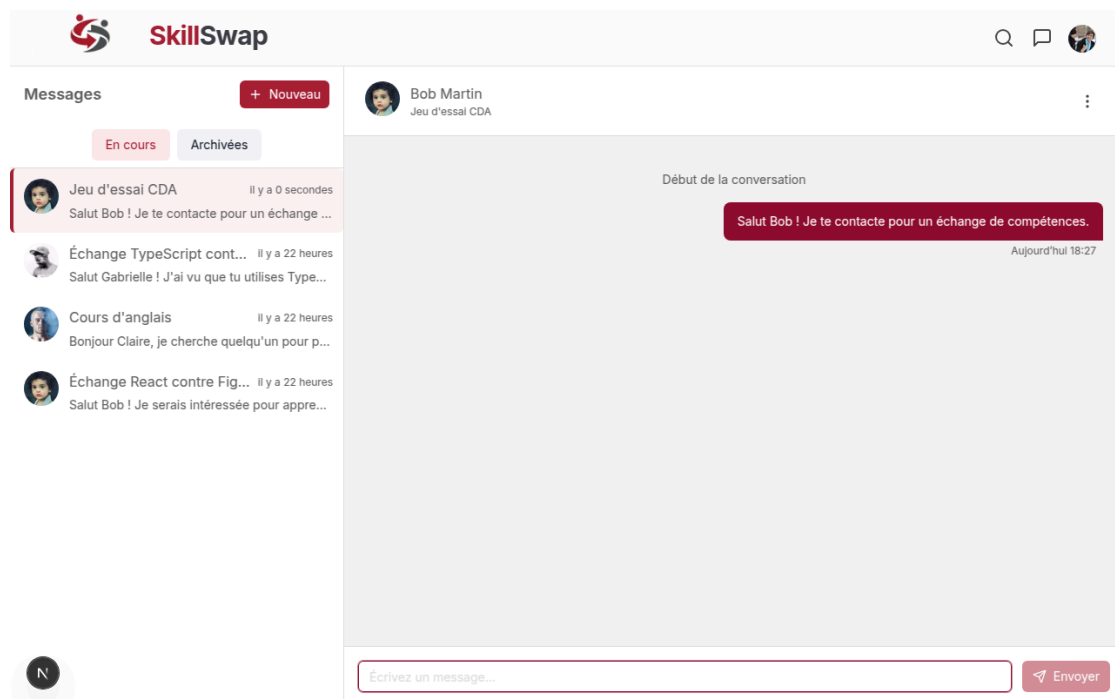


Fig. 27. – Étape 5 — après retour serveur, le fil d'Alice reste à un seul message.

11.7. Preuves en base de données

Les requêtes ci-dessous ont été exécutées directement sur PostgreSQL, avant et après le scénario.

R1 — le lien de suivi existe.

```
SELECT f.id, u1.firstname AS suiveur, u2.firstname AS suivi, f.created_at
FROM follow f
JOIN "user" u1 ON u1.id = f.follower_id
JOIN "user" u2 ON u2.id = f.followed_id
WHERE u1.email = 'alice.dupont@example.com'
AND u2.email = 'bob.martin@example.com';
```

id	suiveur	suivi	created_at
1	Alice	Bob	2026-08-22 18:37:48.027

R1bis — la contrainte d'unicité interdit le doublon. Tentative volontaire d'insertion d'un lien déjà existant :

```
INSERT INTO follow (follower_id, followed_id) VALUES (1, 27);

ERROR:  duplicate key value violates unique constraint
        "follow_followed_id_follower_id_key"
DETAIL:  Key (followed_id, follower_id)=(27, 1) already exists.
```

La règle « un seul suivi par couple » n'est pas seulement applicative : elle est garantie au niveau de la base. Un `INSERT` direct, qui contournerait tout le code métier, échoue quand même.

R2 — la conversation compte exactement deux participants.

```
SELECT c.id, c.title, c.status,
       COUNT(uhc.user_id) AS nb_participants,
       STRING_AGG(u.firstname, ' ↔ ' ORDER BY u.firstname) AS participants
FROM conversation c
JOIN user_has_conversation uhc ON uhc.conversation_id = c.id
JOIN "user" u ON u.id = uhc.user_id
WHERE c.title = 'Jeu d'essai CDA'
GROUP BY c.id, c.title, c.status
HAVING COUNT(uhc.user_id) = 2;
```

id	title	status	nb_participants	participants
17	Jeu d'essai CDA	Open	2	Alice ↔ Bob

R3 — comptage des messages et propagation de l'horodatage. Le `LEFT JOIN` est délibéré : il rend visible le zéro de l'étape 2, qu'une jointure interne masquerait en ne renvoyant aucune ligne.

```
SELECT c.id, c.title, COUNT(m.id) AS nb_messages,
       MAX(m.created_at) AS dernier_message, c.updated_at,
       (c.updated_at > c.created_at) AS conversation_touchee
FROM conversation c
LEFT JOIN message m ON m.conversation_id = c.id
WHERE c.title = 'Jeu d'essai CDA'
GROUP BY c.id, c.title, c.updated_at, c.created_at;
```

id	nb_messages	dernier_message	updated_at	conversation_touchee
17	1	2026-08-23 16:27:20.973	2026-08-23 16:27:20.972	t

Le booléen `conversation_touchee` à `t` prouve que l'écriture du message et la mise à jour de la conversation ont bien eu lieu ensemble.

R4 — cohérence de la dénormalisation `receiver_id`. La colonne duplique une information déductible des participants ; la sous-requête vérifie qu'elle reste cohérente.

```
SELECT m.id, exp.firstname AS expéditeur, dest.firstname AS destinataire,
       LEFT(m.content, 40) AS contenu, m.created_at,
       (dest.id IN (SELECT user_id FROM user_has_conversation
                    WHERE conversation_id = m.conversation_id))
       AS destinataire_participe
FROM message m
JOIN "user" exp ON exp.id = m.sender_id
JOIN "user" dest ON dest.id = m.receiver_id
JOIN conversation c ON c.id = m.conversation_id
WHERE c.title = 'Jeu d'essai CDA';
```

id	expéditeur	destinataire	contenu	destinataire_participe
108	Alice	Bob	Salut Bob ! Je te contacte pour un échan	t

R5 — évaluations Alice → Bob. La requête renvoie une ligne, mais **elle provient du seed de développement, pas du scénario** : elle est horodatée à la seconde du seed (22/08, 18:37:48). La conversation « Jeu d'essai CDA » reste au statut `Open`, et le jeu d'essai n'a donc créé aucune évaluation — celle-ci n'étant proposée qu'à la clôture.

R6 — vue d'ensemble après scénario.

```
SELECT u.firstname,
       COUNT(DISTINCT uhc.conversation_id) AS conversations,
       COUNT(DISTINCT m.id)                AS messages_envoyes,
       COUNT(DISTINCT f.followed_id)       AS abonnements
FROM "user" u
LEFT JOIN user_has_conversation uhc ON uhc.user_id = u.id
LEFT JOIN message m ON m.sender_id = u.id
LEFT JOIN follow f ON f.follower_id = u.id
WHERE u.email IN ('alice.dupont@example.com', 'bob.martin@example.com')
GROUP BY u.id, u.firstname ORDER BY u.id;
```

firstname	conversations	messages_envoyes	abonnements
Alice	4	11	2
Bob	2	4	2

11.8. Analyse des écarts

Écart constaté — doublon d’affichage côté destinataire. À l’étape 4, la nouvelle conversation apparaît **deux fois** dans la liste de Bob, alors que la base n’en contient qu’une.

Le contrôle en base est sans ambiguïté : une seule ligne dans `conversation` (`id = 17`) et deux lignes dans `user_has_conversation` — une par participant, comme attendu. **L’anomalie est donc purement côté client** : la persistance est correcte, seul l’affichage duplique l’entrée.

Cause identifiée. Le mutateur `addConversation` (`useConversationList.ts:43-45`) insère la conversation reçue en tête de liste sans vérifier qu’elle s’y trouve déjà :

```
const addConversation = useCallback((conv: Conversation) => {
  setConversations((prev) => [conv, ...prev]);
}, []);
```

Aucune garde sur l’identifiant. Si l’événement `conversation:new` est reçu deux fois, ou s’il arrive alors que la conversation figure déjà dans la liste, l’entrée est dupliquée.

Correctif proposé. Déduplication par identifiant avant insertion :

```
const addConversation = useCallback((conv: Conversation) => {
  setConversations((prev) => {
    prev.some((c) => c.id === conv.id) ? prev : [conv, ...prev],
  });
}, []);
```

Réserve d’interprétation. L’observation a été faite en **mode développement**, où React StrictMode monte les effets deux fois et peut donc enregistrer le listener Socket.IO en double — ce qui suffirait à expliquer la double réception. **Je n’ai pas vérifié si l’anomalie**

se reproduit en production. Ce qui est établi indépendamment du mode d'exécution, en revanche, c'est l'absence de garde dans `addConversation` : le correctif reste pertinent dans les deux cas, et son coût est de deux lignes.

Bilan. Quatre étapes sur cinq sont conformes sans réserve. La cinquième valide le mécanisme temps réel visé — Bob est bien notifié sans être dans la conversation — mais révèle un défaut d'affichage qui n'avait pas été détecté pendant le projet, faute de tests frontend. C'est une illustration concrète de la dette relevée en section 10 : un test d'intégration sur la liste de conversations l'aurait mis en évidence.

12. Veille de sécurité

12.1. Périmètre et constat

Aucune veille de sécurité formalisée n'a été conduite pendant le projet : ni journal dédié, ni source suivie, ni périodicité définie, ni outil automatisé de suivi des vulnérabilités — le dépôt ne contient aucun fichier `dependabot.yml`, aucune configuration Renovate, aucun rapport d'audit de dépendances. La sécurité a été traitée par des choix d'implémentation et par une réaction ponctuelle à un incident, décrits ci-dessous. C'est une lacune identifiée, dont la remédiation est proposée en fin de section.

12.2. Une vulnérabilité identifiée et traitée pendant le projet

Le fichier d'environnement `devops/.env.docker` est ajouté au suivi Git le 12 janvier, puis complété d'un `JWT_SECRET` en clair le 13. Je l'ai retiré du suivi le 14 janvier (commit `6bd3e25`, « fix(security): remove .env.docker from git tracking »), et le `.gitignore` couvre désormais ce chemin, ce qui prévient la récurrence.

Analyse de la mesure. Retirer un fichier du suivi empêche les commits ultérieurs de le réintroduire, mais ne supprime pas la valeur de l'historique déjà écrit. Une remédiation complète aurait supposé une rotation du secret côté serveur, et le cas échéant une réécriture d'historique. Cette limite est assumée.

12.3. Mesures de sécurité retenues pendant le projet

- **argon2** pour le hachage des mots de passe (`auth.service.ts:21`) — fonction à coût mémoire, résistante aux attaques par matériel dédié.
- **JWT en cookies** `httpOnly` + `secure` + `sameSite=strict` — le jeton n'est pas accessible au JavaScript client, ce qui neutralise son vol par XSS. L'ADR-007 rejette explicitement `localStorage` pour ce motif.
- **Rotation du refresh token** à chaque renouvellement, avec invalidation du précédent (`auth.service.ts:103-108`).
- **Origine CORS unique**, fournie par variable d'environnement, appliquée à Express et au handshake Socket.IO ; en production, le serveur refuse de démarrer si elle est absente.
- **Validation systématique des entrées** par schémas Zod, et requêtes paramétrées via l'ORM contre l'injection SQL.

Ces choix n'ont pas fait l'objet d'une justification écrite pendant le projet — la documentation de sécurité d'époque mentionnait même bcrypt alors que le code utilise argon2. L'écart a été relevé et corrigé lors de la reprise documentaire pour ce dossier.

12.4. Analyse OWASP conduite sur l'application

La couverture du référentiel OWASP Top 10 est présentée en section 9. Quatre dettes y sont identifiées et assumées : Helmet installé mais non monté, absence de Content-Security-Policy, absence de limitation de débit, accessibilité non mesurée.

12.5. Ce que je mettrais en place

Outillage automatisé. Activer Dependabot ou Renovate sur le dépôt, avec regroupement des mises à jour mineures et de correctifs en une seule Pull Request périodique — pour éviter le bruit qui fait qu'on finit par ne plus les lire — et alerte immédiate, isolée, sur les avis critiques. La valeur de l'outil tient à ce réglage : mal configuré, il produit un flux que l'équipe apprend à ignorer.

Contrôle en intégration continue. Exécuter `npm audit` à chaque passage de pipeline, bloquant au-delà d'un seuil de sévérité défini — par exemple *high* sur les dépendances directes — avec une dérogation explicite et tracée pour les cas jugés non exploitables. La dérogation doit être écrite quelque part : un seuil qu'on contourne silencieusement ne protège plus.

Distinguer la vulnérabilité déclarée du risque réel. Une dépendance vulnérable n'est dangereuse que si le code appelle effectivement le chemin concerné. La démarche consiste à vérifier trois choses : si le paquet est une dépendance directe ou transitive ; s'il est chargé en production ou seulement en développement et en test ; et si la condition d'exploitation décrite par l'avis est réunie dans le contexte de l'application. Un paquet importé uniquement par un fichier de test n'expose pas le service ; à l'inverse, une bibliothèque traversée par chaque requête entrante mérite un traitement immédiat.

Sources suivies. Les bulletins de sécurité des projets critiques du socle — Next.js, Express, Prisma, Socket.IO —, la base GitHub Advisory, les avis de l'ANSSI et du CERT-FR, et les recherches en anglais sur les termes de vulnérabilité (`CVE`, `security advisory`, `GHSA`) associés aux paquets utilisés.

Périodicité. Revue hebdomadaire des alertes automatiques, revue mensuelle des versions majeures du socle. La revue mensuelle sert à décider des montées de version qui ne sont pas déclenchées par un avis mais par le risque de retard accumulé.

Traitement des dettes déjà identifiées. Monter Helmet côté Express, définir une Content-Security-Policy en mode observation avant enforcement, et ajouter une limitation de débit sur les routes d'authentification.

13. Difficultés rencontrées et solutions

Les difficultés relatées ici sont documentées par deux sources : le journal de bord tenu par l'équipe pendant le projet¹⁹ et l'historique du dépôt d'équipe.

13.1. La mise au point de l'environnement conteneurisé

La stabilisation de l'environnement Docker a occupé les premiers jours du sprint 1 et a demandé plusieurs itérations. Le journal de bord d'équipe enregistre le 14 janvier un « debug docker seeding + frontend qui restart en boucle » ; le même jour et le suivant, j'ai traité la configuration de production, les healthchecks, le support du rechargement à chaud (HMR) dans le conteneur, le passage des images de développement à Node 24, la correction du script d'attente de PostgreSQL — chemin absolu, ordonnancement du démarrage base/API — puis les volumes nommés et le montage sélectif de la configuration Nginx. Quatorze commits sur trois jours ont été nécessaires pour obtenir un environnement stable.

Une régression est survenue une semaine plus tard : le 19 janvier, une erreur 502 bad gateway a été constatée en environnement conteneurisé ; elle provenait d'un port de proxy Nginx désaccordé avec le backend, que j'ai corrigé. L'infrastructure s'est révélée sensible à chaque ajout de service — Meilisearch le 22 janvier, la gestion des avatars le 29.

13.2. Le contrat d'interface entre le frontend et le backend

Le projet ne dispose pas de package de types partagé : les types sont déclarés des deux côtés. Cette duplication, assumée au cadrage, a fait du contrat d'interface la zone la plus instable du code — `api-client.ts` et `api-types.ts` totalisent soixante-trois retouches sur les quatre semaines. Chaque évolution du schéma Prisma se répercutait en plusieurs points du frontend, comme en témoignent les messages de commits (« align api-types with Prisma schema »).

Une difficulté connexe a été le rattrapage d'une convention de nommage non tranchée au cadrage : la coexistence du camelCase côté TypeScript et du snake_case côté base a nécessité, du 17 au 19 janvier, quatre commits correctifs et une migration dédiée (`fix_snake_case`), renommant notamment `avatarUrl` en `avatar_url`. Corriger une convention en cours de projet suppose d'intervenir simultanément en base, dans le backend et dans le frontend.

13.3. Une revue de code à mi-parcours

Le 20 janvier, j'ai conduit une revue systématique du frontend, organisée en quatre phases et documentée au fil de son avancement. Elle a identifié trente points de correction, dont des bugs de logique : mutation de tableau, problème de synchronisation asynchrone, mapping de données incomplet. Elle a également porté sur la cohérence des conventions — renommage des fichiers de composants en PascalCase — et sur la factorisation, avec la mise en place d'un *factory pattern* pour les icônes. Cette journée

¹⁹Tableur partagé, hors dépôt Git — cf. section 5.

entière consacrée à la qualité plutôt qu'à de nouvelles fonctionnalités a été un arbitrage assumé.

13.4. Le cycle de vie de la session

La gestion de session a été ajustée par retouches successives sur l'ensemble du projet : correction de la déconnexion et de l'inscription le 15 janvier, de l'expiration côté frontend le 20, enrichissement des réponses d'authentification et traitement des erreurs 401. Le fournisseur de contexte d'authentification compte vingt-quatre retouches.

La difficulté la plus tenace est apparue le dernier jour : la pose des cookies lors des redirections a nécessité quatre passes successives. La navigation côté client de Next.js ne déclenchant pas la prise en compte des cookies posés par le serveur, il a fallu forcer un rechargement complet via `window.location.href` pour la connexion, la déconnexion et la suppression de compte.

13.5. Les tests d'intégration backend

Cinq des sept suites de tests d'intégration échouaient à l'issue du projet. La cause a été identifiée et documentée dans la documentation qualité : les données de rôle ne sont pas créées avant les utilisateurs de test dans la fixture. Le diagnostic est établi — il s'agit d'un ordonnancement d'initialisation, non d'un défaut du code applicatif — mais le correctif n'a pas été appliqué avant la fin du projet.

13.6. Ce que ces difficultés ont en commun

Trois d'entre elles — le contrat d'interface, la convention de nommage, la gestion de session — ne relèvent pas d'un obstacle technique isolé mais d'une décision de cadrage insuffisamment tranchée au départ, dont le coût s'est étalé sur tout le projet. À l'inverse, la journée de revue de code du 20 janvier a montré qu'un temps consacré à la qualité en cours de route se rentabilise. J'en retiens qu'il vaut mieux fixer tôt les conventions transverses — nommage, types partagés, gestion de session — quitte à y consacrer une demi-journée de cadrage supplémentaire.

14. Conclusion

14.1. Bilan du projet

L'apothéose SkillSwap a livré, en quatre semaines, une plateforme web complète et déployée en production, intégrant les sept fonctionnalités du périmètre MVP cible : page d'accueil, authentification, profils avec compétences et disponibilités, moteur de recherche, suivi de profils, messagerie temps réel, système d'évaluation. Les deux fonctionnalités initialement classées comme « évolutions possibles » — messagerie temps réel via WebSocket et recherche avancée via Meilisearch — ont été promues au périmètre MVP en cours de projet et livrées en production. La couverture fonctionnelle atteinte dépasse donc le périmètre minimum acté en sprint 0.

L'équipe a appliqué la méthode Scrum sur quatre sprints d'une semaine, maintenu une qualité de code outillée (ESLint, Prettier, Husky), écrit sept suites de tests d'intégration backend, et conduit une mise en production sur VPS avec Docker, Nginx et certificat TLS renouvelé automatiquement.

14.2. Évolutions identifiées pour la V2

Cinq catégories de dettes techniques tracées au fil du projet :

Sécurité — monter Helmet (installé mais non monté), ajouter une limitation de débit sur les routes d'authentification, définir une Content Security Policy (en observation puis en application).

Tests — corriger le défaut de fixture qui met cinq des sept suites d'intégration en échec, puis mettre en place la couverture frontend décidée à l'ADR-010 (unitaire, E2E) et non implémentée.

DevOps — compléter le déploiement continu par une intégration continue (build, lint, tests), mettre en place supervision et sauvegarde automatique de la base.

Parcours — l'inscription perd la destination d'origine du visiteur. Le lien d'un profil public produit bien `/inscription?redirect=<chemin>` (`ProfileTeaser.tsx:39`), mais la page d'inscription ne lit jamais ce paramètre : elle redirige en dur vers `/mon-profil` (`inscription/page.tsx:41`). La page de connexion, elle, le consomme correctement (`connexion/page.tsx:18` puis `:28`). Défaut relevé lors de la reconstruction du parcours utilisateur (sous-section 6.3.3). **Correctif** : aligner l'inscription sur la connexion, en lisant `searchParams.get('redirect') || '/mon-profil'` — le comportement actuel devenant le simple cas par défaut.

Fonctionnel — blocage de profils, suggestion automatique de correspondances, groupes thématiques.

14.3. Apprentissage personnel

Ce projet a été l'occasion d'appliquer rigoureusement l'Atomic Design à un frontend de production — sept pages, dix-huit atomes, neuf molécules et trente-neuf fichiers d'organismes regroupés en cinq ensembles fonctionnels —, de concevoir et déployer une chaîne de production complète (images multi-étapes, reverse proxy, TLS), et

d'amorcer une démarche de documentation continue en rédigeant une documentation Arc42 partagée à l'équipe — démarche que je poursuis dans mes projets suivants.

La reprise de ce dossier m'a par ailleurs conduit à vérifier systématiquement chaque affirmation documentaire contre le code livré. L'exercice a révélé des écarts, corrigés depuis. J'en retiens qu'une documentation n'a de valeur que si elle est vérifiable, et qu'il vaut mieux assumer une dette identifiée que présenter une couverture qu'on n'a pas.

15. Lexique

Les termes ci-dessous sont ceux effectivement employés dans ce dossier. Les définitions visent la compréhension par un lecteur non spécialiste du projet.

Terme	Définition
ADR (Architecture Decision Record)	Fiche courte qui consigne une décision d'architecture : le contexte, les options envisagées, le choix retenu et ses conséquences. Sert à retrouver plus tard pourquoi une option a été préférée à une autre.
Architecture en couches	Organisation du code en niveaux de responsabilité successifs — ici router, middleware, controller, service, accès aux données — où chaque couche ne s'adresse qu'à la suivante.
argon2id	Algorithme de hachage de mots de passe, lauréat du Password Hashing Competition 2015 et variante recommandée par l'OWASP. Son coût en mémoire le rend résistant aux attaques par matériel dédié.
Atomic Design	Méthode d'organisation des composants d'interface proposée par Brad Frost, des plus élémentaires aux plus composés : atomes, molécules, organismes, pages.
Cardinalité	Nombre minimal et maximal d'occurrences d'une entité pouvant participer à une association. Notée par un couple, par exemple (0,2) : de zéro à deux.
CI/CD	Intégration continue et déploiement continu. L'intégration vérifie automatiquement chaque modification (compilation, tests) ; le déploiement publie automatiquement la version validée.
Clé composite	Clé primaire formée de plusieurs colonnes plutôt que d'une seule. Employée ici pour identifier un participant à une conversation par le couple utilisateur-conversation.
Contrainte d'intégrité / applicative	Une contrainte d' intégrité est garantie par la base elle-même et ne peut être contournée. Une contrainte applicative n'est assurée que par le code : une écriture directe en base pourrait la violer. La distinction est essentielle pour savoir sur quoi l'on peut réellement compter.
Cookie httpOnly	Cookie que le JavaScript de la page ne peut pas lire. Il reste transmis automatiquement au serveur, ce qui protège le jeton de session contre le vol par injection de script.
CORS	Mécanisme par lequel un serveur déclare quelles origines web sont autorisées à l'appeler. Il empêche un site tiers d'interroger l'API au nom d'un utilisateur connecté.

CSP (Content Security Policy)	En-tête HTTP énumérant les sources de contenu qu'une page a le droit de charger. Il limite fortement la portée d'une injection de script.
Dénormalisation	Duplication volontaire d'une information déjà déductible ailleurs, pour simplifier ou accélérer les lectures — au prix d'un risque d'incohérence à surveiller.
Docker	Outil d'exécution d'applications en conteneurs : chaque service embarque son environnement complet, ce qui rend son comportement identique en développement et en production.
DTO (Data Transfer Object)	Objet dédié au transport de données entre deux couches, exposant uniquement les champs nécessaires plutôt que l'entité complète.
ERD (Entity-Relationship Diagram)	Diagramme représentant les tables, leurs colonnes et les liens qui les unissent. Vue la plus proche du schéma réellement implémenté.
Event (Socket.IO)	Message nommé échangé sur une connexion temps réel, par exemple <code>message:send</code> . L'émetteur le publie, les destinataires abonnés le reçoivent.
Façade de hooks	Hook React unique servant de point d'entrée à plusieurs hooks spécialisés. Le composant appelle la façade et ignore le détail de la composition.
Fixture	Jeu de données préparé avant l'exécution de tests, garantissant qu'ils partent tous du même état connu.
Handshake	Phase initiale d'une connexion, pendant laquelle client et serveur négocient et vérifient les conditions de l'échange — ici, la validité du jeton d'authentification avant d'accepter la connexion temps réel.
HSTS	En-tête HTTP par lequel un site impose aux navigateurs de ne plus l'atteindre qu'en HTTPS, y compris lors des visites suivantes.
Injection SQL	Attaque consistant à glisser du code SQL dans une donnée saisie pour détourner une requête. Les requêtes paramétrées, générées ici par l'ORM, la neutralisent.
JWT (JSON Web Token)	Jeton d'authentification signé, contenant les informations d'identité de l'utilisateur. Sa signature permet d'en vérifier la validité sans consulter la base.
Mapper	Fonction convertissant une donnée d'une représentation vers une autre, par exemple d'une entité de base vers le format attendu par le moteur de recherche.
MCD (Modèle Conceptuel de Données)	Premier niveau Merise : les concepts métier et leurs associations, indépendamment de toute technologie de base de données.

Meilisearch	Moteur de recherche plein texte open source, tolérant aux fautes de frappe. Utilisé ici pour l'indexation et la recherche de membres.
Merise	Méthode française de modélisation des données procédant en trois niveaux successifs — conceptuel, logique, physique.
Middleware	Fonction intercalée dans le traitement d'une requête, exécutée avant le traitement principal. Sert notamment à authentifier, valider ou refuser une requête.
MLD (Modèle Logique de Données)	Deuxième niveau Merise : traduction du modèle conceptuel en tables, colonnes et clés, sans encore dépendre d'un moteur précis.
MPD (Modèle Physique de Données)	Troisième niveau Merise : le schéma tel qu'il est réellement créé dans le moteur, avec ses types, index et contraintes.
Normalisation / 3NF	Démarche d'organisation des données visant à éliminer les redondances. La troisième forme normale (3NF) exige que chaque colonne dépende de la clé primaire, et d'elle seule.
Optimistic UI	Technique consistant à afficher immédiatement le résultat probable d'une action, sans attendre la réponse du serveur, pour supprimer la latence perçue.
ORM (Object-Relational Mapping)	Couche logicielle traduisant les tables d'une base relationnelle en objets du langage de programmation, et inversement.
OWASP	Organisation à but non lucratif de référence en sécurité applicative, connue notamment pour son Top 10 des risques les plus critiques.
Prisma	ORM pour Node.js qui génère un client typé à partir d'un schéma déclaratif, et gère les migrations de base de données.
Rate limiting	Limitation du nombre de requêtes acceptées d'une même source sur un intervalle donné. Protège notamment contre les tentatives de mot de passe en série.
Refresh token rotation	Mécanisme où chaque usage du jeton de renouvellement le remplace par un nouveau et invalide l'ancien, réduisant la fenêtre d'exploitation d'un jeton dérobé.
Reverse proxy	Serveur placé devant les applications, qui reçoit toutes les requêtes entrantes et les redirige vers le bon service. Assure ici le chiffrement TLS et le routage.
Room (Socket.IO)	Groupe nommé de connexions temps réel. Émettre vers une room n'atteint que les clients qui l'ont rejointe, ce qui cloisonne les échanges.

sameSite	Attribut de cookie déterminant s'il accompagne les requêtes venant d'un autre site. La valeur <code>strict</code> l'interdit, ce qui limite les attaques par requête falsifiée.
Scrum	Cadre de travail agile organisant le développement en cycles courts, avec des rôles et des rituels définis.
Seed	Script peuplant une base vierge de données initiales, afin de disposer d'un environnement exploitable pour le développement ou la démonstration.
Server Component (Next.js)	Composant React rendu sur le serveur, dont le code n'est pas envoyé au navigateur.
Socket.IO	Bibliothèque de communication temps réel bidirectionnelle entre client et serveur, reposant sur WebSocket avec repli automatique si celui-ci est indisponible.
Sprint	Cycle de travail de durée fixe en Scrum, au terme duquel un incrément utilisable est livré. Ici, une semaine.
SSR (Server-Side Rendering)	Génération des pages HTML côté serveur avant envoi au navigateur. Améliore l'affichage initial et permet l'indexation par les moteurs de recherche.
User story	Formulation courte d'un besoin du point de vue de l'utilisateur, du type « je souhaite pouvoir ... afin de ... ».
WebSocket	Protocole maintenant une connexion ouverte en permanence entre client et serveur, permettant au serveur d'envoyer des données sans que le client ait à les demander.
XSS (Cross-Site Scripting)	Attaque consistant à faire exécuter un script malveillant dans le navigateur d'un autre utilisateur, typiquement pour lui dérober sa session.
Zod	Bibliothèque TypeScript de validation de données par schémas, qui vérifie les entrées et en déduit automatiquement les types.

16. Annexes

Les présentes annexes regroupent les éléments matériels demandés par le référentiel pour la **fonctionnalité la plus représentative** du projet — la messagerie temps réel. Elles permettent à l'évaluateur d'inspecter le code réel sans devoir alterner entre le dossier et le dépôt source.

16.1. Annexe A — Schéma Prisma de la fonctionnalité messagerie

Extrait de `backend/prisma/schema.prisma` couvrant les modèles directement impliqués dans la messagerie : `Conversation`, `UserHasConversation`, `Message`, `Follow` (gating), et les relations vers `User`.

```
model Conversation {
  id          Int          @id @default(autoincrement())
  status      StatusOfConversation @default(Open)
  title       String
  users       UserHasConversation[]
  messages    Message[]

  createdAt DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @default(now()) @updatedAt @map("updated_at")

  @@map("conversation")
}

enum StatusOfConversation {
  Open
  Close
}

model UserHasConversation {
  user      User          @relation(fields: [userId], references: [id],
onDelete: Cascade)
  userId    Int           @map("user_id")
  conversation Conversation @relation(fields: [conversationId], references:
[id], onDelete: Cascade)
  conversationId Int       @map("conversation_id")

  createdAt DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @default(now()) @updatedAt @map("updated_at")

  @@id([userId, conversationId])
  @@map("user_has_conversation")
}

model Message {
  id          Int          @id @default(autoincrement())
  sender      User          @relation("SenderUser", fields: [senderId],
references: [id], onDelete: Cascade)
  senderId    Int          @map("sender_id")
  receiver    User          @relation("ReceiverUser", fields: [receiverId],
references: [id], onDelete: Cascade)
  receiverId  Int          @map("receiver_id")
}
```

```

    content      String
    conversation Conversation @relation(fields: [conversationId], references:
[id], onDelete: Cascade)
    conversationId Int          @map("conversation_id")

    createdAt     DateTime @default(now()) @map("created_at")
    updatedAt     DateTime @default(now()) @updatedAt @map("updated_at")

    @@map("message")
  }

model Follow {
  id          Int      @id @default(autoincrement())
  followed    User      @relation("FollowedUser", fields: [followedId], references:
[id], onDelete: Cascade)
  followedId  Int      @map("followed_id")
  follower    User      @relation("FollowerUser", fields: [followerId], references:
[id], onDelete: Cascade)
  followerId  Int      @map("follower_id")

  createdAt    DateTime @default(now()) @map("created_at")
  updatedAt    DateTime @default(now()) @updatedAt @map("updated_at")

  @@unique([followedId, followerId])
  @@map("follow")
}

```

16.1.1. Extraits SQL générés par Prisma — migrations clés

Migration `init_db` (création de la table `user`) — extrait du fichier `backend/prisma/migrations/20260112133206_init_db/migration.sql` :

```

CREATE TABLE "user" (
  "id"          SERIAL NOT NULL,
  "firstname"   TEXT NOT NULL,
  "lastname"    TEXT NOT NULL,
  "email"       TEXT NOT NULL,
  "password"    TEXT NOT NULL,
  "address"     TEXT,
  "postal_code" INTEGER,
  "city"        TEXT,
  "age"         INTEGER,
  "avatarUrl"   TEXT,
  "description" TEXT,
  "role_id"     INTEGER NOT NULL,
  "created_at"  TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updated_at"  TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "user_pkey" PRIMARY KEY ("id")
);

CREATE UNIQUE INDEX "user_email_key" ON "user"("email");

ALTER TABLE "user"
  ADD CONSTRAINT "user_role_id_fkey"
  FOREIGN KEY ("role_id") REFERENCES "role"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;

```

Migration `add_unique_constraint` (matérialisation des contraintes d'unicité métier) :

```
CREATE UNIQUE INDEX "evaluation_evaluator_id_evaluated_id_key"
ON "evaluation"("evaluator_id", "evaluated_id");

CREATE UNIQUE INDEX "follow_followed_id_follower_id_key"
ON "follow"("followed_id", "follower_id");
```

16.2. Annexe B — Handler Socket.IO `message:send` (code complet)

Reproduction **verbatim** de `backend/src/realtime/socket.ts:167-347` (dépôt d'équipe, branche `main`) — handler complet de l'envoi d'un message : validation, vérification de participation, persistance Prisma en parallèle, et diffusion multi-rooms. Seule adaptation de mise en page : le bloc est désindenté de quatre espaces, le handler étant imbriqué dans `io.on('connection')` dans le fichier source. Commentaires d'origine conservés en anglais, tels qu'ils figurent dans le livrable.

```
socket.on('message:send', async (payload) => {
  const conversationId = Number(payload?.conversationId);
  const content = String(payload?.message ?? '').trim();

  // Validate input
  if (!Number.isInteger(conversationId) || conversationId <= 0) {
    socket.emit('error', {
      code: 'VALIDATION',
      message: 'Invalid conversationId',
    });
    return;
  }

  if (!content || content.length > 2000) {
    socket.emit('error', {
      code: 'VALIDATION',
      message: 'Invalid message',
    });
    return;
  }

  // Get conversation with participants and message count
  const conv = await prisma.conversation.findUnique({
    where: { id: conversationId },
    select: {
      id: true,
      title: true,
      status: true,
      users: {
        select: {
          userId: true,
          user: {
            select: {
              id: true,
              firstname: true,
              lastname: true,
              avatarUrl: true,
            },
          },
        },
      },
    },
  });
```

```

    },
    },
  },
  _count: {
    select: { messages: true },
  },
},
});

if (!conv || conv.status === 'Close') {
  socket.emit('error', {
    code: 'FORBIDDEN',
    message: 'Conversation closed',
  });
  return;
}

// Verify that the sender is a participant in the conversation
const userIds = conv.users.map((u) => u.userId);
if (!userIds.includes(userId)) {
  socket.emit('error', {
    code: 'FORBIDDEN',
    message: 'Not a participant',
  });
  return;
}

const participantIds = new Set(userIds);
const receiverId = userIds.find((id) => id !== userId);

if (!receiverId) {
  socket.emit('error', { code: 'FORBIDDEN', message: 'No receiver' });
  return;
}

// Check if this is the first message
const isFirstMessage = conv._count.messages === 0;

// Create the message AND update conversation's updatedAt
const [msg] = await Promise.all([
  prisma.message.create({
    data: {
      conversationId,
      senderId: userId,
      receiverId,
      content,
    },
  },
  select: {
    id: true,
    content: true,
    createdAt: true,
    sender: {
      select: {
        id: true,
        firstname: true,
        lastname: true,
        avatarUrl: true,

```



```

    },
  },
},
}),
prisma.conversation.update({
  where: { id: conversationId },
  data: { updatedAt: new Date() },
}),
]);

// Prepare DTO
const dto: MessageDTO = {
  id: msg.id,
  sender:
    msg.sender && participantIds.has(msg.sender.id)
      ? {
          id: msg.sender.id,
          firstname: msg.sender.firstname,
          lastname: msg.sender.lastname,
          avatarUrl: msg.sender.avatarUrl ?? undefined,
        }
      : undefined,
  content: msg.content,
  timestamp: msg.createdAt.toISOString(),
};

// Emit message to conversation room (for users actively viewing)
io.to(room(conversationId)).emit('message:new', {
  conversationId,
  message: dto,
});

// If first message, notify the receiver about the new conversation
if (isFirstMessage) {
  // Get follow/rating status for the receiver
  const [followStatus, ratingStatus] = await Promise.all([
    prisma.follow.findUnique({
      where: {
        followedId_followerId: {
          followerId: receiverId,
          followedId: userId,
        },
      },
    }),
    prisma.rating.findUnique({
      where: {
        evaluatorId_evaluatedId: {
          evaluatorId: receiverId,
          evaluatedId: userId,
        },
      },
    }),
  ]),
  ]);

  // Get sender info for the receiver's perspective
  const senderUser = conv.users.find((u) => u.userId === userId)?.user;

```

```

if (senderUser) {
  // Emit new conversation to receiver only
  io.to(`user:${receiverId}`).emit('conversation:new', {
    conversation: {
      id: conv.id,
      title: conv.title,
      status: conv.status as 'Open' | 'Close',
      participant: {
        id: senderUser.id,
        firstname: senderUser.firstname,
        lastname: senderUser.lastname,
        avatarUrl: senderUser.avatarUrl ?? undefined,
        isFollowing: !!followStatus,
        isRated: !!ratingStatus,
      },
      lastMessage: dto,
    },
  });
}

// Emit conversation update to all participants (via user rooms)
userIds.forEach((participantId) => {
  io.to(`user:${participantId}`).emit('conversation:updated', {
    conversationId,
    lastMessage: dto,
  });
});
});
});

```

16.3. Annexe C — Orchestrateur frontend `useMessaging` (139 LOC, complet)

Reproduction **verbatim** et intégrale de `frontend/src/hooks/useMessaging.ts` (dépôt d'équipe, branche `main`) — la façade qui compose les sept hooks spécialisés et écoute les trois events Socket globaux (`conversation:updated`, `conversation:closed`, `conversation:new`).

```

'use client';
import {
  useConversationList,
  useSelectedConversation,
  useConversationMessages,
  useConversationActions,
  useFollowedUsers,
  useGlobalSocket,
} from './messaging';
import { useMemo, useEffect } from 'react';
import { ConversationWithMessages } from '@lib/api-types';
import { useAuth } from '@components/providers/AuthProvider';
import { toast } from 'sonner';

export function useMessaging() {
  const { user } = useAuth();

```

```

const {
  conversations,
  isLoading: isConversationLoading,
  addConversation,
  updateConversation,
  updateConversationLastMessage,
  updateConversationStatus,
  removeConversation,
} = useConversationList();

const { selectedConvId, setSelectedConvId, selectedConv, clearSelection } =
  useSelectedConversation(conversations);

const {
  messages,
  isLoading: isLoadingMessages,
  hasMore: hasMoreMessages,
  loadMore: loadMoreMessages,
  addMessage,
  addOptimisticMessage,
} = useConversationMessages({
  conversationId: selectedConvId,
  limit: 30,
});

const { followedUsers, fetchFollowedUsers } = useFollowedUsers();

const { onConversationUpdate, onConversationClosed, onConversationNew } =
  useGlobalSocket();

// Listen for lastMessage updates
useEffect(() => {
  onConversationUpdate((conversationId, lastMessage) => {
    updateConversationLastMessage(conversationId, lastMessage);
  });
}, [onConversationUpdate, updateConversationLastMessage]);

// Listen for conversation closures
useEffect(() => {
  onConversationClosed((conversationId, closedBy) => {
    updateConversationStatus(conversationId, 'Close');

    if (closedBy && closedBy.id !== user?.id) {
      toast.info(`${closedBy.firstname} à clôturer un échange`);
    }

    if (conversationId === selectedConvId) {
      clearSelection();
    }
  });
}, [
  onConversationClosed,
  updateConversationStatus,
  selectedConvId,
  clearSelection,
  user,
]);

```

```

useEffect(() => {
  onConversationNew((conversation) => {
    // Add the new conversation to the list
    addConversation(conversation);

    // Show notification
    toast.info(
      `${conversation.participant?.firstname} a démarré un nouvel échange`,
    );
  });
}, [onConversationNew, addConversation]);

const selectedConvWithMessages: ConversationWithMessages | undefined =
  useMemo(() => {
    if (!selectedConv) return undefined;

    return {
      ...selectedConv,
      messages,
      hasMoreMessages,
      isLoadingMessages,
    };
  }, [selectedConv, messages, hasMoreMessages, isLoadingMessages]);

const {
  handleBack,
  handleViewProfile,
  handleNewConversation,
  handleAddConversation,
  handleRatingUser,
  handleEncloseConversation,
  handleDeleteConversation,
  handleSendMessage,
} = useConversationActions({
  selectedConvId,
  selectedConv: selectedConvWithMessages,
  setSelectedConvId,
  addConversation,
  updateConversation,
  removeConversation,
  clearSelection,
  fetchFollowedUsers,
  addMessage,
  addOptimisticMessage,
});

return {
  selectedConvId,
  setSelectedConvId,
  selectedConv: selectedConvWithMessages,
  conversations,
  followedUsers,
  isConversationLoading,
  loadMoreMessages,
  handleBack,
  handleNewConversation,

```

```

    handleAddConversation,
    handleSendMessage,
    handleViewProfile,
    handleRatingUser,
    handleEncloseConversation,
    handleDeleteConversation,
  };
}

```

16.4. Annexe D — Composants UI clés de la messagerie

16.4.1. `MessageList.tsx` — affichage virtualisé du fil de discussion

Reproduction **verbatim** et intégrale de `frontend/src/components/organisms/ConversationPage/MessageThread/MessageList.tsx` (dépôt d'équipe, branche `main`) — le composant qui rend la liste de messages, gère le spinner de chargement de la pagination cursor-based, l'empty state et la détection de doublons.

```

'use client';
import { RefObject, useEffect } from 'react';
import { MessageBubble } from '@components/molecules/MessageBubble';
import { EmptyState } from '@components/molecules/EmptyState';
import { Loader2 } from 'lucide-react';
import type { Message } from '@lib/api-types';

interface MessageListProps {
  messages: Message[] | undefined;
  currentUserId?: number;
  scrollRef: RefObject<HTMLDivElement | null>;
  isLoading?: boolean;
  hasMore?: boolean;
}

export function MessageList({
  messages,
  currentUserId,
  scrollRef,
  isLoading = false,
  hasMore = false,
}: MessageListProps) {
  useEffect(() => {
    if (messages) {
      const ids = messages.map((m) => m.id);
      const duplicates = ids.filter((id, index) => ids.indexOf(id) !== index);

      if (duplicates.length > 0) {
        console.error('Duplicate message IDs detected:', duplicates);
        console.error('All messages:', messages);
      }
    }
  }, [messages]);

  return (
    <div

```

```

ref={scrollRef}
className="flex-1 overflow-y-auto p-6"
style={{ overflowAnchor: 'none' }} // Important pour éviter le scroll jump
>
<div className="flex flex-col">
  {/* Indicateur de chargement en haut */}
  {isLoading && (
    <div className="flex justify-center py-4">
      <Loader2 className="h-6 w-6 animate-spin text-muted-foreground" />
    </div>
  )}

  {/* Indicateur de début de conversation */}
  {!hasMore && messages && messages.length > 0 && (
    <div className="flex justify-center py-4">
      <p className="text-sm text-muted-foreground">
        Début de la conversation
      </p>
    </div>
  )}

  {/* Messages */}
  {messages && messages.length > 0 ? (
    <div className="flex flex-col gap-4">
      {messages.map((m) => (
        <MessageBubble
          key={m.id}
          content={m.content}
          timestamp={m.timestamp}
          isOwn={m.sender?.id === currentUserid}
          sender={m.sender}
        />
      ))}
    </div>
  ) : (
    !isLoading && (
      <EmptyState
        title="Aucun message"
        description="Commencez la conversation"
      />
    )
  )}
</div>
</div>
);
}

```

16.4.2. MessageInput.tsx — saisie utilisateur et envoi

Reproduction **verbatim** et **intégrale** de
 frontend/src/components/organisms/ConversationPage/MessageThread/MessageInput.tsx
 (dépôt d'équipe, branche `main`) — l'input désactivé automatiquement quand la conversation est clôturée (cohérent avec le refus serveur dans le handler `message: send`).

```
'use client';
```

```

import { Button } from '@components/atoms/Button';
import { Input } from '@components/atoms/Input';
import { Send } from 'lucide-react';
import { FilterStatus } from '../useConversationState';

interface MessageInputProps {
  value: string;
  onChange: (value: string) => void;
  onSend: () => void;
  isLoading: boolean;
  conversationStatus: FilterStatus;
}

/**
 * Zone de saisie et envoi de message
 */
export function MessageInput({
  value,
  onChange,
  onSend,
  isLoading,
  conversationStatus,
}: MessageInputProps) {
  const handleKeyDown = (e: React.KeyboardEvent) => {
    if (e.key === 'Enter') {
      onSend();
    }
  };

  return (
    <div className="border-t bg-background p-4">
      <div className="flex gap-3">
        <Input
          placeholder="Écrivez un message..."
          value={value}
          onChange={(e) => onChange(e.target.value)}
          onKeyDown={handleKeyDown}
          aria-label="Saisir un message"
          disabled={conversationStatus === 'Close'}
        />
        <Button
          onClick={onSend}
          disabled={!value.trim() || conversationStatus === 'Close'}
          className="flex items-center gap-2"
          isLoading={isLoading}
        >
          <Send className="h-4 w-4" />
          <span>Envoyer</span>
        </Button>
      </div>
    </div>
  );
}

```

16.5. Annexe E — Plan de tests détaillé de la messagerie

Le plan de tests est détaillé en section 10 ; les jeux d'essai de la fonctionnalité représentative figurent en section 11.

16.6. Annexe F — Maquettes Figma des écrans principaux

Maquettes produites sous Figma pendant le sprint 0, le 8 janvier 2026, avant toute implémentation. Archivées sur le Drive d'équipe, elles ont été versées au dépôt d'équipe le 9 mai 2026 sur la branche `presentation-seb`, jamais mergée sur `main`. Elles sont reproduites à l'échelle de l'écran conçu : elles documentent la composition, la hiérarchie visuelle et le parcours retenus avant développement, non le détail des libellés.

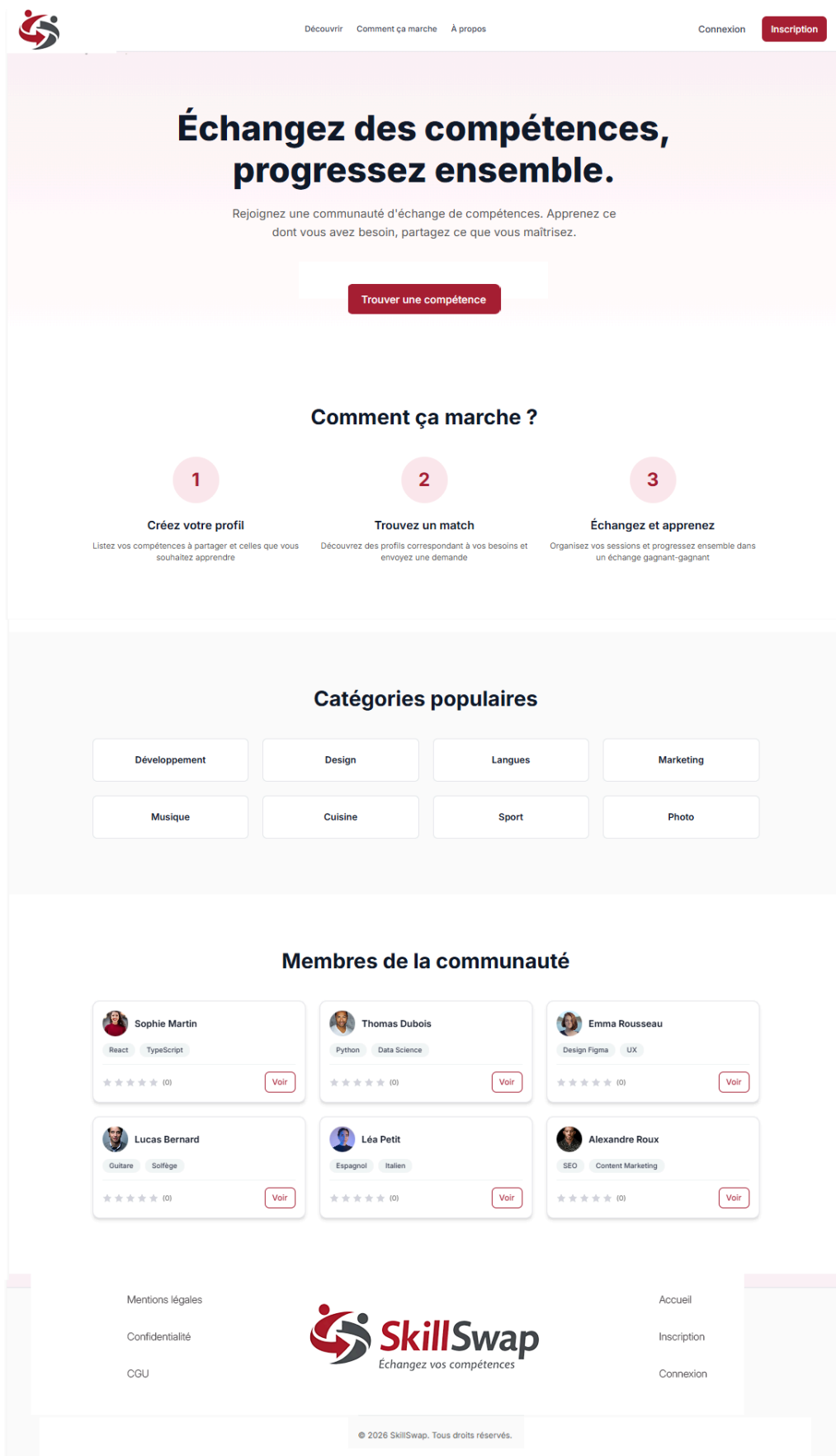


Fig. 28. – Maquette Figma — page d'accueil.

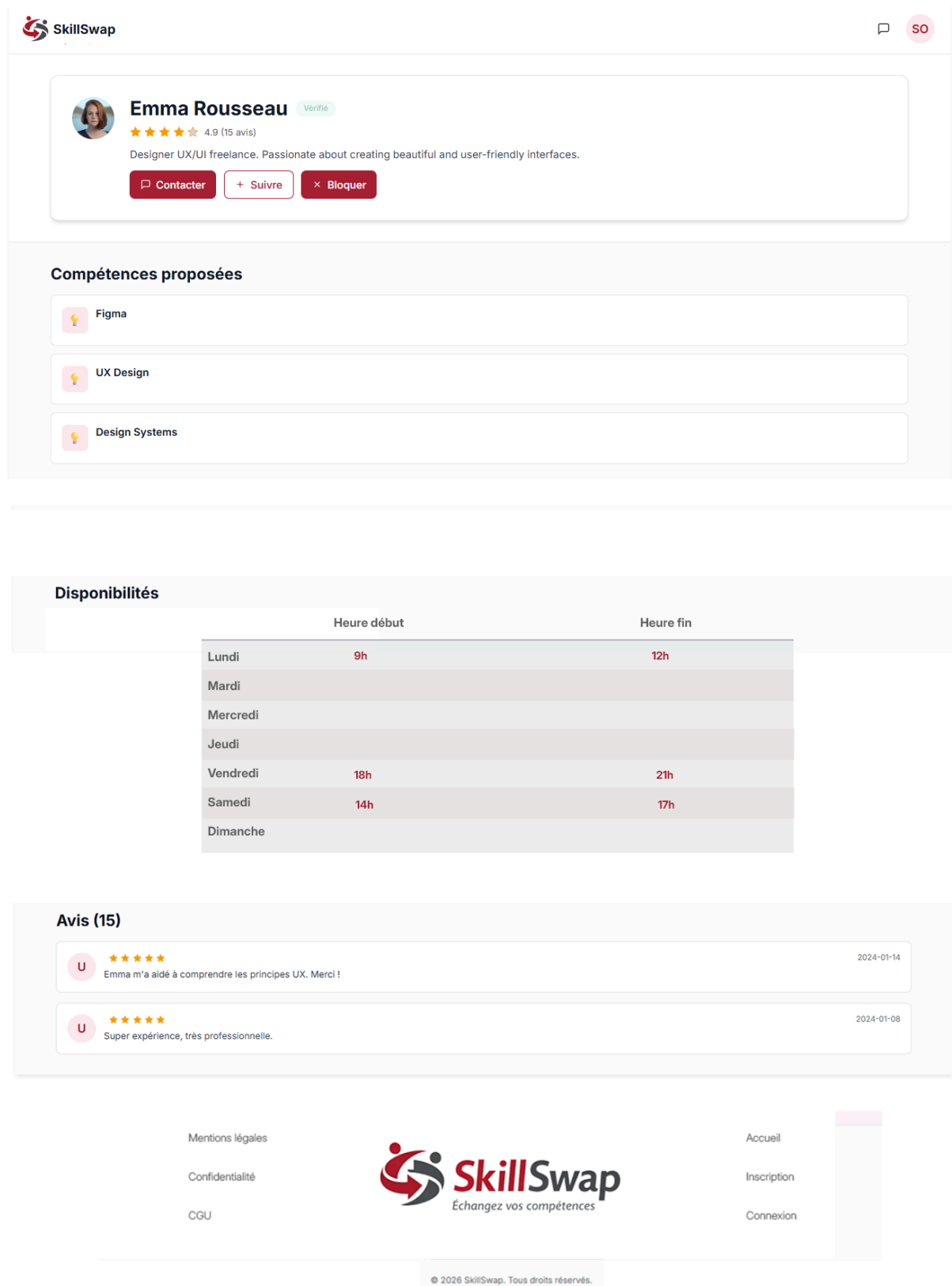


Fig. 29. – Maquette Figma — profil public d'un membre.

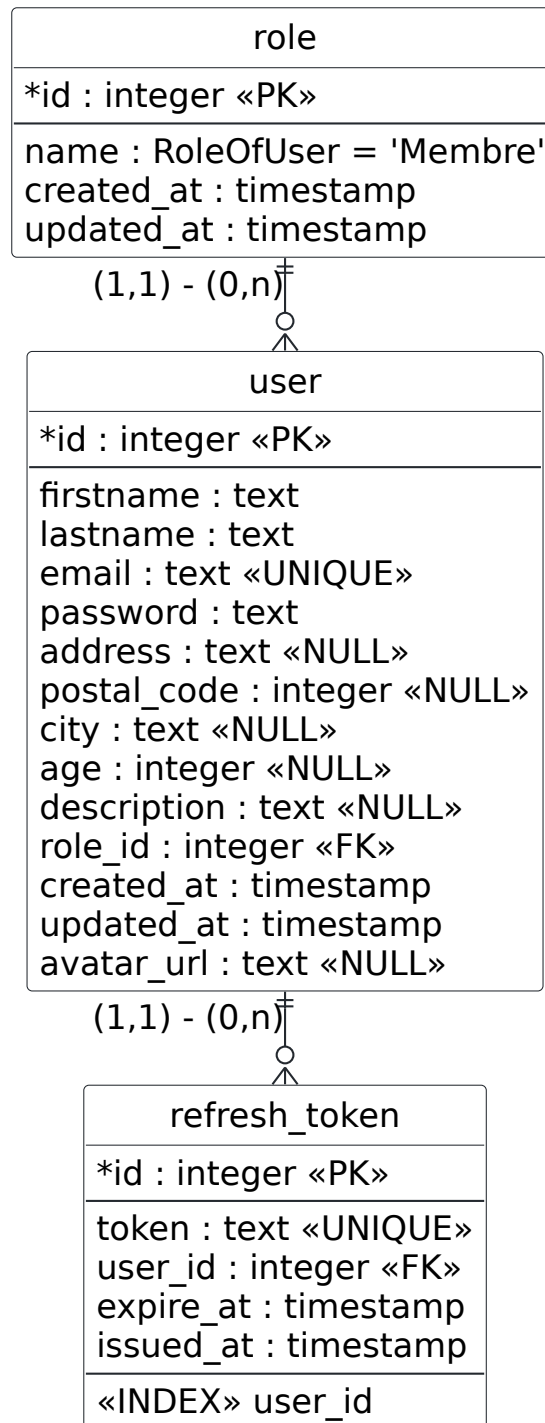
Fig. 30. – Maquette Figma — page de recherche.

Fig. 31. – Maquette Figma — messagerie.

Les cinq figures ci-dessous détaillent, colonnes complètes, les cinq domaines fonctionnels présentés en section 6. Elles sont dérivées du catalogue PostgreSQL de la base

montée depuis les six migrations réelles, et non du code applicatif ; un contrôle automatisé a vérifié la concordance de **quatorze tables, soixante-dix-huit colonnes et dix-huit clés étrangères** — nom, type, nullabilité et ordre des colonnes. La table `user`, commune à quatre domaines, n'est détaillée qu'une fois, en Fig. 32 ; elle figure ailleurs en encart grisé `«externe»` réduit à sa clé primaire.

ERD SkillSwap — domaine Identité



Types = catalogue PostgreSQL.
 timestamp = timestamp(3) without time zone.
 NOT NULL sauf «NULL». FK : ON DELETE/UPDATE CASCADE.
 Ordre des colonnes = pg_attribute.attnum.

Fig. 32. – Domaine Identité — `role`, `user` et `refresh_token`.

ERD SkillSwap — domaine Compétences

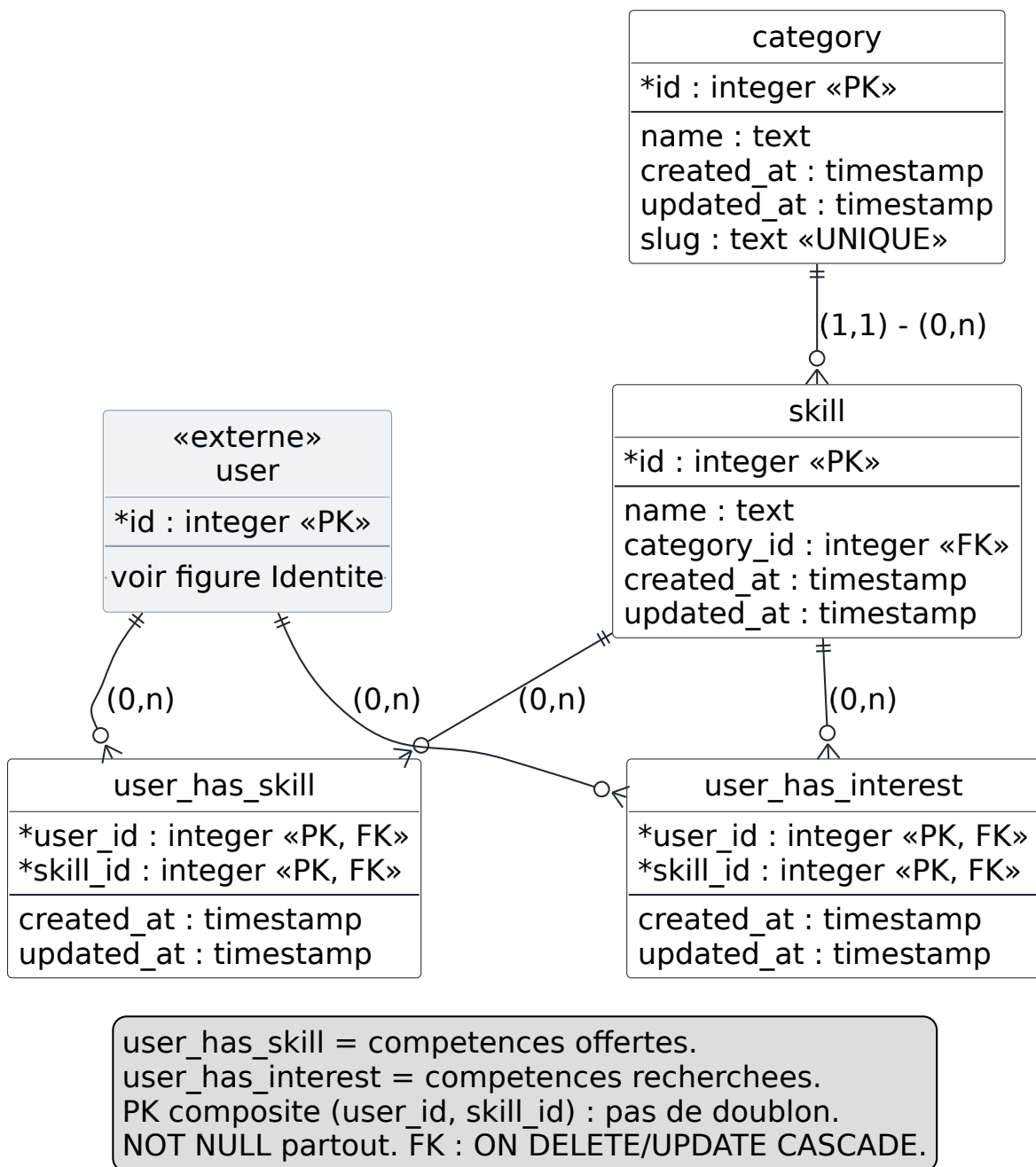
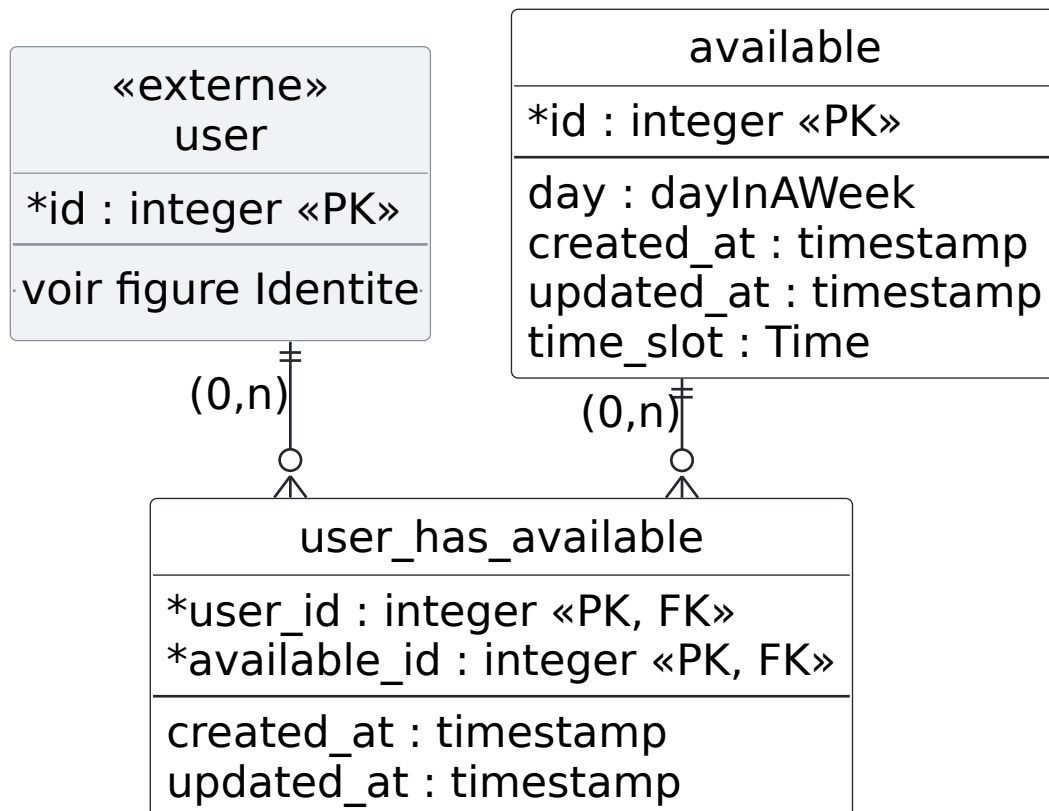


Fig. 33. – Domaine Compétences — `category`, `skill`, `user_has_skill` et `user_has_interest`.

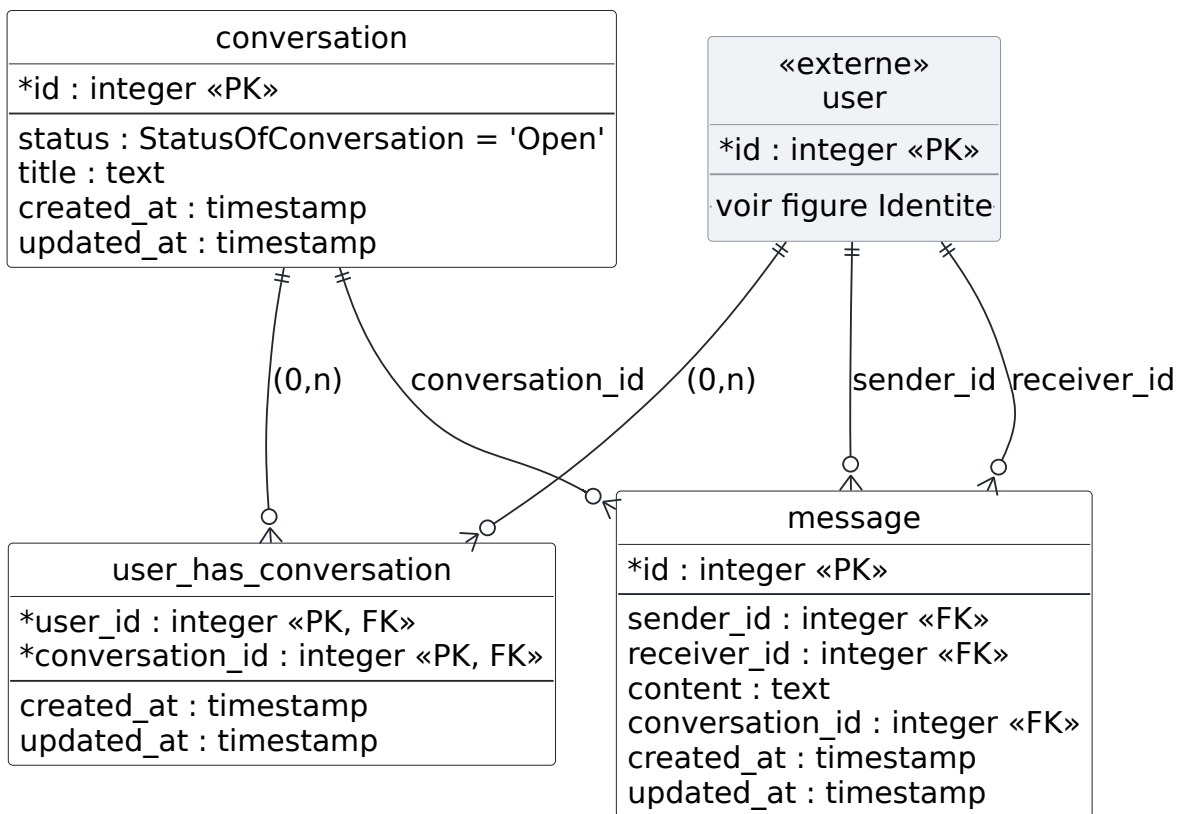
ERD SkillSwap — domaine Disponibilités



dayInAWeek : Lundi..Dimanche (7 valeurs).
 Time : Morning, Afternoon.
 available est un referentiel de creneaux partage.
 Depuis la migration 20260116161218, available ne porte plus de user_id : le lien passe par la table de jonction user_has_available.

Fig. 34. – Domaine Disponibilités — `available` et `user_has_available`.

ERD SkillSwap — domaine Échange



StatusOfConversation : Open, Close.
 message porte 3 FK dont 2 vers user : le destinataire est stocké sur chaque message, en plus de l'appartenance à la conversation via user_has_conversation.
 NOT NULL partout. FK : ON DELETE/UPDATE CASCADE.

Fig. 35. – Domaine Échange — conversation, user_has_conversation et message.

ERD SkillSwap — domaine Social

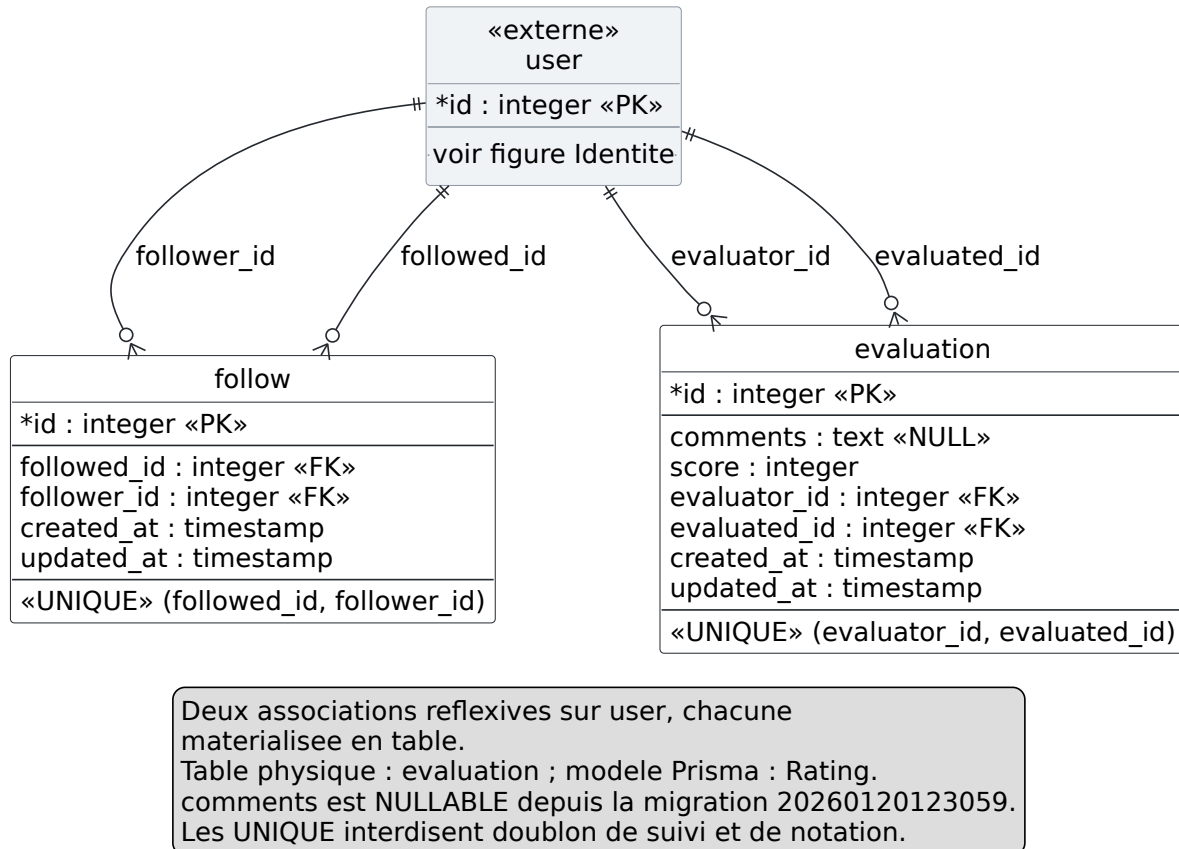


Fig. 36. – Domaine Social — `follow` et `evaluation`, nom physique du modèle Prisma `Rating`.

16.8. Liens utiles

- **Application en production** : skill-swap.fr

La documentation Arc42 et le guide utilisateur Diátaxis publiés ci-dessous constituent une **extension post-projet, hors périmètre du livrable d'équipe** :

- **Documentation technique Arc42** : skillswap-docs.vercel.app
- **Guide utilisateur (Diátaxis)** : skillswap-guide.vercel.app
- **Repo GitHub doc** : [Sojeremy/documentation-skillswap](https://github.com/Sojeremy/documentation-skillswap)

16.9. Annexe H — Chemin d'exécution de `message:send`

L'annexe B reproduit le code du handler serveur ; la présente annexe en donne la contre-partie graphique, et l'étend au chemin complet — de la frappe de l'utilisateur jusqu'à l'écriture en base et aux trois diffusions. Chaque nœud porte son fichier et ses lignes, chaque arête l'appel exact. Aucun élément n'y figure qui ne soit pas dans le code du livrable d'équipe.

Le chemin traverse **dix étages successifs**. Rendu d'un seul tenant, le diagramme mesurerait 213 × 517 mm — plus de deux pages A4 — pour rester lisible. Il est donc découpé en trois vues, avec coutures nommées : la vue 1/3 se raccorde à la 2/3 par `useMessaging.ts`, la 2/3 à la 3/3 par `socket-client.ts`.

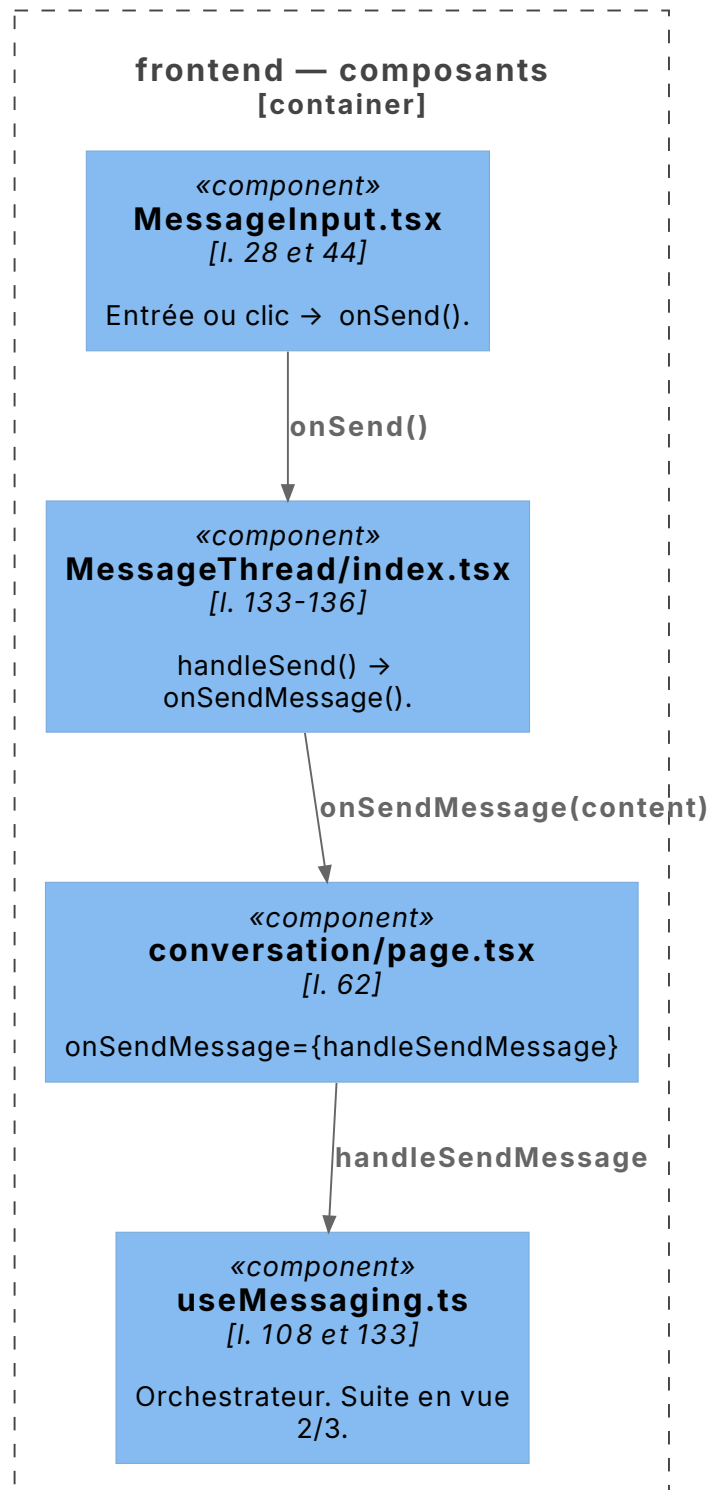
`message:send` (C4 niveau 4, vue 1/3) — de la saisie au hook orchestrateur

Fig. 37. – **Vue 1/3 — de la saisie au hook orchestrateur.** Quatre fichiers, de l'événement d'interface jusqu'à la façade `useMessaging`. Couture avec la vue suivante : `useMessaging.ts`.

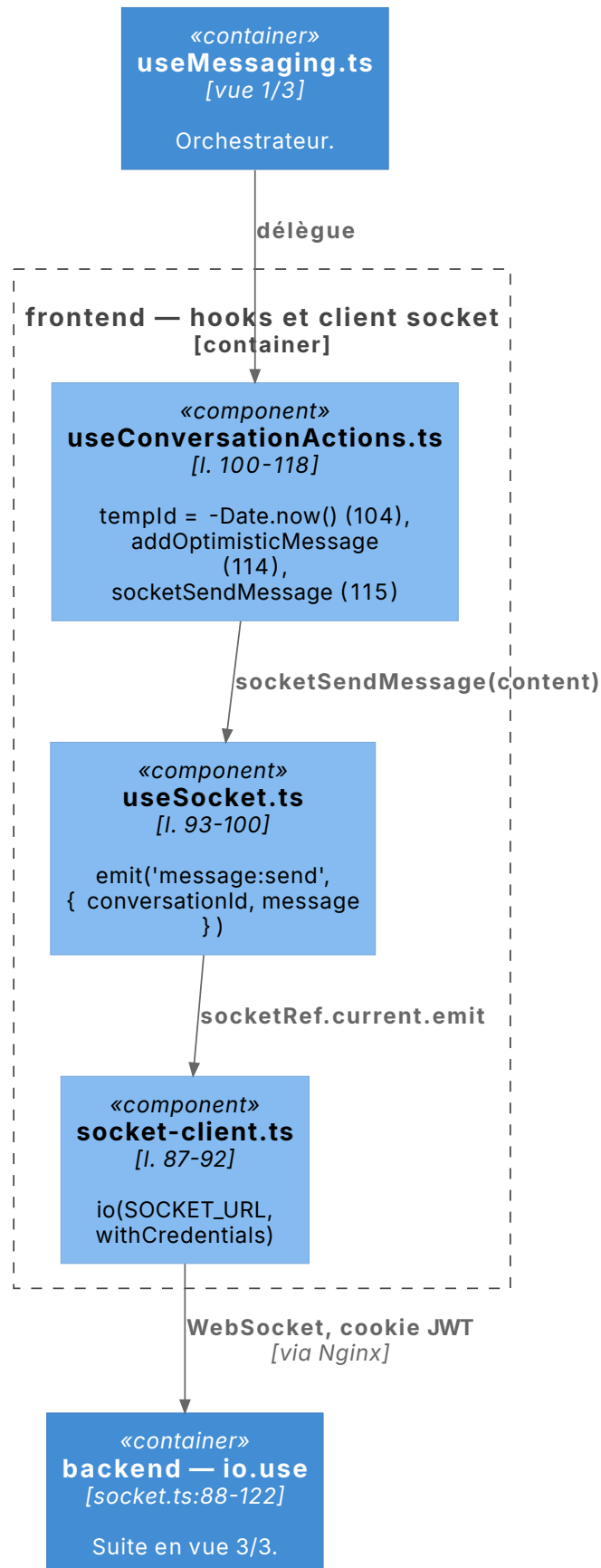
`message:send` (C4 niveau 4, vue 2/3) — du hook à l'émission

Fig. 38. – **Vue 2/3 — du hook à l'émission Socket.IO.** Le message optimiste est posé avec un `tempId` négatif avant l'émission. Couture suivante : `socket-client.ts`.

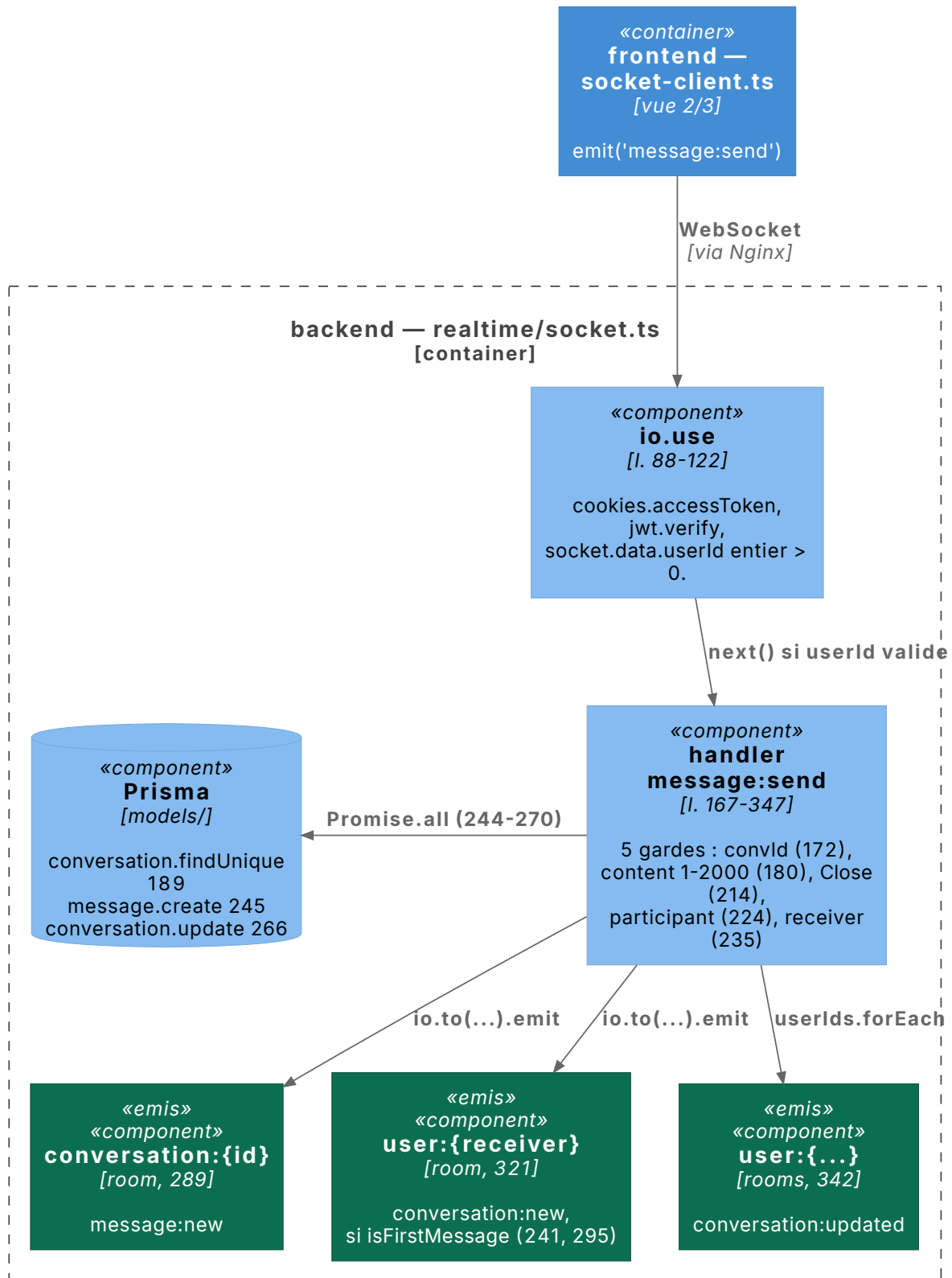
`message:send` (C4 niveau 4, vue 3/3) — serveur

Fig. 39. – **Vue 3/3 — handshake, persistance et diffusions.** Le handler, reproduit verbatim en Chapitre 16.2, applique cinq gardes successives puis deux `Promise.all`. En vert, les trois émissions : `message:new` vers la room de conversation, `conversation:new` vers le seul destinataire et uniquement au premier message, `conversation:updated` vers les rooms personnelles des deux participants.